SteadyScreen: source code

UnivaBio 2026. Team: Aditya Raut, Neeraj Movva, Sagar Raut, Sathvik Loke.

SteadyScreen is a browser app (plain HTML, CSS and JavaScript ES modules, no framework or build step). The pose model is Google's MediaPipe Pose Landmarker (`@mediapipe/tasks-vision` 1.0.1, Apache-2.0). It is downloaded and checked against fixed SHA-256 hashes by `scripts/setup_vendor.sh` and is not included here, since we did not write it.

How the code is organised:

- `app/src/core/`: the measurement logic. It tracks the participant, measures hip height and knee and hip angles, compares each rise with the confirmed practice stand, applies the CDC 30-second rule (`chair_stand.js`), checks which review decisions have images behind them (`coverage.js`), and decides what can be reported (`decision.js`). The same code runs in the browser and in the tests.
- `app/src/ui/`: the screens, camera and frame loop, temporary evidence images, and the printable summary.
- `tests/`: automated tests run with `npm test` (Node's built-in test runner). `eval/synth.mjs` generates synthetic body poses for those tests.

To run it: `./scripts/setup_vendor.sh`, then `python3 -m http.server 8932 --bind 127.0.0.1 --directory app`, then open http://127.0.0.1:8932/ in Chrome or Safari.

Files in this document:

- `app/index.html` (220 lines)
- `app/styles.css` (109 lines)
- `app/src/core/chair_stand.js` (629 lines)
- `app/src/core/coverage.js` (64 lines)
- `app/src/core/decision.js` (218 lines)
- `app/src/core/norms.js` (29 lines)
- `app/src/core/params.js` (102 lines)
- `app/src/core/pose_features.js` (112 lines)
- `app/src/core/tracker.js` (58 lines)
- `app/src/ui/app.js` (784 lines)
- `app/src/ui/audio.js` (11 lines)
- `app/src/ui/demo.js` (186 lines)
- `app/src/ui/draw.js` (63 lines)
- `app/src/ui/engine.js` (114 lines)
- `app/src/ui/evidence.js` (97 lines)
- `app/src/ui/report.js` (106 lines)
- `tests/core.test.mjs` (82 lines)
- `tests/decision_matrix.test.mjs` (76 lines)
- `tests/demo.test.mjs` (210 lines)
- `tests/regression_cp3.test.mjs` (234 lines)
- `tests/regression_cp4.test.mjs` (277 lines)
- `tests/regression_overnight.test.mjs` (135 lines)
- `tests/regression_overnight_core.test.mjs` (355 lines)
- `tests/test_strengthening_overnight.test.mjs` (227 lines)
- `eval/synth.mjs` (171 lines)
- `scripts/setup_vendor.sh` (25 lines)

app/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<!-- Resources may only load from this origin (plus blob:/data: for the camera stream, local files and images).
     The vendored MediaPipe library tries to send usage logs to Google; this policy blocks those requests. -->
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' 'wasm-unsafe-eval'; worker-src 'self' blob:; connect-src 'self' blob: data:; img-src 'self' blob: data:; media-src 'self' blob: mediastream:; style-src 'self'; object-src 'none'; base-uri 'none'; form-action 'none'">
<title>SteadyScreen — 30-Second Chair Stand assistant (prototype)</title>
<link rel="stylesheet" href="styles.css">
<link rel="icon" href="data:,">
</head>
<body>
<header class="topbar">
  <div><strong>SteadyScreen</strong> <span class="muted">30-Second Chair Stand assistant · prototype · not a medical device</span></div>
  <div id="modeBadge" class="badge hidden" aria-live="polite"></div>
</header>

<main>
<!-- Shown on every screen of a demo session (camera check, test, review, summary, print) -->
<p id="demo-banner" class="demobanner hidden" role="note"><strong>SIMULATED DEMO — not a real person, not a real measurement.</strong> The participant, the picture and the body points are generated by the app; no camera and no pose model are used.</p>
<!-- 1. SETUP -->
<section id="scr-setup" class="screen" aria-labelledby="h-setup">
  <h1 id="h-setup">Before you start</h1>
  <p class="lead">For a supervising adult running the CDC STEADI <em>30-Second Chair Stand</em>. The app gives the “Go”, times 30 seconds and counts full stands from the camera, so you can watch the participant. Afterwards you check each counted or doubtful rise against still images. It flags many situations where it could not measure reliably — not all of them — so every count must be checked.</p>
  <p class="small"><strong>What it does not do:</strong> it does not diagnose or predict falls, and it shows the CDC comparison table only after the count has been checked.</p>
  <div class="card demoquick"><p><strong>Just looking, with no chair or helper?</strong> Try the whole test with a computer-drawn <strong>simulated participant</strong>, about 2 minutes. It is labelled <strong>SIMULATED DEMO</strong> everywhere and measures no one. This button ticks the boxes below for the simulated scene and keeps still images so you can try the checks.</p>
    <button id="btn-demo-quick" class="big primary" type="button">Try the demo</button></div>
  <p id="support-msg" class="warn" role="alert"></p>
  <p id="setup-msg" class="warn" role="alert"></p>
  <div class="grid2">
    <fieldset class="card">
      <legend>Safety and set-up (you confirm these)</legend>
      <label class="check"><input type="checkbox" id="c-helper"><span>A helper will stand <strong>next to</strong> the participant for the whole test (CDC: “Stand next to the patient for safety”).</span></label>
      <label class="check"><input type="checkbox" id="c-chair"><span>The chair has a straight back, <strong>no arm rests</strong>, about 17 in (43 cm) seat height, and cannot slide.</span></label>
      <label class="check"><input type="checkbox" id="c-okay"><span>The participant agrees, is comfortable standing up repeatedly, and knows to <strong>stop at any discomfort</strong>.</span></label>
      <label class="check"><input type="checkbox" id="c-camera"><span>The camera sees the participant from the <strong>side</strong>, head to feet. The helper stays beside them within reach and should not pass between the camera and the participant unless needed for safety.</span></label>
    </fieldset>
    <fieldset class="card">
      <legend>Needed for the CDC comparison table</legend>
      <p class="small">Only used to look up the CDC table (ages 60–94). Never estimated from video. Whole numbers only. You can also enter or correct these on the review screen.</p>
      <label>Age <input type="text" id="in-age" inputmode="numeric" maxlength="3" autocomplete="off"></label>
      <label>Table column
        <select id="in-sex"><option value="">not entered</option><option value="men">men</option><option value="women">women</option></select>
      </label>
      <p id="age-msg" class="small warn"></p>
    </fieldset>
  </div>
  <fieldset class="card">
    <legend>Evidence images for checking afterwards</legend>
    <label class="check"><input type="checkbox" id="c-evidence"><span><strong>Keep temporary still images (needed for the app to report a camera count)</strong> of the practice stand and the test (about 6 per second) so I can check each rise afterwards. They stay inside this browser tab, are <strong>not uploaded</strong>, not saved to files and not in the downloaded record, and are released at New test, Back or when the tab closes.</span></label>
    <details class="small"><summary>Browser storage note</summary>The browser itself may hold image data in its own temporary storage, which the app cannot control.</details>
    <p class="small"><strong>Needed for a reported camera count:</strong> without images the app still times and counts, but nobody can check the count afterwards, so it is not reported or compared with the CDC table. (You can report your own count instead.)</p>
  </fieldset>
  <fieldset class="card">
    <legend>Sound</legend>
    <label class="check"><input type="checkbox" id="c-quiet"><span><strong>Quiet counting:</strong> no tone for each counted stand (only the GO and END tones). Use this when a separate recording will be counted by someone else, so the recording does not reveal the app's count.</span></label>
  </fieldset>
  <fieldset class="card">
    <legend>Choose a video source</legend>
```

```
    <div class="grid2">
      <div>
        <label class="check"><input type="checkbox" id="c-consent"><span>The participant agreed to be on camera. I understand the video is analysed <strong>in this browser tab only</strong> and is <strong>not uploaded</strong>.</span></label>
        <button id="btn-camera" class="big primary" disabled>Use camera (live test)</button>
        <p id="camera-hint" class="small">Tick the 4 safety boxes and the camera-consent box above to enable.</p>
      </div>
      <div>
        <p class="small">Practice / demonstration: analyse a video file already on this computer. Results are labelled <strong>REPLAY</strong> and are not a live test.</p>
        <label class="filebtn big" id="lbl-file">Replay a video file…<input type="file" id="in-file" accept="video/*"></label>
      </div>
    </div>
    <div class="demo-row">
      <p class="small"><strong>No camera, chair or helper at hand?</strong> Try the whole workflow with a computer-drawn <strong>simulated participant</strong>. Everything in it is labelled <strong>SIMULATED DEMO</strong>: it is not a real person and not a real measurement. Nothing is filmed. In the demo, the 4 safety boxes above describe the simulated scene; tick them as you would for a real test. Tick “Keep temporary still images” as well to try the full check afterwards.</p>
      <button id="btn-demo" class="big" disabled>Try a demo with a simulated participant (no camera needed)</button>
      <p id="demo-hint" class="small">Tick the 4 safety boxes above to enable the demo.</p>
    </div>
  </fieldset>
</section>

<!-- 2. CAMERA CHECK + PRACTICE STAND -->
<section id="scr-ready" class="screen hidden" aria-labelledby="h-ready">
  <h1 id="h-ready">Camera check and practice stand</h1>
  <div class="stage">
    <div id="slot-ready"><div class="videowrap" id="videowrap"><video id="video" playsinline muted></video><canvas id="overlay" aria-label="Camera view; click on the participant to select them"></canvas></div></div>
    <div class="side">
      <div id="ready-status" class="status" aria-live="polite">Loading the pose model…</div>
      <ol class="steps">
        <li id="step-instr"><strong>Read the CDC instructions to the participant:</strong> sit in the middle of the chair, hands on the opposite shoulders crossed at the wrists, feet flat on the floor, back straight; on “Go”, rise to a full standing position, then sit back down; repeat for 30 seconds.</li>
        <li id="step-seen">Participant fully visible (outlined in <span class="swatch green"></span> green)</li>
        <li id="step-practice"><strong>Practice stand:</strong> stand up fully with arms crossed, hold still for about 1 second, sit down.</li>
        <li id="step-still">Sit still, arms crossed, until “GO”.</li>
      </ol>
      <ul id="ready-reasons" class="reasons"></ul>
      <p class="small">More than one person seated? <strong>Click on the participant</strong> in the picture to select them.</p>
      <div id="ref-card" class="card hidden" aria-live="polite">
        <h2>Was that a full stand?</h2>
        <div id="ref-img" class="strip"></div>
        <p id="ref-warn" class="warn"></p>
        <p>Standing straight up the way this person normally stands, knees and hips straightened.</p>
        <button id="btn-ref-yes" class="big primary">Yes, full stand</button>
        <button id="btn-ref-no" class="big">No — repeat the practice stand</button>
      </div>
      <button id="btn-start" class="big primary" disabled>Start 30-second test <kbd>Space</kbd></button>
      <button id="btn-noref" class="big">Count without a practice stand (every rise will need your check)</button>
      <button id="btn-back" class="big">Back</button>
      <p id="engine-info" class="small muted"></p>
    </div>
  </div>
</section>

<!-- 3. TEST -->
<section id="scr-test" class="screen hidden" aria-labelledby="h-test">
  <h1 id="h-test" class="sr-only">Test running</h1>
  <div class="stage">
    <div id="slot-test"></div>
    <div class="side">
      <div class="bignum" id="timer" aria-live="off">30</div>
      <div class="label" id="timer-label">seconds left</div>
      <div class="bignum count" id="count" aria-live="polite">0</div>
      <div class="label">full stands counted <span id="count-uncertain" class="small"></span></div>
      <div id="test-status" class="status" aria-live="polite"></div>
      <button id="btn-stop" class="big danger">STOP test <kbd>Esc</kbd></button>
      <p class="small"><strong>STOP at once</strong> for any safety concern, or if the participant must use their arms to stand (CDC: record 0 — answer “Yes” to the arm question afterwards). Stopping for any other reason gives no score.</p>
    </div>
  </div>
</section>

<!-- 4. REVIEW -->
<section id="scr-review" class="screen hidden" aria-labelledby="h-review">
  <h1 id="h-review">Check the result</h1>
  <div id="result-banner" class="banner" role="status"></div>
  <div id="todo" class="card todo" aria-live="polite"></div>
  <div class="card" id="reference-card">
    <h2>1. The practice stand used as the full-standing reference</h2>
    <div id="reference-body"></div>
  </div>
  <div class="card" id="identity-card">
```

```
    <h2>2. Was the right person measured?</h2>
    <div id="identity-body"></div>
  </div>
  <div class="card">
    <h2>3. Each rise</h2>
    <p class="small" id="rises-help"></p>
    <div id="rises"></div>
  </div>
  <div class="card" id="scan-card">
    <h2>4. Did the camera miss any stand?</h2>
    <div id="scan-body"></div>
  </div>
  <div class="card" id="flags-card">
    <h2>5. Other things to check</h2>
    <div id="flags"></div>
  </div>
  <div class="grid2">
    <fieldset class="card">
      <legend>6. Protocol questions (observer answers)</legend>
      <p id="q-arms">Did the participant <strong>use their arms</strong> to stand up?</p>
      <div class="radios" role="radiogroup" aria-labelledby="q-arms">
        <label><input type="radio" name="arms" value="no"> No</label>
        <label><input type="radio" name="arms" value="yes"> Yes</label>
        <label><input type="radio" name="arms" value="unsure"> Not sure</label>
      </div>
      <p id="arms-hint" class="small"></p>
      <p id="q-setup">Were the chair and set-up as described (straight back, no arm rests, seat about 17 in / 43 cm, helper besid
e) and the CDC instructions given (arms crossed at the wrists on the chest, feet flat)?</p>
      <div class="radios" role="radiogroup" aria-labelledby="q-setup">
        <label><input type="radio" name="setup" value="yes"> Yes</label>
        <label><input type="radio" name="setup" value="no"> No</label>
      </div>
      <p id="q-age">For the CDC table (observer-entered, never estimated):</p>
      <label>Age <input type="text" id="rv-age" inputmode="numeric" maxlength="3" autocomplete="off" aria-describedby="q-age"></l
abel>
      <label>Table column
        <select id="rv-sex" aria-describedby="q-age"><option value="">not entered</option><option value="men">men</option><option
value="women">women</option></select>
      </label>
    </fieldset>
    <fieldset class="card" id="manual-card">
      <legend>7. Your own count (optional, kept separately)</legend>
      <p class="small">Only if you counted full stands yourself <strong>during</strong> this completed 30-second test. It never r
eplaces the camera count unless you choose it below.</p>
      <label>My count <input type="text" id="in-manual" inputmode="numeric" maxlength="2" autocomplete="off"></label>
      <label>How I counted (required) <input type="text" id="in-manual-reason" maxlength="200" placeholder="e.g. counted silently
while spotting"></label>
      <button id="btn-manual" class="big">Save my count</button>
      <p id="manual-msg" class="small warn" role="alert"></p>
      <div id="basis-body"></div>
    </fieldset>
  </div>
  <div class="card">
    <h2>What will be reported</h2>
    <div id="decision"></div>
    <h3>Change record</h3>
    <ul id="audit" class="audit"></ul>
    <details><summary>Measurement details</summary><canvas id="timeline" height="230" aria-label="Timeline of hip height with ris
es"></canvas><pre id="quality" class="small"></pre></details>
  </div>
  <div class="actions">
    <button id="btn-report" class="big primary">Show summary</button>
    <button id="btn-again" class="big">New test</button>
  </div>
</section>

<!-- 5. REPORT -->
<section id="scr-report" class="screen hidden" aria-labelledby="h-report-title">
  <div id="report"></div>
  <div class="actions noprint">
    <button id="btn-print" class="big primary">Print / save as PDF</button>
    <button id="btn-json" class="big">Download record (JSON, no images)</button>
    <button id="btn-review" class="big">Back to review</button>
    <button id="btn-new" class="big">New test</button>
  </div>
</section>
</main>
<!-- Enlarged evidence image (opened from any image's Enlarge button; the same in-tab image, nothing is downloaded) -->
<dialog id="zoom" class="zoom" aria-labelledby="zoom-cap">
  <div class="zoombody">
    <div class="zoomfig" id="zoom-fig"><img id="zoom-img" alt=""></div>
    <p id="zoom-cap" class="zoomcap"></p>
    <p id="zoom-when" class="small"></p>
    <button id="zoom-close" type="button" class="big">Close <kbd>Esc</kbd></button>
  </div>
</dialog>
<script type="module" src="src/ui/app.js"></script>
```

```
</body>
</html>
```

app/styles.css

```
:root { --fg:#111; --muted:#555; --bg:#fafaf7; --card:#fff; --line:#d7d7d0; --green:#127a3a; --orange:#b85c00; --red:#b3261e; --blue:#1d4ed8; --focus:#ffbf00; }
* { box-sizing: border-box; }
body { margin: 0; font: 20px/1.45 system-ui, -apple-system, "Segoe UI", sans-serif; color: var(--fg); background: var(--bg); }
.topbar { display:flex; justify-content:space-between; align-items:center; padding:10px 20px; border-bottom:1px solid var(--line); background:#fff; font-size:18px; }
main { max-width: 1200px; margin: 0 auto; padding: 16px 20px 40px; }
h1 { font-size: 32px; margin: 8px 0 12px; } h2 { font-size: 24px; margin: 4px 0 8px; }
.lead { font-size: 22px; max-width: 60em; }
.muted { color: var(--muted); } .small { font-size: 17px; color: var(--muted); }
.hidden { display: none !important; }
.sr-only { position:absolute; width:1px; height:1px; overflow:hidden; clip:rect(0 0 0 0); }
.card { background: var(--card); border:1px solid var(--line); border-radius:10px; padding: 14px 18px; margin: 10px 0; }
fieldset.card legend { font-weight: 700; padding: 0 6px; }
.grid2 { display:grid; grid-template-columns: 1fr 1fr; gap: 16px; } @media (max-width: 900px) { .grid2 { grid-template-columns: 1fr; } }
label { display:block; margin: 8px 0; } label.check { display:flex; gap: 12px; align-items:flex-start; }
input[type=checkbox], input[type=radio] { width: 26px; height: 26px; flex: none; margin-top: 2px; }
input[type=number], input[type=text], select { font: inherit; padding: 8px 10px; border:2px solid var(--line); border-radius:8px; margin-left: 8px; }
input[type=text] { width: 100%; margin: 6px 0 0; }
button, .filebtn { font: inherit; font-weight: 600; border: 2px solid #333; background:#fff; color:#111; border-radius: 10px; padding: 12px 20px; cursor: pointer; margin: 6px 8px 6px 0; }
.big { min-height: 60px; font-size: 22px; }
.primary { background: var(--blue); border-color: var(--blue); color: #fff; }
.danger { background: var(--red); border-color: var(--red); color: #fff; width: 100%; min-height: 90px; font-size: 30px; }
button:disabled { opacity: .45; cursor: not-allowed; }
:focus-visible { outline: 4px solid var(--focus); outline-offset: 2px; }
.filebtn { display:inline-block; } .filebtn input { position:absolute; opacity:0; width:1px; height:1px; }
kbd { font-size: 15px; border:1px solid currentColor; border-radius:4px; padding: 0 5px; margin-left: 6px; }
.stage { display:grid; grid-template-columns: minmax(0, 3fr) minmax(280px, 2fr); gap: 18px; align-items:start; } @media (max-width: 900px) { .stage { grid-template-columns: 1fr; } }
.videowrap { position: relative; background:#000; border-radius: 10px; overflow:hidden; }
.videowrap video, .videowrap canvas { width: 100%; display:block; max-height: 70vh; object-fit: contain; }
.videowrap canvas { position:absolute; inset:0; height:100%; }
.status { font-size: 24px; font-weight: 700; padding: 10px 14px; border-radius: 10px; background:#eee; margin-bottom: 8px; }
.status.ok { background:#dff3e6; color: var(--green); } .status.bad { background:#fde4e1; color: var(--red); } .status.warn { background:#fff0d6; color: var(--orange); }
.reasons li { margin: 4px 0; }
.swatch { display:inline-block; width: 14px; height: 14px; border-radius:3px; vertical-align: middle; } .swatch.green { background:#18c964; } .swatch.grey { background:#999; }
.bignum { font-size: 120px; font-weight: 800; line-height: 1; text-align:center; font-variant-numeric: tabular-nums; }
.bignum.count { color: var(--green); }
.label { text-align:center; font-size: 22px; margin-bottom: 14px; }
.badge { font-weight: 800; padding: 6px 12px; border-radius: 8px; background: var(--orange); color:#fff; letter-spacing: .05em; }
.banner { border-radius: 10px; padding: 14px 18px; font-size: 22px; margin: 8px 0; border: 2px solid; }
.banner.measured { background:#dff3e6; border-color: var(--green); } .banner.needs_review { background:#fff0d6; border-color: var(--orange); }
.banner.withheld, .banner.stopped, .banner.incomplete { background:#fde4e1; border-color: var(--red); }
.banner ul { margin: 6px 0 0; font-size: 19px; }
#timeline { width: 100%; background:#fff; border:1px solid var(--line); border-radius:8px; }
table.events { width:100%; border-collapse: collapse; margin-top: 8px; } table.events td, table.events th { border-bottom:1px solid var(--line); padding: 6px 8px; text-align:left; }
.radios { display:flex; gap: 20px; } .radios label { display:flex; gap: 8px; align-items:center; }
.audit { font-size: 17px; }
.actions { margin-top: 12px; }
.warn { color: var(--red); font-weight: 600; }
/* report */
.report { background:#fff; border:1px solid var(--line); border-radius:10px; padding: 24px 28px; }
.report h1 { font-size: 28px; } .report table { border-collapse: collapse; width: 100%; margin: 8px 0 14px; } .report td, .report th { border:1px solid var(--line); padding: 6px 8px; text-align:left; vertical-align: top; font-size: 18px; }
.report .final { font-size: 26px; font-weight: 800; }
.report .disclaimer { font-size: 16px; color: var(--muted); border-top: 1px solid var(--line); padding-top: 10px; }
@media print { .topbar, .noprint, .screen:not(#scr-report) { display:none !important; } body { background:#fff; font-size: 13pt; } .report { border: none; padding: 0; } }
/* checkpoint 3: evidence review */
.strip { display: flex; flex-wrap: wrap; gap: 6px; margin: 8px 0; }
.thumb { position: relative; margin: 0; width: 130px; border: 2px solid transparent; border-radius: 6px; overflow: hidden; background: #000; }
.thumb.hl { border-color: var(--orange); }
.thumb img { display: block; width: 100%; }
.thumb .box { position: absolute; border: 3px solid #18c964; box-sizing: border-box; pointer-events: none; }
.thumb figcaption { position: absolute; left: 0; bottom: 0; background: rgba(0,0,0,.6); color: #fff; font-size: 13px; padding: 1px 5px; }
.rise { border-top: 1px solid var(--line); padding: 8px 0; }
.rise h3 { font-size: 19px; margin: 4px 0; }
.rise.uncertain h3 { color: var(--orange); } .rise.partial h3 { color: var(--muted); } .rise.full h3 { color: var(--green); } .rise.expiry h3 { color: var(--blue); }
.choices button, .radios button { font-size: 18px; padding: 8px 14px; min-height: 48px; }
button.chosen { background: #111; color: #fff; }
.steps li { margin: 6px 0; } .steps li.done { color: var(--green); } .steps li.done::after { content: " ✓"; font-weight: 800; }
.status.countdown { font-size: 64px; text-align: center; background: #111; color: #fff; }
.flag { border-top: 1px solid var(--line); padding: 6px 0; }
.okmsg { color: var(--green); font-weight: 700; }
```

```
fieldset.disabled { opacity: .55; }
/* checkpoint 4 */
.thumb.missing { display: flex; align-items: center; justify-content: center; min-height: 120px; background: #fde4e1; color: var(--red); font-size: 13px; text-align: center; padding: 6px; border: 2px dashed var(--red); }
.thumb.pick { cursor: pointer; } .thumb.pick:hover { border-color: var(--blue); }
#ref-card h2 { font-size: 22px; }
/* overnight: added-stand picking (ON-02) */
.thumb.pick:focus-visible { outline: 4px solid var(--focus); outline-offset: 2px; }
.thumb.nopick { opacity: .6; cursor: not-allowed; }
.thumb.added { border-color: var(--green); opacity: 1; }
/* overnight UX */
.filebtn:focus-within { outline: 4px solid var(--focus); outline-offset: 2px; }
.filebtn.disabled { opacity: .5; pointer-events: none; }
h1[tabindex="-1"]:focus { outline: none; }
.todo { border: 2px solid var(--blue); }
.todo ul { margin: 4px 0 0; }
.cdcline { font-weight: 600; }
#ref-card .thumb { width: 220px; }
.report table { table-layout: fixed; overflow-wrap: anywhere; }
@media (max-width: 900px) { #scr-test .stage { display: flex; flex-direction: column; } #scr-test .side { order: -1; } #btn-stop { position: sticky; bottom: 8px; } }
/* judge review: neutral result banner until a camera count is actually reported; click-to-enlarge evidence images */
.banner.pending { background: #f1f1ee; border-color: #8a8a85; }
.thumb .zoombtn { position: absolute; top: 3px; right: 3px; margin: 0; padding: 1px 6px; font-size: 13px; font-weight: 600; border: 1px solid #fff; border-radius: 5px; background: rgba(0,0,0,.6); color: #fff; min-height: 0; }
.thumb .zoombtn:hover { background: rgba(0,0,0,.85); }
.thumb:not(.pick) img { cursor: zoom-in; }
dialog.zoom { padding: 0; border: none; border-radius: 10px; max-width: 96vw; max-height: 96vh; overflow: auto; }
dialog.zoom::backdrop { background: rgba(0,0,0,.7); }
.zoombody { padding: 14px 18px; width: min-content; min-width: min(420px, 90vw); }
.zoomfig { position: relative; display: inline-block; background: #000; line-height: 0; }
.zoomfig img { display: block; height: auto; }
.zoomfig .box { position: absolute; border: 3px solid #18c964; box-sizing: border-box; pointer-events: none; }
.zoomcap { font-weight: 700; margin: 8px 0 0; }
/* demo mode (simulated participant) */
.demo-row { border-top: 1px solid var(--line); margin-top: 12px; padding-top: 6px; }
.demobanner { background: #5b2a86; color: #fff; border-radius: 10px; padding: 10px 16px; margin: 8px 0; font-size: 19px; }
.demolabel { font-weight: 800; color: #5b2a86; border: 2px solid #5b2a86; border-radius: 8px; padding: 6px 10px; background: #f4ecfa; }
.demotag { display: inline-block; font-size: 15px; font-weight: 700; color: #fff; background: #5b2a86; border-radius: 6px; padding: 2px 8px; vertical-align: middle; margin-left: 8px; }
@media print { .demobanner, .demotag { background: #fff; color: #000; border: 2px solid #000; } }
.demoquick { border: 2px solid #5b2a86; background: #faf6fd; display: flex; gap: 18px; align-items: center; justify-content: space-between; flex-wrap: wrap; }
.demoquick p { margin: 0; flex: 1 1 28em; font-size: 18px; }
.demoquick button { margin: 0; background: #5b2a86; border-color: #5b2a86; }

```

app/src/core/chair_stand.js

```
// Causal 30-Second Chair Stand measurement core, v0.7 (shared by the browser app and Node evaluation).
// v0.7 (checkpoint 4): the practice stand is only DETECTED by the core; it becomes the standing reference only when
// the observer ACCEPTS it (docs/CONTRACTS_CP4.md s1). Acceptance is bound to the hold id, the tracker lock generation,
// the camera scale (shin length) and its age. Expiry reports observedMs exactly (float bug found in checkpoint 4).
//
// Causality: every decision uses only frames already received (trailing medians, no look-ahead, no whole-clip
// normalisation). A full stand is counted at the first frame where full extension is confirmed.
//
// What changed in 0.6 (checkpoint-3 audit repairs, see docs/AUDIT_RESPONSE_CP3.md):
//  A. Each rise attempt is classified against the participant's OWN practice stand (hip height + knee + hip angle):
//       full      -> counted automatically (event time = first frame of confirmed full extension)
//       uncertain -> not counted automatically; needs observer review (count is reported as a range)
//       partial   -> not counted; only with strong evidence (shortfall beyond twice the data-derived band)
//     Without a practice stand, or when the camera is too frontal to see joint angles, extension cannot be
//     verified and every rise is "uncertain".
//  B. Status is decided LAST, after every event and session flag exists.
//  D. 30-s expiry uses the CDC position condition (no velocity requirement); near-halfway goes to review, and so
//     does a never-stood attempt that has dropped more than the full-stand band below its peak (0.8.1: plus raw/smoothed
//     disagreement, a rise not yet registered, or a rise after a perched sit).
//  G. Readiness needs recent, continuous observations; start() refuses a stale "Go".
//  H. Quality reports missing observations separately from tolerated gaps.
import { PARAMS, CORE_VERSION } from "./params.js";
import { legGeometry, armsCrossed, torsoGeometry } from "./pose_features.js";
import { ParticipantTracker } from "./tracker.js";

const median = (a) => { const s = a.filter((x) => Number.isFinite(x)).sort((x, y) => x - y); const n = s.length; return n ? (n %
2 ? s[n >> 1] : (s[n / 2 - 1] + s[n / 2]) / 2) : NaN; };
const spread = (a) => { const s = a.filter((x) => Number.isFinite(x)).sort((x, y) => x - y); if (!s.length) return Infinity; cons
t q = (p) => s[Math.min(s.length - 1, Math.max(0, Math.round(p * (s.length - 1))))]; return q(0.9) - q(0.1); };
const r3 = (x) => (Number.isFinite(x) ? +x.toFixed(3) : null);
const mergeWins = (w) => { if (!w) return []; const s = [...w].sort((a, b) => a[0] - b[0]); const out = [];
  for (const [a, b] of s) { if (out.length && a <= out[out.length - 1][1] + 300) out[out.length - 1][1] = Math.max(out[out.length
- 1][1], b); else out.push([a, b]); } return out; };
const r1 = (x) => (Number.isFinite(x) ? +x.toFixed(1) : null);

export function createChairStand({ width, height, params = PARAMS, calibrationMs, practiceStand = "require" } = {}) {
  const P = { ...params, ...(calibrationMs ? { calibrationMs } : {}) };
  const tracker = new ParticipantTracker(width, height, P);
  let phase = "calibrating";                // calibrating -> running -> done
  let shinHist = [];
  let calib = [];                           // {t, usable, h, knee, hip, view, status}
  let standRef = null;                      // detected practice stand {id, h, knee, hip, viewRatio, shinPx, t0, t1, lockGen, acc
epted}
  let suppressEndLevel = null, nextRefId = 1, lastHoldT = -Infinity, rejectedIds = new Set(), noReference = false, refRecord = nu
ll, suppressUntilBreak = false;
  let lastFrameT = null, pendingSelect = null, participantPose = null, lastOthersT = -Infinity;

  // running state
  let tGo = null, tEnd = null, hSit = null, angleOK = false, verified = false;
  let buf = { h: [], knee: [], hip: [] }, lastUsable = null, lastRaw = null, suspect = false;
  let state = "seated", consec = 0, consecT = null, attempt = null, upEvent = null, perched = null;
  let lostSince = null, lastTrackOkT = null;
  const events = [], gaps = [], trace = [];
  const flags = new Set(); const reviewNotes = []; const flagWindows = {};
  const winAt = (f, t0, t1) => { (flagWindows[f] = flagWindows[f] || []).push([t0, t1]); };   // records a time window only
  const flagAt = (f, t0, t1) => { flags.add(f); winAt(f, t0, t1); };                            // raises a review flag + window
  const q = { framesInWindow: 0, usableFrames: 0, noPoseFrames: 0, notParticipantFrames: 0, legsNotVisibleFrames: 0,
    ambiguousFrames: 0, overlapFrames: 0, otherPersonFrames: 0, discontinuityFrames: 0, maxFrameIntervalMs: 0 };
  let usableTimes = [], armsSeen = 0, armsUncrossed = 0, seatMin = Infinity; const seatTroughs = [];
  let reacquiredT = [];
  let result = null;

  const shinRef = () => (shinHist.length >= 5 ? median(shinHist) : null);
  const rel = (t) => ((t - tGo) / 1000).toFixed(1);

  // ---------- participant choice ----------
  function chooseInitial(poses, geoms) {
    if (pendingSelect) {                    // observer clicked the participant in the camera check
      let best = -1, bd = Infinity;
      geoms.forEach((g, i) => { const d = Math.hypot(g.centre[0] - pendingSelect.x, g.centre[1] - pendingSelect.y); if (d < bd) {
bd = d; best = i; } });
      pendingSelect = null;
      return best;
    }
    if (poses.length === 1) return 0;
    const seated = poses.map((lm, i) => ({ i, g: legGeometry(lm, width, height, null) })).filter((c) => c.g && c.g.h < P.seatedMa
xH);
    return seated.length === 1 ? seated[0].i : -1;   // several people and not exactly one seated -> not ready
  }

  function measure(frame) {
    const tr = tracker.update(frame.poses, phase === "calibrating" ? chooseInitial : null, frame.t);
    participantPose = tr.status === "ok" ? tr.pose : null;
    if (tr.status === "ok" && tr.others > 0) {
```

```
      const sizes = frame.poses.filter((p) => p !== tr.pose).map((p) => torsoGeometry(p, width, height).size);
      if (sizes.some((s) => s >= P.minRelativeSize * tr.geom.size)) lastOthersT = frame.t;
    } else if (frame.poses.length > 1) lastOthersT = frame.t;
    if (tr.status !== "ok") return { usable: false, status: tr.status, overlap: tr.overlap, others: tr.others };
    const g = legGeometry(tr.pose, width, height, shinRef());
    if (!g) return { usable: false, status: "legs_not_visible", overlap: tr.overlap, pose: tr.pose, geom: tr.geom, others: tr.oth
ers };
    shinHist.push(g.shin); if (shinHist.length > 30) shinHist.shift();
    return { usable: true, status: "ok", g, overlap: tr.overlap, pose: tr.pose, geom: tr.geom, others: tr.others };
  }

  // ---------- readiness ----------
  // Readiness windows: >= readinessMinUsableFrac of frames usable, and no gap between USABLE frames above
  // readinessMaxGapMs (a single dropped camera frame is tolerated; sparse or stale observation is not).
  function windowStats(tFrom, tTo) {
    const all = calib.filter((c) => c.t >= tFrom && c.t <= tTo);
    const win = all.filter((c) => c.usable);
    let maxGap = win.length ? win[0].t - tFrom : Infinity;
    for (let i = 1; i < win.length; i++) maxGap = Math.max(maxGap, win[i].t - win[i - 1].t);
    const usableFrac = all.length ? win.length / all.length : 0;
    return { win, maxGap, allUsable: win.length > 0 && usableFrac >= P.readinessMinUsableFrac };
  }

  function seatedWindow() {
    const last = calib[calib.length - 1]; if (!last) return null;
    const { win, maxGap, allUsable } = windowStats(last.t - P.calibrationMs, last.t);
    const minFrames = Math.ceil(P.minEffectiveFps * P.calibrationMs / 1000);
    const hs = win.map((c) => c.h);
    return { ok: allUsable && maxGap <= P.readinessMaxGapMs && win.length >= minFrames && median(hs) < P.seatedMaxH && spread(hs)
<= P.calibrationMaxSpread,
      hSit: median(hs), t0: last.t - P.calibrationMs, moving: allUsable && spread(hs) > P.calibrationMaxSpread, notSeated: allUsa
ble && median(hs) >= P.seatedMaxH, sparse: maxGap > P.readinessMaxGapMs || win.length < minFrames };
  }

  // Practice stand DETECTION: a still standing hold (>= standHoldMs), preceded by seated frames.
  // Detection alone never makes the reference trusted (checkpoint 4, audit A).
  // 0.8.1 (overnight ON-09/ON-14/ON-21): one practice stand = one stand, from its first still hold until the participant
  // descends below halfway ("ended"). While it continues, the reference keeps the MOST EXTENDED still window of that stand
  // (never a later sag, never an early not-yet-straight window). It is offered for confirmation only AFTER it ended, so
  // the card shows the final values and their image; showing (freezeReference) or accepting freezes them. A new stand
  // after an ended one gets a new id, which voids any earlier acceptance.
  // 0.8.2 (ON-27): compare like with like - by joint angles when both windows have them, otherwise by hip height
  const hasAngles = (v) => Number.isFinite(v.knee) && Number.isFinite(v.hip);
  function representative(cands) {
    const withA = cands.filter(hasAngles);
    const pool = withA.length ? withA : cands;
    const score = (v) => (withA.length ? v.knee + v.hip : 100 * v.h);
    const best = Math.max(...pool.map(score));
    const near = pool.filter((v) => score(v) >= best - (withA.length ? P.refTopBandDeg : 100 * 0.02)).sort((a, b) => score(a) - s
core(b));
    return near[(near.length - 1) >> 1];
  }
  const moreExtended = (a, b) => (hasAngles(a) && hasAngles(b) ? a.knee + a.hip > b.knee + b.hip : hasAngles(a) !== hasAngles(b)
? hasAngles(a) : a.h > b.h);
  function updateStandRef(t) {
    const last = calib[calib.length - 1]; if (!last || !last.usable) return;
    const { win, maxGap, allUsable } = windowStats(last.t - P.standHoldMs, last.t);
    if (!allUsable || maxGap > P.readinessMaxGapMs || win.length < Math.ceil(P.minEffectiveFps * P.standHoldMs / 1000)) return;
    const seatedEarlier = calib.filter((c) => c.usable && c.t < last.t - P.standHoldMs && c.h < P.seatedMaxH).map((c) => c.h);
    if (!seatedEarlier.length) return;
    const hs = win.map((c) => c.h);
    if (median(hs) < Math.max(P.standAbsMin, median(seatedEarlier) + P.standHoldMinRise)) return;
    // 0.8.2 (ON-27): the window supplies joint angles only if >= refMinAngleFrac of its frames have both; otherwise its
    // angles are "not measured" (-> view unverifiable, every rise needs review) instead of coming from one or two frames
    const angFrac = win.filter((c) => Number.isFinite(c.knee) && Number.isFinite(c.hip)).length / win.length;
    const angOK = angFrac >= P.refMinAngleFrac && angFrac * win.length >= P.minAngleFrames;
    if (spread(hs) > P.standHoldMaxSpreadH || (angOK && (spread(win.map((c) => c.knee)) > P.standHoldMaxSpreadDeg || spread(win.m
ap((c) => c.hip)) > P.standHoldMaxSpreadDeg))) return;
    if (suppressUntilBreak) return;                                  // the rejected stand has not ended yet: don't re-ask
    const continuing = standRef && !standRef.ended && standRef.lockGen === tracker.lockGen;
    if (continuing && (standRef.accepted || standRef.frozen)) return;  // frozen once shown or accepted
    const vals = { h: median(hs), knee: angOK ? median(win.map((c) => c.knee)) : NaN, hip: angOK ? median(win.map((c) => c.hip))
: NaN, viewRatio: angOK ? median(win.map((c) => c.view)) : NaN,
      shinPx: median(win.map((c) => c.shin)), t0: win[0].t, t1: last.t, lockGen: tracker.lockGen };
    lastHoldT = t;
    // 0.8.3 (ON-32): "most extended" by the maximum of noisy windows biased the reference upward (more review); use the
    // MEDIAN window among those within refTopBand of the most extended one
    if (continuing) { standRef.cands.push(vals); Object.assign(standRef, representative(standRef.cands)); }
    else { standRef = { id: nextRefId++, accepted: null, frozen: false, ended: false, endLevel: (median(seatedEarlier) + vals.h)
/ 2, cands: [vals], ...vals }; }
    standRef.sideView = standRef.viewRatio <= P.sideViewMaxShoulderRatio;
    standRef.postureWarning = standRef.sideView && (standRef.knee < P.referenceWarnKneeDeg || standRef.hip < P.referenceWarnHipDe
g);
  }
  // The practice stand has ended once the participant descends below halfway after its last still frame.
  // 0.8.2 (ON-28): it must stay below for standEndMs (a brief landmark glitch must not end a stand mid-hold)
  let belowSince = null;
```

```
  function markStandEnded(c) {
    if (!c.usable) return;
    const level = standRef && !standRef.ended ? standRef.endLevel : suppressUntilBreak ? suppressEndLevel : null;
    if (level === null || c.t <= lastHoldT || c.h > level) { belowSince = null; return; }
    if (belowSince === null) belowSince = c.t;
    if (c.t - belowSince < P.standEndMs) return;
    belowSince = null;
    if (standRef && !standRef.ended) standRef.ended = true;
    else if (suppressUntilBreak) { suppressUntilBreak = false; suppressEndLevel = null; }
  }

  // Why an accepted reference is no longer valid (or null if it is).
  function referenceProblem(tNow) {
    if (!standRef || !standRef.accepted) return null;
    if (standRef.lockGen !== tracker.lockGen) return "reference_person_changed";
    if (tNow - standRef.t1 > P.referenceMaxAgeMs) return "reference_stale";
    const shins = calib.filter((c) => c.usable && c.t >= tNow - P.calibrationMs).map((c) => c.shin);
    if (shins.length && Math.abs(median(shins) - standRef.shinPx) / standRef.shinPx > P.referenceMaxShinChange) return "reference
_camera_or_person_changed";
    return null;
  }

  function readiness() {
    const reasons = [];
    const last = calib[calib.length - 1];
    if (!last) return { ready: false, reasons: ["no_frames"], practice: practiceInfo() };
    const st = last.status;
    if (st === "none") reasons.push("no_person_detected");
    if (st === "unlocked") reasons.push("several_people_select_participant");
    if (st === "ambiguous") reasons.push("another_person_overlaps_participant");
    if (st === "lost") reasons.push("participant_lost");
    if (st === "legs_not_visible") reasons.push("hips_knees_ankles_not_all_visible");
    const sw = seatedWindow();
    if (!reasons.length && practiceStand !== "skip" && !noReference) {
      const prob = referenceProblem(last.t);
      if (!standRef || rejectedIds.has(standRef.id)) reasons.push("practice_stand_needed");
      else if (!standRef.accepted && !standRef.ended) reasons.push("practice_stand_in_progress");
      else if (!standRef.accepted) reasons.push("practice_stand_needs_confirmation");
      else if (prob) reasons.push(prob);
    }
    if (!reasons.length && sw) {
      if (sw.sparse) reasons.push("camera_frames_too_sparse");
      else if (sw.notSeated) reasons.push("participant_not_seated");
      else if (!sw.ok) reasons.push("hold_still_seated");
      else if (standRef && standRef.t1 >= sw.t0) reasons.push("hold_still_seated");
    }
    return { ready: reasons.length === 0, reasons, hSit: sw ? sw.hSit : null, practice: practiceInfo() };
  }
  const practiceInfo = () => (practiceStand === "skip" || noReference ? { mode: "skipped", state: "none" } : standRef && !rejecte
dIds.has(standRef.id)
    ? { mode: "calibrated", state: standRef.accepted ? "accepted" : standRef.ended ? "detected" : "in_progress", refId: standRef.
id, h: r3(standRef.h), knee: r1(standRef.knee), hip: r1(standRef.hip),
        viewRatio: r3(standRef.viewRatio), sideView: standRef.sideView, postureWarning: standRef.postureWarning, t0: standRef.t0,
t1: standRef.t1, accepted: standRef.accepted }
    : { mode: "needed", state: "none" });

  // ---------- running ----------
  function recordGap(t0, t1, hBefore, hAfter) {
    const dur = t1 - t0;
    if (dur < P.reviewGapMs) return;
    const moving = (hBefore !== null && hBefore > hSit + P.movementGapRise) || (hAfter !== null && hAfter > hSit + P.movementGapR
ise) || attempt !== null || state === "up";
    gaps.push({ t0, t1, dur, moving });
    if (dur > P.maxGapMs) return;
    if (moving) { flagAt("gap_during_movement", t0, t1); reviewNotes.push(`The participant could not be measured for ${(dur / 100
0).toFixed(2)} s while moving (${rel(t0)}–${rel(t1)} s).`); }
  }

  let recentIntervals = [];
  const smoothSpanMs = () => { const m = median(recentIntervals); return Number.isFinite(m) ? m * (P.smoothFrames - 1) : 0; };
  const deficits = (s) => ({
    h: standRef ? standRef.h - s.h : NaN,
    knee: angleOK ? standRef.knee - s.knee : NaN,
    hip: angleOK ? standRef.hip - s.hip : NaN,
  });
  const fullOK = (d) => verified && d.h <= P.fullH && d.knee <= P.fullKneeDeg && d.hip <= P.fullHipDeg;
  const candidateUp = (s) => s.h >= P.standAbsMin && s.h >= hSit + P.standRiseMin && (!standRef || standRef.h - s.h <= P.fullH);

  function newEvent(cls, fields) {
    const ev = { n: events.length + 1, cls, counted: cls === "full" || cls === "expiry", tRise: null, tFull: null, tPeak: null, t
Seat: null,
      peak: null, shortfall: null, flags: [], ...fields };
    if (reacquiredT.some((t) => ev.tRise !== null && ev.tRise - t < 3000 && ev.tRise >= t - 3000)) ev.flags.push("after_participa
nt_reacquired");
    events.push(ev); return ev;
  }

```

```
  function topEvidence(a) {
    const smp = a.samples || []; if (!smp.length) return;
    const hs = smp.map((x) => x.h).sort((x, y) => x - y);
    const peak = hs[Math.max(0, Math.ceil(0.9 * hs.length) - 1)];                  // 90th percentile: one noisy frame cannot set it
    const top = smp.filter((x) => x.h >= peak - P.peakBandH && Number.isFinite(x.k) && Number.isFinite(x.p));
    a.topAngleN = top.length;
    a.best.knee = top.length ? Math.min(...top.map((x) => x.k)) : Infinity;
    a.best.hip = top.length ? Math.min(...top.map((x) => x.p)) : Infinity;
  }
  function closeAttempt(t, reasonFlag = null) {
    const a = attempt; attempt = null;
    topEvidence(a);
    const best = a.best;
    let cls;
    const k = P.partialFactor;
    // 0.8: "strong evidence" needs MEASURED values (ON-05: a non-finite angle shortfall means the angles were never seen,
    // not that they were bad) and a smoothing window short enough to resolve the peak (ON-08: low frame rates flatten it).
    const coarse = smoothSpanMs() > P.maxPartialSmoothSpanMs;
    const anglesSeen = Number.isFinite(best.knee) && Number.isFinite(best.hip) && (a.topAngleN || 0) >= P.minAngleFrames;   // >= 3 frames at the top (ON-26)
    if (verified) {
      const strong = !coarse && (best.h > k * P.fullH || (anglesSeen && best.knee > k * P.fullKneeDeg && best.hip > k * P.fullHipDeg));
      cls = strong ? "partial" : "uncertain";
      if (!strong && coarse) reasonFlag = reasonFlag || "frame_rate_too_low_to_rule_out_full_stand";
      else if (!strong && !anglesSeen) reasonFlag = reasonFlag || "joint_angles_not_measured";
    } else if (standRef) {
      cls = !coarse && best.h > k * P.fullH ? "partial" : "uncertain";
    } else {
      cls = a.peakH < P.standAbsMin ? "partial" : "uncertain";
    }
    const ev = newEvent(cls, { tRise: a.tRise, tPeak: a.tPeak, tSeat: t, peak: { h: r3(a.peakH), knee: r1(a.peakKnee), hip: r1(a.peakHip) },
      shortfall: { h: r3(best.h), knee: r1(best.knee), hip: r1(best.hip) } });
    if (reasonFlag) ev.flags.push(reasonFlag);
    if (a.seatNotReached) ev.flags.push("seat_return_not_confirmed");
    if (cls === "uncertain") ev.flags.push(verified ? "extension_not_confirmed" : standRef ? "extension_unverifiable_view" : practiceStand === "skip" ? "no_practice_stand" : "no_accepted_reference");
    return ev;
  }

  const halfwayH = () => hSit + ((standRef ? standRef.h : P.defaultStandRef) - hSit) / 2;
  function stepState(t, s, raw = null) {
    const d = deficits(s);
    // 0.8.2 (ON-26): top-of-rise angle evidence uses this frame's RAW angles (a smoothed median still contains frames from
    // lower in the rise, which at low frame rates made a genuine full stand look like a partial)
    const rd = raw ? deficits(raw) : { knee: NaN, hip: NaN };
    if (state === "seated" && !attempt) seatMin = Math.min(seatMin, s.h);
    if (state === "seated") {
      // 0.8 (ON-06): after a descent below halfway that never reached the calibrated seat level ("perched"), a new rise
      // starts only once h climbs perchRiseMin above the lowest point; such a rise can never be counted automatically.
      if (perched) {
        if (s.h <= hSit + P.seatReturnRise) perched = null;
        else {
          perched.min = Math.min(perched.min, s.h);
          if (!attempt && s.h > perched.min + P.perchRiseMin) attempt = { tRise: t, seatNotReached: true, best: { h: Infinity, knee: Infinity, hip: Infinity }, peakH: -Infinity, tPeak: t, peakKnee: NaN, peakHip: NaN };
        }
      }
      if (!attempt && !perched && s.h > hSit + P.partialRiseMin) {
        attempt = { tRise: t, best: { h: Infinity, knee: Infinity, hip: Infinity }, peakH: -Infinity, tPeak: t, peakKnee: NaN, peakHip: NaN };
        if (Number.isFinite(seatMin)) { seatTroughs.push(seatMin); seatMin = Infinity; }
      }
      if (attempt) {
        if (s.h > attempt.peakH) { attempt.peakH = s.h; attempt.tPeak = t; attempt.peakKnee = s.knee; attempt.peakHip = s.hip; }
        // 0.8.1 (ON-20): angle shortfalls count as evidence only from frames AT the top of this rise (h within
        // peakBandH of its highest point so far); hip height saturates near standing, so angles seen lower in the rise
        // say nothing about whether the participant straightened fully.
        if (Number.isFinite(d.h)) attempt.best.h = Math.min(attempt.best.h, d.h);
        // raw samples of this rise; "at the top" is decided when the rise closes (from past frames only), against a
        // spike-robust peak of the RAW hip height (the smoothed h lags and would admit frames from lower in the rise)
        (attempt.samples = attempt.samples || []).push({ h: raw ? raw.h : s.h, k: rd.knee, p: rd.hip });
        const up = verified ? fullOK(d) : candidateUp(s);
        // 0.8.3 (ON-33): a frame at full height whose joint angles are missing neither confirms nor breaks the run
        const unknown = verified && !up && Number.isFinite(d.h) && d.h <= P.fullH && (!Number.isFinite(d.knee) || !Number.isFinite(d.hip));
        if (unknown) { /* hold */ }
        else if (up) {
          if (consec === 0) consecT = t;
          consec += 1;
          if (consec >= P.standConfirmFrames) {
            const a = attempt; attempt = null; consec = 0;
            const auto = verified && !a.seatNotReached;
            upEvent = newEvent(auto ? "full" : "uncertain", { tRise: a.tRise, tFull: auto ? consecT : null, tPeak: t, peak: { h: r3(s.h), knee: r1(s.knee), hip: r1(s.hip) }, shortfall: null });
```

```
            if (a.seatNotReached) upEvent.flags.push("seat_return_not_confirmed");
            if (!verified) upEvent.flags.push(standRef ? "extension_unverifiable_view" : practiceStand === "skip" ? "no_practice_
stand" : "no_accepted_reference");
            upEvent._best = { h: Infinity, knee: Infinity, hip: Infinity };
            state = "up";
          }
        } else consec = 0;
        if (attempt && s.h <= hSit + P.seatReturnRise) closeAttempt(t);
      }
    } else {                                  // state up (counted or unverified stand), until back seated
      const ev = upEvent;
      if (s.h > ev.peak.h) { ev.peak = { h: r3(s.h), knee: r1(s.knee), hip: r1(s.hip) }; ev.tPeak = t; }
      for (const kk of ["h", "knee", "hip"]) if (Number.isFinite(d[kk])) ev._best[kk] = Math.min(ev._best[kk], d[kk]);
      // 0.8.2 (ON-29): a deep dip that stays above halfway and comes back up (a half-sit, or a squat) is not a new stand
      // by rule, but it must not pass silently: flag it for review on its images
      if (!ev._dip || s.h < ev._dip.h) ev._dip = { h: s.h, t };
      if (ev.peak.h - ev._dip.h > P.dipFlagH && standRef && s.h >= standRef.h - P.fullH) { flagAt("stand_dipped_without_sitting",
ev._dip.t - 500, t); ev._dip = { h: s.h, t }; }
      if (s.h <= hSit + P.seatReturnRise) { ev.tSeat = t; state = "seated"; upEvent = null; seatMin = Infinity; }
      else if (s.h < halfwayH()) { ev.tSeat = t; state = "seated"; upEvent = null; seatMin = Infinity; perched = { t, min: s.h };
}   // ON-06
    }
  }

  function applyExpiry() {
    // [CDC] "If the patient is over halfway to a standing position when 30 seconds have elapsed, count it as a stand."
    const standH = standRef ? standRef.h : P.defaultStandRef;
    const halfway = hSit + (standH - hSit) / 2;
    const out = { evaluated: true, state, halfway: r3(halfway), h: lastUsable ? r3(lastUsable.h) : null, decision: "none" };
    if (state === "up" || !lastUsable) return out;                    // already counted, or not observed
    // 0.8 (ON-07): the smoothed h lags a moving hip, so the decision also looks at the last RAW observation. A rise that
    // the smoothed signal has not registered yet, or raw and smoothed on different sides of halfway, goes to review.
    const raw = lastRaw && lastRaw.t >= lastUsable.t - 1 ? lastRaw.h : lastUsable.h;
    out.hRaw = r3(raw);
    if (!attempt) {
      // seated (or perched after a sit that did not reach the seat level: ON-19, no phantom rise while sitting down)
      if (raw <= (perched ? perched.min + P.perchRiseMin : hSit + P.partialRiseMin)) return out;
      const tr = lastRaw ? lastRaw.t : lastUsable.t;
      attempt = { tRise: tr, seatNotReached: !!perched, best: { h: Infinity, knee: Infinity, hip: Infinity }, peakH: raw, tPeak:
tr, peakKnee: NaN, peakHip: NaN };
      if (raw >= halfway - P.expiryNearHalfBand) {
        const ev = closeAttempt(tEnd, "expiry_rise_in_progress"); ev.cls = "uncertain"; ev.counted = false;
        out.decision = "review_rise_in_progress";
        reviewNotes.push("At 30 s the participant was part-way up and the camera could not tell whether they were rising: decide
from the 30-second image whether they were over halfway up on a rise.");
      } else {
        const ev = closeAttempt(tEnd, "expiry_under_halfway"); ev.cls = "partial"; ev.counted = false; ev.flags = ["expiry_under_
halfway"];
        out.decision = "not_counted_under_halfway";
      }
      return out;
    }
    const h = lastUsable.h;
    const descending = attempt.peakH - h > P.fullH;
    const split = (h > halfway) !== (raw > halfway) || Math.abs(raw - halfway) <= P.expiryNearHalfBand;
    // (ON-39: "descending" compares the attempt's highest point with the lagging smoothed h; a NEW push inside the same
    //  attempt that is rising over halfway - raw above both halfway and the smoothed h - is not a descent)
    if (descending && (raw <= halfway || raw <= h) && raw <= halfway + P.expiryNearHalfBand) {
      // 0.8.3/0.8.4 (ON-31, ON-35): the rise already peaked and the participant is coming down, at or below about halfway,
      // at 30 s. The question is whether that rise WAS a full stand before 30 s, not the 30-s halfway question: classify it
      // normally (review unless strong partial evidence). Checked BEFORE the near-halfway test (the smoothed height lags
      // a descent). The flag is neutral and does not start with "expiry_".
      closeAttempt(tEnd, "rise_peaked_before_30s");
      out.decision = "closed_descending_rise";
      return out;
    }
    if (Math.abs(h - halfway) <= P.expiryNearHalfBand || split) {
      const ev = closeAttempt(tEnd, "expiry_near_halfway"); ev.cls = "uncertain"; ev.counted = false;
      out.decision = "review_near_halfway";
      reviewNotes.push(`At 30 s the participant was about halfway up (h=${h.toFixed(2)}, halfway=${halfway.toFixed(2)}): decide w
hether the last rise counts.`);
    } else if (h > halfway && !descending && attempt.seatNotReached) {        // ON-18b: a rise after a perched sit is never auto
-counted
      const ev = closeAttempt(tEnd, "expiry_over_halfway"); ev.cls = "uncertain"; ev.counted = false;
      out.decision = "review_after_unconfirmed_sit";
      reviewNotes.push("At 30 s the participant was over halfway up, but had not clearly sat down before this rise: decide whethe
r it counts.");
    } else if (h > halfway && !descending) {
      const ev = newEvent("expiry", { tRise: attempt.tRise, tFull: null, tPeak: attempt.tPeak, tSeat: null, peak: { h: r3(attemp
t.peakH), knee: r1(attempt.peakKnee), hip: r1(attempt.peakHip) } });
      ev.flags.push("expiry_over_halfway"); ev.tExpiry = tEnd; attempt = null;
      if (!standRef) { ev.flags.push("expiry_default_standing_reference"); }
      out.decision = "counted_over_halfway";
    } else if (h > halfway && descending) {
      const ev = closeAttempt(tEnd, "expiry_attempt_descending"); ev.cls = "uncertain"; ev.counted = false;
      out.decision = "review_descending_attempt";
      reviewNotes.push("At 30 s the participant was above halfway but moving down from a rise that never reached full standing: d
```

```
ecide whether it counts.");
    } else {
      const ev = closeAttempt(tEnd, "expiry_under_halfway"); ev.cls = "partial"; ev.counted = false;
      ev.flags = ev.flags.filter((f) => f === "expiry_under_halfway");
      out.decision = "not_counted_under_halfway";
    }
    return out;
  }

  return {
    get phase() { return phase; },
    get params() { return P; },

    /** Feed one frame {t: ms (monotonic), poses: [...]}. Returns a live snapshot for the UI. */
    push(frame) {
      if (phase === "done") return this.snapshot();
      if (lastFrameT !== null && frame.t <= lastFrameT) throw new Error("timestamps must increase");
      const interval = lastFrameT === null ? 0 : frame.t - lastFrameT;
      lastFrameT = frame.t;

      if (phase === "calibrating") {
        const m = measure(frame);
        if (m.status === "lost") { tracker.reset(); shinHist = []; }   // 0.8 (ON-13): a new lock starts a new shin history
        calib.push({ t: frame.t, usable: m.usable, h: m.usable ? m.g.h : null, knee: m.usable ? m.g.knee : NaN, hip: m.usable ?
m.g.hip : NaN, view: m.usable ? m.g.viewRatio : NaN, shin: m.usable ? m.g.shin : NaN, status: m.status });
        while (calib.length && calib[0].t < frame.t - 20000) calib.shift();
        if (practiceStand !== "skip") { updateStandRef(frame.t); markStandEnded(calib[calib.length - 1]); }
        trace.push({ t: frame.t, h: m.usable ? r3(m.g.h) : null, s: m.status, ph: "cal" });
        if (trace.length > 3000) trace.shift();
        return this.snapshot();
      }

      if (frame.t > tEnd) { this.finish("expired"); return this.snapshot(); }
      q.framesInWindow += 1; q.maxFrameIntervalMs = Math.max(q.maxFrameIntervalMs, interval);
      const m = measure(frame);
      if (!frame.poses.length) q.noPoseFrames += 1;
      else if (["lost", "ambiguous", "unlocked", "none"].includes(m.status)) q.notParticipantFrames += 1;
      if (m.status === "legs_not_visible") q.legsNotVisibleFrames += 1;
      if (m.status === "ambiguous") q.ambiguousFrames += 1;
      if (m.overlap) { q.overlapFrames += 1; winAt("another_person_overlapped_candidate", frame.t - 300, frame.t + 300); }
      if (frame.poses.length > 1) q.otherPersonFrames += 1;
      // identity continuity: participant lost/ambiguous for >= reviewGapMs, then found again while someone else is around
      if (m.status === "ok") {
        if (lostSince !== null && frame.t - lostSince >= P.reviewGapMs && frame.t - lastOthersT <= P.otherPersonRecentMs) {
          flagAt("participant_reacquired_with_other_person_in_view", lostSince, frame.t); reacquiredT.push(frame.t);
          reviewNotes.push(`The participant was out of sight for ${((frame.t - lostSince) / 1000).toFixed(1)} s while another per
son was in view (${rel(lostSince)}–${rel(frame.t)} s): check that the same person was measured afterwards.`);
        }
        lostSince = null; lastTrackOkT = frame.t;
      } else if (lostSince === null) lostSince = lastTrackOkT ?? frame.t;
      if (m.usable && lastRaw) {
        const dt = Math.min((frame.t - lastRaw.t) / 1000, suspect ? 0.1 : Infinity);
        if (Math.abs(m.g.h - lastRaw.h) > P.jumpBase + P.jumpPerSec * dt) {
          m.usable = false; m.status = "discontinuity"; q.discontinuityFrames += 1; suspect = true;
          winAt("identity_or_landmark_discontinuity_candidate", frame.t - 200, frame.t + 200);
          if (q.discontinuityFrames >= 3) flags.add("identity_or_landmark_discontinuity");
        }
      }
      if (m.usable) { lastRaw = { t: frame.t, h: m.g.h }; suspect = false; }
      let sm = null;
      if (m.usable) {
        const prevT = lastUsable ? lastUsable.t : tGo;
        if (lastUsable) { recentIntervals.push(frame.t - lastUsable.t); if (recentIntervals.length > 15) recentIntervals.shift();
}
        if (frame.t - prevT >= P.reviewGapMs) buf = { h: [], knee: [], hip: [] };
        recordGap(prevT, frame.t, lastUsable ? lastUsable.h : hSit, m.g.h);
        for (const kk of ["h", "knee", "hip"]) { buf[kk].push(m.g[kk]); if (buf[kk].length > P.smoothFrames) buf[kk].shift(); }
        // 0.8.2 (ON-26): a smoothed angle exists only if THIS frame has it and >= 3 of the buffered frames do; otherwise the
        // median of a few older frames (lower in the rise) would pose as the angle at the top.
        const angleOf = (kk) => (Number.isFinite(m.g[kk]) && buf[kk].filter(Number.isFinite).length >= P.minAngleFrames ? median
(buf[kk]) : NaN);
        sm = { h: median(buf.h), knee: angleOf("knee"), hip: angleOf("hip") };
        stepState(frame.t, sm, m.g);
        lastUsable = { t: frame.t, h: sm.h };
        usableTimes.push(frame.t); q.usableFrames += 1;
        const crossed = armsCrossed(m.pose, width, height, m.geom.size);
        if (crossed !== null) { armsSeen += 1; if (!crossed) armsUncrossed += 1; }
      }
      trace.push({ t: frame.t, h: sm ? r3(sm.h) : null, k: sm ? r1(sm.knee) : null, hp: sm ? r1(sm.hip) : null, s: m.status, o:
m.overlap ? 1 : 0 });
      return this.snapshot();
    },

    readiness,

    /** Observer clicked the participant (normalised image coordinates) during the camera check. */
    selectParticipant(xn, yn) {
```

```
      if (phase !== "calibrating") return false;
      tracker.reset(); shinHist = []; pendingSelect = { x: xn * width, y: yn * height }; calib = []; standRef = null; noReference
= false;
      return true;
    },

    /** The app shows the practice-stand card for `refId`: freeze its values so what is accepted is what was shown. */
    freezeReference(refId) {
      if (phase !== "calibrating" || !standRef || standRef.id !== refId || !standRef.ended) return false;
      standRef.frozen = true; return true;
    },
    /** Observer accepts the DETECTED practice stand `refId` as a full stand. basis: 'image' | 'observer_watched'. */
    acceptReference({ refId, basis }) {
      if (phase !== "calibrating" || !standRef || standRef.id !== refId || rejectedIds.has(refId) || !standRef.ended) return fals
e;
      if (basis !== "image" && basis !== "observer_watched") return false;
      standRef.accepted = { basis, atT: lastFrameT }; standRef.frozen = true; noReference = false;
      return true;
    },
    /** Observer says the detected practice stand was NOT a full stand: it is discarded; a new one is needed. */
    rejectReference(refId) {
      if (phase !== "calibrating" || !standRef || standRef.id !== refId) return false;
      rejectedIds.add(refId); if (!standRef.ended) { suppressUntilBreak = true; suppressEndLevel = standRef.endLevel; } standRef
= null;   // a new hold needs the stand to be broken first
      return true;
    },
    /** Observer chooses to count without a reference (every rise will be 'uncertain'). */
    countWithoutReference(on = true) { if (phase === "calibrating") noReference = !!on; return noReference; },

    /** Start the 30-s window at the "Go" cue. Requires readiness based on recent, continuous frames and an ACCEPTED,
     *  still-valid reference (or an explicit choice to count without one). */
    start(tGoMs, { withoutReference = false } = {}) {
      if (withoutReference) noReference = true;
      const r = readiness();
      if (!r.ready) throw new Error("not ready: " + r.reasons.join(","));
      if (tGoMs - lastFrameT > P.maxStartStaleMs) throw new Error(`stale: last camera frame ${Math.round(tGoMs - lastFrameT)} ms
before Go`);
      hSit = r.hSit; tGo = tGoMs; tEnd = tGoMs + P.windowMs; phase = "running";
      if (noReference || !standRef || !standRef.accepted) { refRecord = standRef ? { ...standRef, usedAsReference: false } : nul
l; standRef = null; }
      else refRecord = { ...standRef, usedAsReference: true };
      angleOK = !!(standRef && standRef.sideView && Number.isFinite(standRef.knee) && Number.isFinite(standRef.hip));
      verified = angleOK;
      lastUsable = null; lastRaw = null; buf = { h: [], knee: [], hip: [] };
      return { hSit, tGo, tEnd, practice: practiceInfo(), extensionCheck: verified ? "verified_side_view" : standRef ? "unverifia
ble_view" : noReference ? "no_accepted_reference" : "no_practice_stand" };
    },

    /** Close the test. reason: 'expired' | 'stream_end' | 'stopped' | 'page_hidden' | 'camera_error' */
    finish(reason, tNow = lastFrameT) {
      if (result) return result;
      if (phase !== "running") throw new Error("finish() called while " + phase);
      phase = "done";
      const withheld = [];
      let expiry = { evaluated: false };
      if (reason === "expired") {
        const endGap = tEnd - (lastUsable ? lastUsable.t : tGo);
        if (endGap > P.maxGapMs) withheld.push(`tracking_lost_before_end (${(endGap / 1000).toFixed(1)} s without the participan
t)`);
        else if (endGap >= P.reviewGapMs) {
          flagAt("expiry_endpoint_not_observed", tEnd - endGap - 500, tEnd);
          reviewNotes.push(`The participant was not measured during the last ${(endGap / 1000).toFixed(2)} s before 30 s: check t
he final position.`);
        }
        expiry = applyExpiry();
      } else if (attempt) {
        closeAttempt(Math.min(tNow ?? tGo, tEnd), "test_ended_during_rise");
      }
      // ---- quality (audit H): missing observations vs tolerated gaps are reported separately ----
      const tStop = reason === "expired" ? tEnd : Math.min(tNow ?? tGo, tEnd);
      const obsMs = reason === "expired" ? P.windowMs : Math.max(1, tStop - tGo);   // exact on expiry (float bug, cp4)
      const pts = [tGo, ...usableTimes.filter((t) => t <= tStop), tStop];
      let beyondTolMs = 0, longestGap = 0;
      for (let i = 1; i < pts.length; i++) { const g = pts[i] - pts[i - 1]; longestGap = Math.max(longestGap, g); beyondTolMs +=
Math.max(0, g - P.gapToleranceMs); }
      for (const g of gaps) if (g.dur > P.maxGapMs) withheld.push(`tracking_lost ${rel(g.t0)}-${rel(g.t1)} s (${(g.dur / 1000).to
Fixed(1)} s)`);
      if (beyondTolMs / obsMs > P.maxUnusableFraction) withheld.push(`participant_not_measurable_for_${Math.round(100 * beyondTol
Ms / obsMs)}pct_of_the_test`);
      const effFps = q.usableFrames / (obsMs / 1000);
      if (effFps < P.minEffectiveFps) withheld.push(`frame_rate_too_low (${effFps.toFixed(1)} usable fps)`);
      if (q.maxFrameIntervalMs > P.maxGapMs) withheld.push(`video_frames_stopped_for_${(q.maxFrameIntervalMs / 1000).toFixed(1)}_
s`);
      const missingPct = q.framesInWindow ? (100 * (q.framesInWindow - q.usableFrames)) / q.framesInWindow : 100;
      if (missingPct > P.intermittentMissingPct) {
        flags.add("intermittent_tracking");
        reviewNotes.push(`The participant was measured in only ${(100 - missingPct).toFixed(0)}% of the video frames (short gaps
```

```
were bridged).`);
      }
      if (Number.isFinite(seatMin)) seatTroughs.push(seatMin);
      const typicalSeat = seatTroughs.length >= 2 ? median(seatTroughs) : null;
      if (typicalSeat !== null && hSit - typicalSeat > 0.2) {
        flags.add("seated_baseline_drift");
        reviewNotes.push(`The seated position during the test (h~${typicalSeat.toFixed(2)}) was lower than at calibration (h=${hS
it.toFixed(2)}).`);
      }
      if (flags.has("identity_or_landmark_discontinuity")) flagWindows.identity_or_landmark_discontinuity = mergeWins(flagWindow
s.identity_or_landmark_discontinuity_candidate);
      if (q.discontinuityFrames > 0) reviewNotes.push(`${q.discontinuityFrames} frame(s) were rejected because hip height jumped
faster than a person can move (possible identity switch or landmark error).`);
      if (q.overlapFrames > 0) { flags.add("another_person_overlapped"); flagWindows.another_person_overlapped = mergeWins(flagWi
ndows.another_person_overlapped_candidate); reviewNotes.push(`Another person overlapped the participant in ${q.overlapFrames} frame
(s).`); }
      delete flagWindows.identity_or_landmark_discontinuity_candidate; delete flagWindows.another_person_overlapped_candidate;
      // ---- event-derived review flags (audit B): decided BEFORE the status ----
      for (const ev of events) {
        if (ev.cls === "full" && ev._best) {
          const b = ev._best;                  // shortfall at the best frame while up (for the review display)
          ev.shortfall = { h: r3(b.h), knee: r1(b.knee), hip: r1(b.hip) };
        }
        delete ev._best; delete ev._dip;
        if (ev.cls === "uncertain") flags.add("uncertain_rises");
        if (ev.flags.includes("after_participant_reacquired") && ev.counted) flags.add("participant_reacquired_with_other_person_
in_view");
        if (ev.flags.includes("expiry_default_standing_reference")) flags.add("expiry_without_practice_stand");
      }
      if (!standRef && practiceStand === "skip") flags.add("no_practice_stand");
      else if (!standRef) flags.add("no_accepted_reference");
      else if (!verified) flags.add("extension_unverifiable_view");
      if (standRef && refRecord && refRecord.postureWarning) flagAt("reference_posture_warning", refRecord.t0, refRecord.t1);

      // ---- status LAST ----
      let status;
      if (reason === "page_hidden" || reason === "camera_error") { status = "withheld"; withheld.unshift(reason); }
      else if (reason === "stopped") status = "stopped";
      else if (reason === "stream_end" && (tNow ?? 0) < tEnd) status = "incomplete";
      else if (withheld.length) status = "withheld";
      else if (flags.size) status = "needs_review";
      else status = "measured";
      const countedN = events.filter((e) => e.counted).length;
      const uncertainN = events.filter((e) => e.cls === "uncertain").length;
      const protocolFlags = [];
      if (armsSeen >= 10 && armsUncrossed / armsSeen > P.armsFlagFraction) protocolFlags.push("arms_possibly_uncrossed");
      if (armsSeen < 10) protocolFlags.push("arms_not_observable");

      result = {
        core: CORE_VERSION, reason, status,
        qualityVerdict: withheld.length ? "withheld" : flags.size ? "needs_review" : "ok",
        count: status === "measured" || status === "needs_review" ? countedN : null,
        countRange: status === "measured" || status === "needs_review" ? { min: countedN, max: countedN + uncertainN } : null,
        countSoFar: countedN,
        events, gaps, expiry,
        partials: events.filter((e) => e.cls === "partial"),
        withheldReasons: withheld, reviewFlags: [...flags], reviewNotes, protocolFlags, flagWindows,
        practiceStand: standRef ? "calibrated" : "skipped",
        extensionCheck: verified ? "verified_side_view" : standRef ? "unverifiable_view" : practiceStand === "skip" ? "no_practic
e_stand" : "no_accepted_reference",
        reference: refRecord ? { id: refRecord.id, t0: refRecord.t0, t1: refRecord.t1, h: r3(refRecord.h), knee: r1(refRecord.kne
e), hip: r1(refRecord.hip),
          viewRatio: r3(refRecord.viewRatio), shinPx: r1(refRecord.shinPx), sideView: refRecord.sideView, postureWarning: !!refRe
cord.postureWarning,
          lockGen: refRecord.lockGen, accepted: refRecord.accepted, usedAsReference: refRecord.usedAsReference } : null,
        calibration: { hSit: r3(hSit), stand: standRef ? { h: r3(standRef.h), knee: r1(standRef.knee), hip: r1(standRef.hip), vie
wRatio: r3(standRef.viewRatio), t0: standRef.t0, t1: standRef.t1 } : null,
          typicalSeatDuringTest: r3(typicalSeat), halfway: expiry.halfway ?? r3(hSit + ((standRef ? standRef.h : P.defaultStandRe
f) - hSit) / 2) },
        window: { tGo, tEnd, observedMs: obsMs },
        quality: {
          ...q, usableFps: +effFps.toFixed(1),
          missingObservationPct: +missingPct.toFixed(1),
          gapTimeBeyondTolerancePct: +((100 * beyondTolMs) / obsMs).toFixed(1), gapToleranceMs: P.gapToleranceMs,
          longestGapMs: Math.round(longestGap), armsFramesSeen: armsSeen, armsFramesUncrossed: armsUncrossed,
        },
        trace: trace.filter((r) => r.t >= tGo - 3000),
      };
      return result;
    },

    snapshot() {
      const last = trace[trace.length - 1];
      return {
        phase, count: events.filter((e) => e.counted).length, uncertain: events.filter((e) => e.cls === "uncertain").length, stat
e,
        elapsedMs: tGo !== null && last ? Math.max(0, Math.min(last.t, tEnd) - tGo) : 0,
```

```
        lastStatus: last ? last.s : null, lastH: last ? last.h : null, lastFrameT,
        readiness: phase === "calibrating" ? readiness() : null,
        participantPose, result,
      };
    },

    get trace() { return trace; },
    get hSit() { return hSit; },
  };
}

```

app/src/core/coverage.js

```
// Per-decision evidence coverage (checkpoint 4, audit B). Spec: docs/CONTRACTS_CP4.md section 2.
// Input: the core result and the timestamps (same clock as the core) of images that were ACTUALLY captured.
// Output: for each decision the observer can make, whether suitable images exist within stated bounds.
// An image elsewhere in the session never covers a decision it was not taken for.
export const COVERAGE_VERSION = "coverage-1";
export const COVERAGE_LIMITS = Object.freeze({
  referenceSlackMs: 150,     // image inside the practice-stand hold [t0-150, t1+150]
  anchorMaxMs: 250,          // image within 250 ms of a rise's top (tFull / tExpiry / tPeak)
  flagSlackMs: 200,          // flag window extended by 200 ms each side ...
  flagMaxGapMs: 500,         // ... and covered with no gap > 500 ms
  endpointMaxMs: 250,        // image within 250 ms of the 30-s mark
  scanMaxGapMs: 1000,        // whole-test images with no gap > 1 s (needed to look for missed stands)
});
export const RISE_RESOLVED_FLAGS = new Set(["uncertain_rises", "extension_unverifiable_view", "no_practice_stand", "no_accepted_reference", "expiry_without_practice_stand"]);

export function anchorTime(e) { return e.tFull ?? e.tExpiry ?? e.tPeak ?? e.tRise ?? null; }

function nearest(times, t) {            // times sorted ascending
  let lo = 0, hi = times.length - 1, best = null;
  while (lo <= hi) { const mid = (lo + hi) >> 1; if (times[mid] < t) lo = mid + 1; else hi = mid - 1; }
  for (const i of [lo - 1, lo]) if (i >= 0 && i < times.length && (best === null || Math.abs(times[i] - t) < Math.abs(best - t))) best = times[i];
  return best;
}
function spanCovered(times, a, b, maxGap) {
  const inside = times.filter((t) => t >= a && t <= b);
  const pts = [a, ...inside, b];
  let worst = 0; const gaps = [];
  for (let i = 1; i < pts.length; i++) { const g = pts[i] - pts[i - 1]; worst = Math.max(worst, g); if (g > maxGap) gaps.push([Math.round(pts[i - 1]), Math.round(pts[i])]); }
  return { ok: inside.length > 0 && worst <= maxGap, maxGapMs: Math.round(worst), images: inside.length, gaps };
}

export function computeCoverage(result, imageTimes, L = COVERAGE_LIMITS) {
  const times = [...(imageTimes || [])].filter((t) => Number.isFinite(t)).sort((a, b) => a - b);
  const out = { version: COVERAGE_VERSION, limits: { ...L }, imageCount: times.length, imageTimes: times.map((t) => Math.round(t * 1000) / 1000),
    reference: { ok: false }, events: {}, flags: {}, endpoint: { ok: false, required: false }, scan: { ok: false } };
  const ref = result.reference || (result.calibration && result.calibration.stand) || null;
  if (ref && Number.isFinite(ref.t0) && Number.isFinite(ref.t1)) {
    const inHold = times.filter((t) => t >= ref.t0 - L.referenceSlackMs && t <= ref.t1 + L.referenceSlackMs);
    out.reference = { ok: inHold.length > 0, images: inHold.length, imageT: inHold.length ? inHold[inHold.length >> 1] : null };
  }
  for (const e of result.events || []) {
    const a = anchorTime(e); const n = a === null ? null : nearest(times, a);
    const d = n === null ? null : Math.abs(n - a);
    out.events[e.n] = { ok: d !== null && d <= L.anchorMaxMs, anchorT: a, imageT: n, distMs: d === null ? null : Math.round(d),
      startImageT: e.tRise != null ? nearest(times, e.tRise) : null, endImageT: e.tSeat != null ? nearest(times, e.tSeat) : null };
  }
  const tGo = result.window.tGo, tEnd = result.window.tEnd;
  out.scan = { ...spanCovered(times, tGo, tEnd, L.scanMaxGapMs), from: tGo, to: tEnd };
  for (const f of result.reviewFlags || []) {
    if (RISE_RESOLVED_FLAGS.has(f)) { out.flags[f] = { ok: null, resolvedByRises: true }; continue; }
    const wins = (result.flagWindows && result.flagWindows[f]) || [];
    if (!wins.length) { out.flags[f] = { ok: out.scan.ok, windows: [], viaWholeTest: true }; continue; }   // session-wide flags need the whole test
    const ws = wins.map(([a, b]) => ({ t0: a, t1: b, ...spanCovered(times, a - L.flagSlackMs, b + L.flagSlackMs, L.flagMaxGapMs) }));
    out.flags[f] = { ok: ws.every((w) => w.ok), windows: ws };
  }
  const endpointRequired = (result.events || []).some((e) => e.cls === "expiry" || (e.flags || []).some((x) => /^expiry_/.test(x))) ||
    (result.reviewFlags || []).some((f) => /^expiry_/.test(f)) || (result.expiry && result.expiry.decision && !["none", "closed_descending_rise"].includes(result.expiry.decision));
  const en = nearest(times, tEnd);
  out.endpoint = { required: !!endpointRequired, ok: en !== null && Math.abs(en - tEnd) <= L.endpointMaxMs, imageT: en, distMs: en === null ? null : Math.round(Math.abs(en - tEnd)) };
  return out;
}

/** True if an observer-added stand's time is exactly an image time (it was picked from an image). */
export function hasImageAt(coverage, t) { return !!coverage && coverage.imageTimes.some((x) => Math.abs(x - t) <= 1); }

```

app/src/core/decision.js

```
// Session decision model (checkpoint 3): completion, count basis, evidence, CDC-table eligibility.
// Spec: docs/DECISION_RULES.md (written before this file; an independent oracle derived
// tests/fixtures/decision_matrix_expected.json from the spec + CDC form). One pure function used by the
// report, the review screen and the JSON export, so none of them can bypass a gate (audit B, C).
import { CDC_30S_BELOW_AVERAGE } from "./norms.js";
import { hasImageAt, RISE_RESOLVED_FLAGS } from "./coverage.js";
export const DECISION_VERSION = "decision-2 (checkpoint 4)";


const WINDOW_MS = 30000;
// Plain-language names for review items in blocker messages (overnight UX: no internal keys shown to users).
const FLAG_LABEL = {
  gap_during_movement: "a moment the participant could not be measured while moving",
  another_person_overlapped: "another person overlapping the participant",
  participant_reacquired_with_other_person_in_view: "the participant lost and found again while someone else was visible",
  identity_or_landmark_discontinuity: "a sudden jump of body points (possible person mix-up)",
  intermittent_tracking: "the participant measured in only part of the frames",
  seated_baseline_drift: "the participant sitting lower than at the camera check",
  expiry_endpoint_not_observed: "the participant not measured at the very end",
  reference_posture_warning: "the practice-stand (reference) posture warning: bent knees or hips",
  stand_dipped_without_sitting: "a deep dip while standing that came back up without sitting",
};
const flagLabel = (f) => FLAG_LABEL[f] || f.replace(/_/g, " ");
export const MIN_ADDED_SEPARATION_MS = 1000;   // [ENG] two added stands closer than this are treated as one stand added twice

/** Strict integer parsing (audit I): digits only, no sign, decimals, exponents, spaces. */
export function parseStrictInt(text, maxDigits, min, max) {
  if (typeof text === "number") text = String(text);
  if (typeof text !== "string" || !new RegExp(`^\\d{1,${maxDigits}}$`).test(text)) return null;
  const v = Number(text);
  return Number.isInteger(v) && v >= min && v <= max ? v : null;
}

export function cdcCategory(count, ageText, column) {
  const age = parseStrictInt(ageText, 3, 0, 999);
  if (age === null) return { ok: false, reason: "age not entered as a whole number" };
  if (column !== "men" && column !== "women") return { ok: false, reason: "table column (men/women) not entered" };
  const row = CDC_30S_BELOW_AVERAGE.find((r) => age >= r.min && age <= r.max);
  if (!row) return { ok: false, reason: `the CDC table covers ages 60-94 only (entered age ${age})` };
  if (!Number.isInteger(count) || count < 0) return { ok: false, reason: "no valid count" };
  const threshold = row[column];
  return { ok: true, category: count < threshold ? "below_average" : "not_below_average", threshold, band: `${row.min}-${row.max}, ${column}` };
}

export function completionOf(result, observer) {
  if (observer && observer.arms === "yes") return "protocol_stop_arms";                       // R1 (CDC arm-use rule)
  const r = result.reason;
  // 0.5-ms guard: (tGo + 30000) - tGo can be 29999.999999999996 in floating point (checkpoint-4 diagnostic)
  if (r === "expired") return result.window && result.window.observedMs >= WINDOW_MS - 0.5 ? "completed_30s" : "incomplete";
  if (r === "stopped") return "stopped_early";
  if (r === "stream_end") return "incomplete";
  return "interrupted";                                                                        // page_hidden, camera_error, unknown
}

/**
 * v2 (docs/CONTRACTS_CP4.md): every observer decision is honoured only where its own evidence exists (coverage.js);
 * corrections work in both directions (partial -> full, observer-added missed stands); the supervisor count is a
 * separate basis that is used only by explicit choice; a conflict between eligible automatic and supervisor counts
 * yields no value until the observer chooses a basis with a rationale.
 */
export function decide(session) {
  const { result, source, observer = {}, evidence = {}, manual = null, participant = {}, basisChoice = null } = session;
  const completion = completionOf(result, observer);
  const blockers = [];
  // images exist if the coverage lists any (contract s6 shape: imageTimes; imageCount is optional)
  const nImg = (c) => (c ? (Number.isFinite(c.imageCount) ? c.imageCount : (c.imageTimes || []).length) : 0);
  const cov = evidence.available === true && evidence.coverage && nImg(evidence.coverage) > 0 ? evidence.coverage : null;
  const evCovered = (n) => !!(cov && cov.events[n] && cov.events[n].ok);
  const res = evidence.eventResolutions || {};
  const resOf = (n) => (evCovered(n) ? res[n] : undefined);                                   // resolutions need that rise's image
  const events = result.events || [];
  // Observer-added missed stands (overnight fix ON-02, docs/CONTRACTS_CP4.md s9): each must sit exactly at a kept image,
  // inside the 30-s window, outside every detected rise (a detected rise is corrected on its own card, never re-added)
  // and at least MIN_ADDED_SEPARATION_MS from another added stand. Anything else is not counted AND blocks interpretation.
  const addedOk = [], addedProblems = [], addedReview = [];
  if (cov) {
    const tGo = result.window.tGo, tEnd = result.window.tEnd, at = (t) => ((t - tGo) / 1000).toFixed(1);
    const list = [...(evidence.added || [])].sort((a, b) => ((a && a.t) ?? 0) - ((b && b.t) ?? 0));
    const bad = (a, msg) => { addedProblems.push(msg); addedReview.push({ id: a && a.id, t: a && a.t, counted: false, problem: msg }); };
    for (const a of list) {
```

```
      if (!a || !Number.isFinite(a.t)) { bad(a, "An added stand has no valid time."); continue; }
      if (!hasImageAt(cov, a.t)) { bad(a, `The added stand at ${at(a.t)} s is not at a kept image.`); continue; }
      if (a.t < tGo || a.t > tEnd) { bad(a, `The added stand at ${at(a.t)} s is outside the 30-second test.`); continue; }
      const inRise = events.find((e) => Number.isFinite(e.tRise) && a.t >= e.tRise && a.t <= (e.tSeat ?? e.tExpiry ?? tEnd));
      if (inRise) { bad(a, `The added stand at ${at(a.t)} s is inside detected rise ${inRise.n}: undo it and decide on rise ${inRise.n} instead.`); continue; }
      const prev = addedOk[addedOk.length - 1];
      if (prev && a.t - prev.t < MIN_ADDED_SEPARATION_MS) { bad(a, `The added stands at ${at(prev.t)} s and ${at(a.t)} s are less than ${MIN_ADDED_SEPARATION_MS / 1000} s apart (the same stand added twice?): undo one.`); continue; }
      addedOk.push(a); addedReview.push({ id: a.id, t: a.t, counted: true, problem: null });
    }
  } else for (const a of evidence.added || []) addedReview.push({ id: a && a.id, t: a && a.t, counted: false, problem: "No evidence images: added stands cannot be used." });
  const added = addedOk;
  const countable = completion === "completed_30s" && result.status !== "withheld";          // (ON-30) nothing is counted otherwise
  if (!countable) for (const a of addedReview) if (a.counted) { a.counted = false; a.problem = "Not counted: this was not a completed, measured 30-second test."; }

  // ---- automatic range (completed, not withheld) ----
  let automatic = null;
  if (completion === "completed_30s" && result.status !== "withheld") {
    let min = 0, max = 0;
    for (const e of events) {
      const r = resOf(e.n);
      if (e.counted) { if (r === "not_full") continue; if (r === "cant_tell") { max++; continue; } min++; max++; }
      else if (e.cls === "uncertain") { if (r === "full") { min++; max++; } else if (r !== "not_full") max++; }
      else if (e.cls === "partial") { if (r === "full") { min++; max++; } else if (r === "cant_tell") max++; }
    }
    min += added.length; max += added.length;
    automatic = { min, max, determinate: min === max, added: added.length, eligible: false, blockers: [] };

    // ---- evidence gate for interpreting the automatic count ----
    const ab = automatic.blockers;
    ab.push(...addedProblems);
    if (!cov) ab.push("No evidence images were kept, so the automatic count cannot be checked afterwards.");
    else {
      const ref = result.reference;
      if (!ref || !ref.accepted || ref.usedAsReference === false) ab.push("No practice stand was accepted as the full-standing reference.");
      else if (ref.accepted.basis !== "image") ab.push("The practice-stand reference was accepted without looking at its image, so it cannot be used to check the automatic count.");
      else if (!cov.reference || !cov.reference.ok) ab.push("There is no image of the practice stand, so the reference cannot be checked.");
      const needId = events.filter((e) => e.counted || e.cls === "uncertain" || resOf(e.n) === "full").map((e) => e.n);
      const missing = needId.filter((n) => !evCovered(n));
      if (missing.length) ab.push(`No image near the top of rise ${missing.join(", ")}: those rises cannot be checked.`);
      const confirmedFor = new Set(evidence.identityConfirmedFor || []);
      // (overnight ON-12: with no rise that needs checking - e.g. a genuine automatic 0 - there is nothing to confirm identity on;
      // the whole-test scan below still has to be done)
      // (overnight ON-23, decided: an observer-added stand is itself the observer's judgement on that image - "a stand by the
      // participant happens here" - so it does not need a separate identity answer; the add control says so explicitly)
      if (needId.length && (evidence.identityConfirmed !== true || needId.some((n) => !confirmedFor.has(n)))) ab.push("The identity of the measured person was not confirmed in the images of every rise.");
      const uncertain = events.filter((e) => e.cls === "uncertain");
      const risesDecided = uncertain.every((e) => resOf(e.n) === "full" || resOf(e.n) === "not_full");
      for (const f of result.reviewFlags || []) {
        if (RISE_RESOLVED_FLAGS.has(f)) { if (!risesDecided) ab.push("Section 3: decide every rise marked \"needs your decision\" (full stand or not; at 30 s: over halfway or not)."); continue; }
        const fc = cov.flags[f];
        if (!fc || !fc.ok) ab.push(`Section 5: ${flagLabel(f)} - no continuous images for that time, so it cannot be checked.`);
        else if ((evidence.flagChecks || {})[f] !== "ok") ab.push(`Section 5: check ${flagLabel(f)} in its images.`);
      }
      // the 30-s endpoint matters if any expiry event/flag exists (derived from the result, not trusted from the input)
      const endpointRelevant = events.some((e) => e.cls === "expiry" || (e.flags || []).some((x) => /^expiry_/.test(x))) ||
        (result.reviewFlags || []).some((f) => /^expiry_/.test(f)) || !!(result.expiry && result.expiry.decision && !["none", "closed_descending_rise"].includes(result.expiry.decision));
      if (endpointRelevant && !(cov.endpoint && cov.endpoint.ok)) ab.push("No image at the 30-second mark, so the final (halfway) decision cannot be checked.");
      if (!cov.scan.ok) ab.push(`Images are missing between ${cov.scan.gaps.map(([a, b]) => `${((a - result.window.tGo) / 1000).toFixed(1)}-${((b - result.window.tGo) / 1000).toFixed(1)} s`).join(", ")}: missed stands cannot be ruled out.`);
      else if (evidence.sessionScan !== "no_missed") ab.push(evidence.sessionScan === "missed" ? "You reported a missed stand that has not been added from the images." : "The whole-test images were not checked for missed stands.");
    }
    automatic.eligibleExceptRange = ab.length === 0;
    if (!automatic.determinate) ab.push(`${automatic.max - automatic.min} rise(s) are unresolved (count ${automatic.min}-${automatic.max}).`);
    automatic.eligible = ab.length === 0;
  }

  // ---- supervisor count (separate basis) ----
  let supervisor = { valid: false, value: null };
  if (manual) {
    const v = parseStrictInt(manual.text, 2, 0, 60);
    const ok = v !== null && typeof manual.reason === "string" && manual.reason.trim().length > 0;
    supervisor = { valid: ok, value: ok ? v : null, reason: manual.reason };
```

```
  }
  let choice = basisChoice && (basisChoice.basis === "automatic" || basisChoice.basis === "supervisor") && typeof basisChoice.rationale === "string" && basisChoice.rationale.trim() ? basisChoice.basis : null;
  // overnight fix ON-03: a choice records the counts the observer saw ("at"); if either count changed since, the choice is
  // stale and ignored (the observer must choose again). Callers that omit "at" (pre-overnight tests) are not checked.
  let staleChoice = false;
  if (choice && basisChoice.at) {
    const a = basisChoice.at;
    staleChoice = (a.automaticMin ?? null) !== (automatic ? automatic.min : null) || (a.automaticMax ?? null) !== (automatic ? automatic.max : null) || (a.supervisor ?? null) !== supervisor.value;
    if (staleChoice) choice = null;
  }
  // conflict: the automatic basis is usable (apart from an open range) and a valid supervisor count differs from it, or the
  // automatic value is still a range (oracle v2 case G03: an open range is not silently decided while a supervisor count exists)
  const conflict = completion === "completed_30s" && !!automatic && automatic.eligibleExceptRange && supervisor.valid &&
    (!automatic.determinate || supervisor.value !== automatic.min);

  // ---- value ----
  let value = null, basis = "", valueText = "Not reported", rangeCategory = false;
  if (completion === "protocol_stop_arms") { value = 0; basis = "CDC rule: arms used to stand -> record 0 (observer answer)"; }
  else if (completion !== "completed_30s") { basis = "No score: not a completed 30-second test"; }
  else if (choice === "supervisor" && supervisor.valid && source === "camera") { value = supervisor.value; basis = "Supervisor's own count during a completed 30-s timed test (chosen by the observer; not image-verified)"; }
  else if (conflict && choice !== "automatic") { basis = `Counts disagree: camera (checked) ${automatic.min}, supervisor ${supervisor.value}. Choose which count to report.`; blockers.push("The camera count and your own count differ: choose which one to report, with a reason."); }
  else if (automatic && automatic.eligible) { value = automatic.min; basis = "Automatic count, checked by the observer against the evidence images" + (automatic.added ? `, including ${automatic.added} stand(s) the observer added from images` : "") + (supervisor.valid && supervisor.value === automatic.min ? " (the supervisor's own count agrees)" : conflict ? " (chosen over the differing supervisor count)" : ""); }
  else if (automatic) basis = "Automatic count not usable yet (see blockers)" + (supervisor.valid ? "; your own count is recorded but not chosen as the result" : "");
  else basis = supervisor.valid ? "Camera measurement withheld; your own count is recorded but not chosen as the result" : "Camera measurement withheld and no supervisor count chosen";
  if (staleChoice) blockers.push("The counts changed after you chose which one to report: choose again.");
  if (value !== null) valueText = String(value);
  else if (automatic && completion === "completed_30s") valueText = automatic.determinate ? `${automatic.min} (unchecked)` : `${automatic.min}–${automatic.max} (unresolved)`;

  // ---- CDC table eligibility (R7) ----
  if (completion !== "completed_30s" && completion !== "protocol_stop_arms") blockers.push("This was not a completed 30-second test.");
  if (source !== "camera") blockers.push("Replay of a video file: not a live test.");
  if (observer.setup !== "yes") blockers.push("Chair and helper setup not confirmed by the observer.");
  if (completion === "completed_30s" && observer.arms !== "no") blockers.push("Arm use not answered by the observer.");
  const ageV = parseStrictInt(participant.ageText, 3, 0, 999);
  if (ageV === null) blockers.push(String(participant.ageText ?? "").trim() === "" ? "Age not entered." : "Age not entered as a whole number.");
  else if (ageV < 60 || ageV > 94) blockers.push(`The CDC table covers ages 60–94 only (entered age ${ageV}).`);
  if (participant.column !== "men" && participant.column !== "women") blockers.push("Table column (men/women) not entered.");

  let category = null, threshold = null, band = null;
  // (overnight ON-11: after an explicit "camera count" choice with an open range, the range rule or the range blocker applies —
  // previously this path produced no value AND an empty blocker list)
  if (completion === "completed_30s" && value === null && (!conflict || choice === "automatic")) {
    if (automatic && !automatic.determinate && automatic.eligibleExceptRange && (!supervisor.valid || choice === "automatic")) {
      const a = cdcCategory(automatic.min, participant.ageText, participant.column), b = cdcCategory(automatic.max, participant.ageText, participant.column);
      if (a.ok && b.ok && a.category === b.category) { rangeCategory = true; category = a.category; threshold = a.threshold; band = a.band; valueText = `${automatic.min}–${automatic.max}`;
        basis = "Automatic count still a range (some rises undecided); both ends fall in the same CDC table category"; }
      else blockers.push(...automatic.blockers);
    } else if (automatic) {
      blockers.push(...automatic.blockers);
      if (supervisor.valid && choice !== "automatic") blockers.push("To report your own count instead, choose it explicitly with a reason.");
    } else if (supervisor.valid) blockers.push("To report your own count, choose it explicitly with a reason.");
    else blockers.push("No valid count (camera measurement withheld and no supervisor count).");
  }
  if (value !== null && blockers.length === 0) {
    const c = cdcCategory(value, participant.ageText, participant.column);
    if (c.ok) { category = c.category; threshold = c.threshold; band = c.band; } else blockers.push(c.reason);
  }
  const eligible = blockers.length === 0 && (value !== null || rangeCategory);
  if (!eligible) { category = null; threshold = null; band = null; }
  const title = completion === "completed_30s" || completion === "protocol_stop_arms"
    ? "30-Second Chair Stand — result" : "Chair-stand session — NOT a completed 30-second test";
  const machineCount = events.filter((e) => e.counted).length;
  return { version: DECISION_VERSION, completion, value, valueText, basis, automatic, supervisor, conflict, basisChoice: choice, staleChoice, addedReview,
    counts: { machine: completion === "completed_30s" && result.status !== "withheld" ? machineCount : null, reviewed: automatic ? automatic.min : null, supervisor: supervisor.value },
    manualValid: supervisor.valid, manualValue: supervisor.value, manualUsed: choice === "supervisor" && value === supervisor.value && supervisor.valid,
    reference: { eligible, blockers: [...new Set(blockers)], category, threshold, band, fromRange: rangeCategory }, title };
}

```

app/src/core/norms.js

```
// CDC STEADI 30-Second Chair Stand "Below Average Scores" table (CDC 2017 assessment form, page 1).
// A score BELOW the listed value is "below average", which the form says "indicates a risk for falls".
// The table covers ages 60-94 only. Nothing is extrapolated outside it, and age/sex are NEVER inferred
// from video: they must be entered by the observer.
export const CDC_30S_BELOW_AVERAGE = Object.freeze([
  { min: 60, max: 64, men: 14, women: 12 },
  { min: 65, max: 69, men: 12, women: 11 },
  { min: 70, max: 74, men: 12, women: 10 },
  { min: 75, max: 79, men: 11, women: 10 },
  { min: 80, max: 84, men: 10, women: 9 },
  { min: 85, max: 89, men: 8, women: 8 },
  { min: 90, max: 94, men: 7, women: 4 },
]);

/**
 * @param count   final chair-stand number (must come from a measured or observer-reviewed result)
 * @param age     observer-entered integer age, or null
 * @param sex     'men' | 'women' | null (the CDC table's categories)
 * @returns {applies:boolean, reason?:string, belowAverage?:boolean, threshold?:number, band?:string}
 */
export function cdcReference(count, age, sex) {
  if (count === null || count === undefined || !Number.isInteger(count)) return { applies: false, reason: "no valid count" };
  if (age === null || age === undefined || !Number.isInteger(age)) return { applies: false, reason: "age not entered" };
  if (sex !== "men" && sex !== "women") return { applies: false, reason: "sex not entered (table lists men / women)" };
  const row = CDC_30S_BELOW_AVERAGE.find((r) => age >= r.min && age <= r.max);
  if (!row) return { applies: false, reason: `the CDC table covers ages 60-94 only (entered age ${age})` };
  const threshold = row[sex];
  return { applies: true, belowAverage: count < threshold, threshold, band: `${row.min}-${row.max}, ${sex}` };
}

```

app/src/core/params.js

```
// All tunable thresholds for the chair-stand core, in one place.
// History: 0.5.0 frozen for holdout-2 (tag core-freeze-v0.5.0); 0.5.1/0.5.2 post-hoc review guards;
// 0.6.0 (checkpoint 3) = audit repairs: practice-stand calibration, full-vs-partial classification, CDC position
// rule at expiry, readiness continuity, honest quality accounting, status computed after all event flags.
// Provenance tags: [CDC] official form; [DATA] measured on the 14 clean DEVELOPMENT clips (not a holdout);
// [ENG] engineering assumption, to be revisited with real 30-s recordings.
export const CORE_VERSION = "chairstand-core 0.8.5";

export const PARAMS = Object.freeze({
  // --- protocol ---
  windowMs: 30000,               // [CDC] "Repeat this for 30 seconds"; timing starts on "Go"

  // --- landmark validity ---
  minVisibility: 0.5,            // [ENG] hip, knee, ankle (and shoulder for the hip angle) visibility needed
  shinJumpTolerance: 0.35,       // [ENG] side rejected if shin length deviates >35% from the running reference
  hPlausibleMin: -0.4,           // [ENG] h = (knee_y - hip_y) / shin_length; outside this range = landmark swap
  hPlausibleMax: 1.9,
  smoothFrames: 5,               // [ENG] causal trailing median over the last N usable frames (no look-ahead)

  // --- readiness: seated calibration (continuous, recent observations) ---
  calibrationMs: 1000,           // [ENG] seated and still for 1 s before "Go"
  calibrationMaxSpread: 0.15,    // [ENG] max (p90 - p10) of h while "still"
  seatedMaxH: 0.85,              // [ENG] calibration h must be below this to count as seated
  readinessMaxGapMs: 250,        // [ENG] no gap between usable frames above this inside a readiness window (audit G)
  readinessMinUsableFrac: 0.85,  // [ENG] >= 85% of frames in the window must be usable (one dropped frame is tolerated)
  maxStartStaleMs: 300,          // [ENG] start() refused if the last frame is older than this at "Go" (audit G)

  // --- practice stand = per-person standing reference (audit A) ---
  // [precedent] Rikli & Jones protocol sheet: "practice one or two repetitions to ensure proper form".
  standHoldMs: 600,              // [ENG] a practice stand must be held still at least this long
  standHoldMinRise: 0.40,        // [ENG] ... with h at least this far above the seated level
  standHoldMaxSpreadH: 0.10,     // [ENG] stillness during the hold
  standHoldMaxSpreadDeg: 8,      // [ENG]
  sideViewMaxShoulderRatio: 0.55, // [ENG] shoulder width / torso length at the practice stand; above this the camera is
                                 //       too frontal (> ~40 deg off side) for knee/hip angles to show flexion -> unverifiable
  // --- reference acceptance (checkpoint 4; docs/CONTRACTS_CP4.md s1) ---
  // [DATA] posture warning when the accepted reference lies OUTSIDE the range of genuine standing postures observed in the
  // 14 clean development clips (per-clip standing-plateau medians: knee min 160.3 deg, hip min 168.4 deg). A warning is
  // not a rejection: it adds a review item that must be checked on the reference image. (cp4: 150 deg for both was an
  // unanchored first guess and missed a 30-deg hip flexion reference in the audit adapter.)
  referenceWarnKneeDeg: 158,
  referenceWarnHipDeg: 165,
  referenceMaxAgeMs: 300000,     // [ENG] an accepted reference older than 5 min at GO is stale
  referenceMaxShinChange: 0.20,  // [ENG] seated shin length vs reference shin length > 20% -> camera/person changed

  // --- full stand vs partial rise, relative to the participant's own practice stand ---
  // [DATA] 95th percentile of genuine full stands' shortfall vs the same person's best stand, 14 clean dev clips
  // (73 labelled full stands): knee 7.0 deg, hip 3.8 deg, h 0.12. By construction ~5% of genuine full stands per
  // criterion fall into "uncertain" (review), which is the price of not certifying incomplete rises.
  fullKneeDeg: 7,
  fullHipDeg: 4,
  fullH: 0.12,
  // A rise is silently "partial" (not counted, no review) only with strong evidence: shortfall beyond TWICE the band
  // in hip height, or beyond twice the band in BOTH knee and hip. Everything between is "uncertain" -> review.
  partialFactor: 2,
  standConfirmFrames: 2,         // [ENG] consecutive (smoothed) frames meeting the full-stand criteria
  // Fallback when extension cannot be verified (no practice stand / frontal view): old hip-height candidate rule;
  // such events are never counted automatically - they are "uncertain" and need review.
  standAbsMin: 0.85,
  standRiseMin: 0.30,
  seatReturnRise: 0.20,          // [ENG] back to seated when h <= hSit + this (hysteresis)
  partialRiseMin: 0.25,          // [ENG] a movement that exceeds hSit + this is a rise attempt
  perchRiseMin: 0.15,            // [ENG] 0.8 (ON-06): after a "perched" sit (descent below halfway that never reached the seat l
evel), a rise of this much starts a new attempt (never auto-counted)
  minAngleFrames: 3,             // [ENG] 0.8.2 (ON-26): a smoothed joint angle needs this many of the smoothFrames buffered fram
es (and the current frame)
  refMinAngleFrac: 0.5,          // [ENG] 0.8.2/0.8.3 (ON-27/33): a practice-stand window supplies joint angles only if this frac
tion of its frames (and >= minAngleFrames) has them
  refTopBandDeg: 3,              // [ENG] 0.8.3 (ON-32): reference = median window among those within this (knee+hip, deg) of the
most extended
  standEndMs: 150,               // [ENG] 0.8.2 (ON-28): the practice stand has ended after this long below its end level
  dipFlagH: 0.25,                // [ENG] 0.8.2 (ON-29): while standing, a dip deeper than this below the stand's peak that comes
back up is flagged
  peakBandH: 0.03,               // [ENG] 0.8.1/0.8.2 (ON-20/26): raw frames within this of a rise's (90th-percentile) highest ra
w h count as "at the top" for angle evidence
  maxPartialSmoothSpanMs: 300,   // [ENG] 0.8 (ON-08): a rise may be classed "partial" (not counted, no review) only if the smoot
hing window spans <= this (fps >= ~13)
  defaultStandRef: 1.10,         // [ENG] standing h used for the halfway point only if no practice stand (-> review)

  // --- 30-s expiry: CDC position rule ---
  // [CDC] "If the patient is over halfway to a standing position when 30 seconds have elapsed, count it as a stand."
  // Halfway = midpoint in hip height between the seated calibration and the practice stand [ENG interpretation].
  // No velocity condition (removed in 0.6.0, audit D). Near the midpoint, or on the way down without ever having
  // stood, the decision is left to the observer (review).
```

```
  expiryNearHalfBand: 0.08,      // [ENG] |h - halfway| within this -> review

  // --- identity / multiple people ---
  trackMaxJump: 0.6,             // [ENG] participant torso centre may move <= 0.6 torso lengths between usable frames
  trackJumpPerSec: 1.5,          // [ENG] ... plus 1.5 torso lengths per second since the last lock
  trackJumpCap: 1.5,             // [ENG] capped; two candidates inside the radius -> ambiguous frame (not measured)
  minRelativeSize: 0.5,          // [ENG] poses < 50% of the participant's torso size are ignored
  overlapIoU: 0.25,              // [ENG] another person's box overlapping the participant's box -> overlap frame
  jumpBase: 0.3,                 // [ENG] raw h may change by at most jumpBase + jumpPerSec * dt between usable frames
  jumpPerSec: 4.0,               //       (a real rise takes >= ~0.35 s); larger jumps = identity/landmark discontinuity
  otherPersonRecentMs: 2000,     // [ENG] a lost-then-found participant with someone else in view within this -> review

  // --- measurement quality ---
  maxGapMs: 700,                 // [ENG] any stretch without a usable participant frame longer than this -> withheld
  maxUnusableFraction: 0.2,      // [ENG] > 20% of the window in gaps beyond the per-gap tolerance -> withheld
  gapToleranceMs: 150,           // [ENG] per-gap allowance used for the line above (reported separately, audit H)
  intermittentMissingPct: 30,    // [ENG] > 30% of frames without a usable participant observation -> review
  minEffectiveFps: 8,            // [ENG] usable frames per second over the window
  reviewGapMs: 250,              // [ENG] shorter gaps are tolerated; longer ones during movement -> review
  movementGapRise: 0.20,

  // --- arms (observer-confirmed; the app only raises a note) ---
  armsFarFactor: 0.9,
  armsFlagFraction: 0.3,
});

```

app/src/core/pose_features.js

```
// Per-frame geometric features from BlazePose landmarks (33 points).
// Landmarks arrive as [x, y, z, visibility, presence] with x, y normalised to image width/height.
// Everything is converted to pixels first so distances are isotropic.
import { PARAMS } from "./params.js";

export const LM = {
  L_SHOULDER: 11, R_SHOULDER: 12, L_WRIST: 15, R_WRIST: 16,
  L_HIP: 23, R_HIP: 24, L_KNEE: 25, R_KNEE: 26, L_ANKLE: 27, R_ANKLE: 28,
};
const SIDES = [
  { hip: LM.L_HIP, knee: LM.L_KNEE, ankle: LM.L_ANKLE },
  { hip: LM.R_HIP, knee: LM.R_KNEE, ankle: LM.R_ANKLE },
];

const px = (lm, i, w, h) => [lm[i][0] * w, lm[i][1] * h];
const dist = (a, b) => Math.hypot(a[0] - b[0], a[1] - b[1]);
const vis = (lm, i) => lm[i][3] ?? 0;
const inFrame = (lm, i) => lm[i][0] >= 0 && lm[i][0] <= 1 && lm[i][1] >= 0 && lm[i][1] <= 1;

/** Torso centre, torso length and a loose bounding box, used for identity tracking. */
export function torsoGeometry(lm, w, h) {
  const ls = px(lm, LM.L_SHOULDER, w, h), rs = px(lm, LM.R_SHOULDER, w, h);
  const lh = px(lm, LM.L_HIP, w, h), rh = px(lm, LM.R_HIP, w, h);
  const centre = [(ls[0] + rs[0] + lh[0] + rh[0]) / 4, (ls[1] + rs[1] + lh[1] + rh[1]) / 4];
  const size = (dist(ls, lh) + dist(rs, rh)) / 2;
  let x0 = Infinity, y0 = Infinity, x1 = -Infinity, y1 = -Infinity;
  for (const p of lm) {
    if ((p[3] ?? 0) < 0.3) continue;
    x0 = Math.min(x0, p[0] * w); y0 = Math.min(y0, p[1] * h);
    x1 = Math.max(x1, p[0] * w); y1 = Math.max(y1, p[1] * h);
  }
  return { centre, size, box: [x0, y0, x1, y1] };
}

/**
 * Normalised hip height h = (knee_y - hip_y) / shin_length for each usable leg side.
 * Seated: hip near knee level (value depends on camera angle). Full stand: thigh vertical, h ~ 1.0–1.3.
 * shinRef (running reference, px) rejects landmark swaps; returns null when no side is usable.
 */
export function hipHeight(lm, w, h, shinRef) {
  const vals = [];
  for (const s of SIDES) {
    if (vis(lm, s.hip) < PARAMS.minVisibility || vis(lm, s.knee) < PARAMS.minVisibility ||
        vis(lm, s.ankle) < PARAMS.minVisibility) continue;
    if (!inFrame(lm, s.hip) || !inFrame(lm, s.knee) || !inFrame(lm, s.ankle)) continue;
    const hip = px(lm, s.hip, w, h), knee = px(lm, s.knee, w, h), ankle = px(lm, s.ankle, w, h);
    const shin = dist(knee, ankle);
    if (shin < 1) continue;
    if (shinRef && Math.abs(shin - shinRef) / shinRef > PARAMS.shinJumpTolerance) continue;
    const hv = (knee[1] - hip[1]) / shin;
    if (hv < PARAMS.hPlausibleMin || hv > PARAMS.hPlausibleMax) continue;
    vals.push({ h: hv, shin, v: Math.min(vis(lm, s.hip), vis(lm, s.knee), vis(lm, s.ankle)) });
  }
  if (!vals.length) return null;
  // Use the better-seen side (averaging a good side with a swapped one produced the dev-set spikes).
  vals.sort((a, b) => b.v - a.v);
  if (vals.length === 2 && Math.abs(vals[0].h - vals[1].h) < 0.25) {
    return { h: (vals[0].h + vals[1].h) / 2, shin: (vals[0].shin + vals[1].shin) / 2, sides: 2 };
  }
  return { h: vals[0].h, shin: vals[0].shin, sides: 1 };
}

/**
 * Arms-crossed heuristic: each wrist near the opposite shoulder.
 * Returns true (crossed), false (look uncrossed) or null (wrists not observable).
 * This only raises a flag for the observer; it never certifies protocol compliance.
 */
export function armsCrossed(lm, w, h, torsoSize) {
  if (vis(lm, LM.L_WRIST) < PARAMS.minVisibility || vis(lm, LM.R_WRIST) < PARAMS.minVisibility) return null;
  const lw = px(lm, LM.L_WRIST, w, h), rw = px(lm, LM.R_WRIST, w, h);
  const ls = px(lm, LM.L_SHOULDER, w, h), rs = px(lm, LM.R_SHOULDER, w, h);
  const far = PARAMS.armsFarFactor * torsoSize;
  return dist(lw, rs) <= far && dist(rw, ls) <= far;
}

export function iou(a, b) {
  const ix = Math.max(0, Math.min(a[2], b[2]) - Math.max(a[0], b[0]));
  const iy = Math.max(0, Math.min(a[3], b[3]) - Math.max(a[1], b[1]));
  const inter = ix * iy;
  const ua = (a[2] - a[0]) * (a[3] - a[1]) + (b[2] - b[0]) * (b[3] - b[1]) - inter;
  return ua > 0 ? inter / ua : 0;
}

const angleDeg = (a, b, c) => {
```

```
  const v1 = [a[0] - b[0], a[1] - b[1]], v2 = [c[0] - b[0], c[1] - b[1]];
  const d = Math.hypot(...v1) * Math.hypot(...v2);
  return d > 0 ? (Math.acos(Math.max(-1, Math.min(1, (v1[0] * v2[0] + v1[1] * v2[1]) / d))) * 180) / Math.PI : NaN;
};

/**
 * Leg/trunk geometry for full-stand classification (checkpoint 3).
 * Uses the better-seen side. Returns h (as hipHeight), 2-D knee angle (hip-knee-ankle), 2-D hip angle
 * (shoulder-hip-knee) and the view ratio (shoulder width / torso length: small = side view).
 * Angles are only meaningful in a side-ish view; the core decides that from the practice stand.
 */
export function legGeometry(lm, w, h, shinRef) {
  const hh = hipHeight(lm, w, h, shinRef);
  if (!hh) return null;
  const sideOf = (s) => Math.min(vis(lm, s.hip), vis(lm, s.knee), vis(lm, s.ankle), vis(lm, s.hip === LM.L_HIP ? LM.L_SHOULDER :
LM.R_SHOULDER));
  const s = sideOf(SIDES[0]) >= sideOf(SIDES[1]) ? SIDES[0] : SIDES[1];
  const sh = s.hip === LM.L_HIP ? LM.L_SHOULDER : LM.R_SHOULDER;
  let knee = NaN, hip = NaN;
  if (sideOf(s) >= PARAMS.minVisibility) {
    knee = angleDeg(px(lm, s.hip, w, h), px(lm, s.knee, w, h), px(lm, s.ankle, w, h));
    hip = angleDeg(px(lm, sh, w, h), px(lm, s.hip, w, h), px(lm, s.knee, w, h));
  }
  const ls = px(lm, LM.L_SHOULDER, w, h), rs = px(lm, LM.R_SHOULDER, w, h);
  const lh = px(lm, LM.L_HIP, w, h), rh = px(lm, LM.R_HIP, w, h);
  const torso = Math.hypot((ls[0] + rs[0] - lh[0] - rh[0]) / 2, (ls[1] + rs[1] - lh[1] - rh[1]) / 2);
  const viewRatio = torso > 0 ? dist(ls, rs) / torso : NaN;
  return { h: hh.h, shin: hh.shin, knee, hip, viewRatio };
}

```

app/src/core/tracker.js

```
// Keeps measuring the SAME person (the seated participant) when a helper or bystander is visible.
// Rule: follow the pose whose torso centre continues the participant's recent position.
// If two poses are both plausible continuations, the frame is "ambiguous" and not used for measurement.
import { PARAMS } from "./params.js";
import { torsoGeometry, iou } from "./pose_features.js";

export class ParticipantTracker {
  constructor(width, height, params = PARAMS) {
    this.w = width; this.h = height; this.p = params;
    this.centre = null; this.size = null; this.box = null; this.sizes = []; this.lockT = null;
    this.lockGen = 0;                       // increments on every NEW acquisition (a reference is bound to it)
  }

  get locked() { return this.centre !== null; }

  reset() { this.centre = null; this.size = null; this.box = null; this.sizes = []; }
  acquire(geom, t) { this.lockGen += 1; this.lock(geom, t); }

  lock(geom, t = null) {
    this.lockT = t;
    this.centre = geom.centre; this.box = geom.box;
    this.sizes.push(geom.size); if (this.sizes.length > 30) this.sizes.shift();
    const s = [...this.sizes].sort((a, b) => a - b); this.size = s[s.length >> 1];
  }

  /**
   * @param poses array of landmark arrays
   * @param chooseInitial optional fn(candidates) -> index, used only while not yet locked
   * @returns {status: 'ok'|'none'|'lost'|'ambiguous'|'unlocked', pose, geom, others, overlap}
   */
  update(poses, chooseInitial = null, t = null) {
    const geoms = poses.map((lm) => torsoGeometry(lm, this.w, this.h));
    if (!this.locked) {
      if (!poses.length) return { status: "none", pose: null, others: 0, overlap: false };
      const idx = chooseInitial ? chooseInitial(poses, geoms) : (poses.length === 1 ? 0 : -1);
      if (idx < 0) return { status: "unlocked", pose: null, others: poses.length, overlap: false };
      this.acquire(geoms[idx], t);
      return { status: "ok", pose: poses[idx], geom: geoms[idx], others: poses.length - 1, overlap: false };
    }
    const cands = [];
    geoms.forEach((g, i) => {
      if (g.size < this.p.minRelativeSize * this.size) return;   // distant or spurious detection
      cands.push({ i, g, d: Math.hypot(g.centre[0] - this.centre[0], g.centre[1] - this.centre[1]) / this.size });
    });
    if (!cands.length) return { status: poses.length ? "lost" : "none", pose: null, others: poses.length, overlap: false };
    cands.sort((a, b) => a.d - b.d);
    const best = cands[0];
    const dt = t !== null && this.lockT !== null ? Math.max(0, (t - this.lockT) / 1000) : 0;
    const radius = Math.min(this.p.trackJumpCap, this.p.trackMaxJump + this.p.trackJumpPerSec * dt);
    if (best.d > radius) return { status: "lost", pose: null, others: poses.length, overlap: false };
    if (cands.length > 1 && cands[1].d <= radius) {
      return { status: "ambiguous", pose: null, others: poses.length - 1, overlap: true };
    }
    const overlap = geoms.some((g, i) => i !== best.i && iou(g.box, best.g.box) > this.p.overlapIoU);
    this.lock(best.g, t);
    return { status: "ok", pose: poses[best.i], geom: best.g, others: poses.length - 1, overlap };
  }
}

```

app/src/ui/app.js

```
// SteadyScreen UI controller (checkpoint 3): setup -> camera check + practice stand -> 30-s test -> evidence review -> summary.
//
// Session ownership (audit G): every session has an id. Every asynchronous step (camera permission, model load,
// countdown, timers, frame callbacks, image capture) checks `alive(sess)` before touching the UI or the core,
// so Back / New test / errors cancel pending work instead of letting it reopen an old session.
import { createChairStand } from "../core/chair_stand.js";
import { CORE_VERSION, PARAMS } from "../core/params.js";
import { decide, parseStrictInt, MIN_ADDED_SEPARATION_MS } from "../core/decision.js";
import { computeCoverage, anchorTime, COVERAGE_LIMITS, hasImageAt } from "../core/coverage.js";
import { loadLandmarker, startFrameLoop } from "./engine.js";
import { drawOverlay, drawTimeline } from "./draw.js";
import { buildReport, plainReason, STATUS_TEXT, COMPLETION_TEXT, FLAG_TEXT, RISE_TEXT, flagResolvedByRises } from "./report.js";
import { createEvidenceStore } from "./evidence.js";
import { beep } from "./audio.js";
import { createDemoScript, startDemoScene, decisionSource, demoRecord, DEMO_BADGE, DEMO_LABEL } from "./demo.js";

const $ = (id) => document.getElementById(id);
const QS = new URLSearchParams(location.search);
const HARNESS = QS.has("harness");
const screens = ["setup", "ready", "test", "review", "report"];
function show(name) {
  closeZoom();                                                              // an enlarged image never stays open across screens
  for (const s of screens) $("scr-" + s).classList.toggle("hidden", s !== name);
  const h = $("scr-" + name).querySelector("h1"); if (h) { h.tabIndex = -1; h.focus({ preventScroll: true }); }
  window.scrollTo(0, 0);
}
const esc = (s) => String(s ?? "").replace(/[&<>"]/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;" }[c]));
const sleep = (ms) => new Promise((r) => setTimeout(r, ms));

// CSP violations (blocked outgoing requests, e.g. the library's usage logging) are counted, not hidden.
const cspBlocked = [];
document.addEventListener("securitypolicyviolation", (e) => cspBlocked.push({ blockedURI: e.blockedURI, directive: e.effectiveDirective || e.violatedDirective }));

const READY_TEXT = {
  no_frames: "Waiting for video…",
  no_person_detected: "No person detected. Check that the participant is in view.",
  several_people_select_participant: "More than one person could be the participant. Click on the participant in the picture.",
  another_person_overlaps_participant: "Someone overlaps the participant in the picture. Helper: stay beside, within reach.",
  participant_lost: "Lost track of the participant — keep them in view.",
  hips_knees_ankles_not_all_visible: "Hips, knees and ankles must be visible. Move the camera back or lower; keep a side view.",
  practice_stand_needed: "Practice stand: stand up fully with arms crossed, hold still ~1 second, then sit down.",
  practice_stand_in_progress: "Practice stand seen: now sit down again; then confirm it on the card.",
  practice_stand_needs_confirmation: "Confirm whether the practice stand was a full stand (card below).",
  reference_stale: "The practice stand is more than 5 minutes old: please repeat it.",
  reference_person_changed: "The tracked person changed after the practice stand: please repeat it.",
  reference_camera_or_person_changed: "The camera or person moved since the practice stand: please repeat it.",
  hold_still_seated: "Participant: sit still with arms crossed.",
  participant_not_seated: "The participant does not look seated. Start from a seated position.",
  camera_frames_too_sparse: "The camera is delivering too few frames. Close other apps or improve lighting.",
};

// ---------- sessions ----------
let S = null, nextId = 1;
let landmarker = null, engineInfo = "", modelPromise = null;
const alive = (sess) => !!sess && S === sess && !sess.dead;
// live camera and the demo run on the performance clock in real time; a replay runs on video time
const liveClock = (sess) => sess.source === "camera" || sess.source === "demo";

function newSession(source) {
  teardown();
  // overnight fix ON-04: review-form fields from a previous session must not carry over (a stale "my count" could be saved)
  for (const id of ["in-manual", "in-manual-reason"]) $(id).value = "";
  $("manual-msg").textContent = ""; $("basis-body").innerHTML = "";
  S = { id: nextId++, source, dead: false, stream: null, core: null, loop: null, result: null, startedAt: null, goT: null,
    countingDown: false, ended: false, timer: null, evidenceOn: $("c-evidence").checked, evidence: createEvidenceStore(),
    replayName: null, replayUrl: null, participant: { ageText: $("in-age").value.trim(), column: $("in-sex").value },
    countdownToken: 0, coverage: null, refShownRefId: null, quiet: $("c-quiet").checked,
    review: { resolutions: {}, flagChecks: {}, identityConfirmed: null, identityConfirmedFor: [], sessionScan: null, added: [], basisChoice: null,
      arms: null, setup: null, manual: null }, audit: [], loopStats: null, evidenceStats: null, demo: null };
  return S;
}
function stopMedia(sess) {
  const v = $("video");
  if (sess && sess.demo) sess.demo.scene.stop();                            // demo: stop drawing the simulated scene
  if (sess && sess.stream) { sess.stream.getTracks().forEach((tr) => tr.stop()); sess.stream = null; }   // camera light off
  v.pause(); v.removeAttribute("src"); v.srcObject = null; v.load();
  if (sess && sess.replayUrl) { URL.revokeObjectURL(sess.replayUrl); sess.replayUrl = null; }
}
function teardown() {
  if (!S) return;
  const sess = S; sess.dead = true; sess.countdownToken++;
```

```
  closeZoom();                                                    // before its image URL is revoked below
  clearTimeout(sess.timer);
  if (sess.loop) { sess.loopStats = sess.loop.stats(); sess.loop.stop(); sess.loop = null; }
  stopMedia(sess);
  sess.evidence.clear();                                // drop every image and revoke object URLs
  setBadge(null); $("demo-banner").classList.add("hidden"); document.title = PAGE_TITLE;
  S = null;
}
window.addEventListener("pagehide", teardown);
// (review F3) a page restored from the back/forward cache has no session any more: start again at setup
window.addEventListener("pageshow", (e) => { if (e.persisted) { show("setup"); $("report").innerHTML = ""; } });

const PAGE_TITLE = document.title;
function log(sess, what) { sess.audit.push({ at: Date.now(), what }); }
function setBadge(text) { const b = $("modeBadge"); b.textContent = text || ""; b.classList.toggle("hidden", !text); }

// ---------- setup ----------
const safetyIds = ["c-helper", "c-chair", "c-okay", "c-camera"];
function supportProblems() {
  const p = [];
  if (!(navigator.mediaDevices && navigator.mediaDevices.getUserMedia)) p.push("camera access (getUserMedia)");
  if (!("requestVideoFrameCallback" in HTMLVideoElement.prototype)) p.push("video frame callbacks (requestVideoFrameCallback)");
  return p;
}
const unsupported = supportProblems();
if (unsupported.length) { $("support-msg").textContent = `This browser lacks ${unsupported.join(" and ")}. Use a recent Chrome or Edge.`; $("in-file").disabled = true; $("lbl-file").classList.add("disabled"); }
// the demo needs no camera, only frame callbacks and a canvas video stream
const demoUnsupported = !("requestVideoFrameCallback" in HTMLVideoElement.prototype) || !("captureStream" in HTMLCanvasElement.prototype);
if (demoUnsupported) $("demo-hint").textContent = "This browser cannot run the demo (needs a recent Chrome or Edge).";
function refreshSetup() {
  const safe = safetyIds.every((id) => $(id).checked);
  $("btn-camera").disabled = !(safe && $("c-consent").checked) || unsupported.length > 0;
  $("camera-hint").classList.toggle("hidden", !$("btn-camera").disabled || unsupported.length > 0);
  $("btn-demo").disabled = !safe || demoUnsupported;
  $("demo-hint").textContent = demoUnsupported ? "This browser cannot run the demo (needs a recent Chrome or Edge)." : "Tick the 4 safety boxes above to enable the demo.";
  $("demo-hint").classList.toggle("hidden", !$("btn-demo").disabled);
  const a = $("in-age").value.trim();
  $("age-msg").textContent = a && parseStrictInt(a, 3, 0, 120) === null ? "Age must be a whole number (digits only)." : "";
}
[...safetyIds, "c-consent", "in-age"].forEach((id) => $(id).addEventListener("input", refreshSetup));
[...safetyIds, "c-consent"].forEach((id) => $(id).addEventListener("change", refreshSetup));

function ensureModel() {
  if (landmarker) return Promise.resolve();
  if (!modelPromise) {
    const modelUrl = HARNESS && QS.get("model") === "missing" ? new URL("../../vendor/models/does-not-exist.task", import.meta.url).href : undefined;
    modelPromise = loadLandmarker({ numPoses: 2, preferGPU: QS.get("delegate") !== "CPU", ...(modelUrl ? { modelUrl } : {}) })
      .then((r) => { landmarker = r.landmarker; engineInfo = `${r.delegate}, loaded in ${r.startupMs} ms`; })
      .catch((e) => { modelPromise = null; throw e; });
  }
  return modelPromise;
}

$("btn-camera").addEventListener("click", () => startCamera());
async function startCamera() {
  const sess = newSession("camera");
  $("setup-msg").textContent = "";
  let stream;
  try {
    stream = await navigator.mediaDevices.getUserMedia({ video: { facingMode: "environment", width: { ideal: 1280 }, height: { ideal: 720 } }, audio: false });
  } catch (e) {
    if (!alive(sess)) return;
    teardown(); show("setup");
    $("setup-msg").textContent = e && e.name === "NotAllowedError" ? "Camera permission was denied. Allow the camera for this page and try again." : `Camera not available: ${e && e.message || e}`;
    return;
  }
  if (!alive(sess)) { stream.getTracks().forEach((t) => t.stop()); return; }
  sess.stream = stream;
  const v = $("video"); v.srcObject = stream; v.muted = true;
  stream.getVideoTracks()[0].addEventListener("ended", () => onTrackEnded(sess));
  try { await v.play(); } catch (e) { /* autoplay of a muted camera stream is normally allowed */ }
  if (!alive(sess)) return;
  log(sess, `Camera started${sess.evidenceOn ? " (temporary evidence images ON)" : " (no evidence images)"}${sess.quiet ? "; quiet counting (no per-stand tones)" : ""}`);
  await enterReadiness(sess);
}
function onTrackEnded(sess) {
  if (!alive(sess)) return;
  if (sess.core && sess.core.phase === "running" && !sess.ended) endTest(sess, "camera_error");
  else if (!sess.ended) { teardown(); show("setup"); $("setup-msg").textContent = "The camera stopped. Check the camera and try again."; }
```

```
}

$("in-file").addEventListener("change", async (ev) => {
  const f = ev.target.files[0]; ev.target.value = ""; if (!f) return;
  await startReplay(URL.createObjectURL(f), f.name);
});
async function startReplay(url, name) {
  const sess = newSession("replay"); sess.replayName = name; sess.replayUrl = url.startsWith("blob:") ? url : null;
  const v = $("video"); v.srcObject = null; v.src = url; v.loop = false; v.muted = true;
  try {
    await new Promise((r, j) => { v.onloadeddata = r; v.onerror = () => j(new Error("cannot open this video file")); });
  } catch (e) { if (alive(sess)) { teardown(); show("setup"); $("setup-msg").textContent = e.message; } return; }
  if (!alive(sess)) return;
  setBadge("REPLAY — not a live test");
  log(sess, `Replay of ${name}${sess.evidenceOn ? " (temporary evidence images ON)" : ""}`);
  await enterReadiness(sess);
  if (!alive(sess)) return;
  v.addEventListener("ended", () => onReplayEnded(sess), { once: true });
  try { await v.play(); } catch (e) { if (alive(sess)) $("ready-status").textContent = "Press play is blocked by the browser: " +
e.message; }
}
// Demo: a computer-drawn participant instead of a camera (app/src/ui/demo.js). Same screens, same core, same review.
$("btn-demo").addEventListener("click", () => startDemo());
// quick start for visitors: the safety boxes describe the simulated scene, and still images are kept so the checks can be tried
$("btn-demo-quick").addEventListener("click", () => {
  for (const id of [...safetyIds, "c-evidence"]) $(id).checked = true;
  refreshSetup();
  if (!$("btn-demo").disabled) startDemo();
  else $("setup-msg").textContent = "This browser cannot run the demo (needs a recent Chrome or Edge).";
});
if (demoUnsupported) $("btn-demo-quick").disabled = true;
async function startDemo() {
  const sess = newSession("demo");
  $("setup-msg").textContent = "";
  const script = createDemoScript();
  let scene;
  try { scene = startDemoScene(script); }
  catch (e) { teardown(); show("setup"); $("setup-msg").textContent = `The demo could not start in this browser (${String(e && e.
message || e).slice(0, 80)}).`; return; }
  sess.demo = { script, scene }; sess.stream = scene.stream;
  const v = $("video"); v.srcObject = scene.stream; v.muted = true;
  setBadge(DEMO_BADGE); $("demo-banner").classList.remove("hidden"); document.title = `SIMULATED DEMO · ${PAGE_TITLE}`;   // (rev
iew F4) also names a saved PDF
  try { await v.play(); } catch (e) { /* a muted canvas stream may autoplay */ }
  if (!alive(sess)) return;
  log(sess, `DEMO started: simulated participant (no camera, no pose model)${sess.evidenceOn ? " (temporary evidence images ON)"
: " (no evidence images)"}${sess.quiet ? "; quiet counting (no per-stand tones)" : ""}`);
  await enterReadiness(sess);
}

function onReplayEnded(sess) {
  if (!alive(sess) || sess.ended) return;
  if (sess.core && sess.core.phase === "running") endTest(sess, "stream_end");
  else { sess.countingDown = false; $("ready-status").textContent = "The video ended before the test could start."; $("ready-stat
us").className = "status bad"; $("btn-start").disabled = true; }
}


// ---------- camera check / practice stand ----------
async function enterReadiness(sess) {
  $("slot-ready").appendChild($("videowrap"));
  show("ready");
  $("btn-start").disabled = true; $("ref-card").classList.add("hidden");
  const v = $("video");
  // (review F2) nothing from an earlier demo session may show on a camera/replay session while the model loads
  $("engine-info").textContent = ""; { const o = $("overlay"); o.getContext("2d").clearRect(0, 0, o.width, o.height); }
  if (sess.source === "demo") {                       // no pose model: the demo generates the body points it draws
    $("ready-status").textContent = "Starting the simulated participant…"; $("ready-status").className = "status";
    $("engine-info").textContent = `Simulated demo: no camera and no pose model; body points come from the demo script. Core: ${C
ORE_VERSION}.`;
    sess.loop = startFrameLoop(v, null, { source: "demo", simulate: (t) => [sess.demo.script.landmarksAt(t)],
      onFrame: (f) => onFrame(sess, f), onError: (e) => { if (alive(sess)) console.error("demo frame error", e); } });
    return;
  }
  $("ready-status").textContent = "Loading the pose model (on this computer)…"; $("ready-status").className = "status";
  try { await ensureModel(); }
  catch (e) {
    if (!alive(sess)) return;
    teardown(); show("setup");
    $("setup-msg").textContent = `The pose model (which runs on this computer) could not be loaded, so the camera has been turned
off. Reload the page to try again. (Detail: ${String(e && e.message || e).slice(0, 80)})`;
    return;
  }
  if (!alive(sess)) return;
  $("engine-info").textContent = `Pose model: ${engineInfo}. Core: ${CORE_VERSION}.`;
  sess.loop = startFrameLoop(v, landmarker, {
    source: sess.source,
```

```
    onFrame: (f) => onFrame(sess, f),
    onError: (e) => { if (alive(sess)) console.error("pose error", e); },
  });
}

function onFrame(sess, f) {
  if (!alive(sess) || sess.ended) return;
  const v = $("video");
  if (!sess.core) sess.core = createChairStand({ width: v.videoWidth, height: v.videoHeight });
  let snap;
  try { snap = sess.core.push(f); } catch (e) { return; }
  sess.lastFrameT = f.t;
  drawOverlay($("overlay"), v, f.poses, snap.participantPose);
  if (sess.evidenceOn) sess.evidence.capture(v, f.t, snap.participantPose, sess.goT !== null ? sess.goT - 15000 : null,
    { priority: sess.goT !== null && Math.abs(f.t - (sess.goT + PARAMS.windowMs)) <= 400 });
  if (snap.phase === "calibrating" && sess.demo) sess.demo.script.react(f.t, snap.readiness);   // demo: repeat the practice stand when asked
  if (snap.phase === "calibrating") { if (!sess.countingDown) updateReadiness(sess, snap.readiness); }
  else if (snap.phase === "running") updateRunning(sess, snap);
  if (snap.result && !sess.ended) endTest(sess, null);
}

function updateReadiness(sess, r) {
  const st = $("ready-status");
  if (r.ready) { st.textContent = "Ready. Helper beside the participant."; st.className = "status ok"; }
  else { st.textContent = "Not ready yet"; st.className = "status warn"; }
  $("ready-reasons").innerHTML = r.reasons.map((x) => `<li>${esc(READY_TEXT[x] || x)}</li>`).join("");
  const pr = r.practice || {};
  $("step-practice").classList.toggle("done", pr.state === "accepted" || pr.mode === "skipped");
  $("step-seen").classList.toggle("done", !r.reasons.some((x) => ["no_person_detected", "several_people_select_participant", "participant_lost", "hips_knees_ankles_not_all_visible", "another_person_overlaps_participant"].includes(x)));
  if (pr.state === "accepted" && !pr.sideView) st.textContent += " (camera not side-on: each rise will need your check)";
  showReferenceCard(sess, pr);
  $("btn-start").disabled = !r.ready || sess.countingDown;
}

// Practice-stand acceptance (docs/CONTRACTS_CP4.md s1): shown once per detected hold; the answer is bound to its refId.
function showReferenceCard(sess, pr) {
  const card = $("ref-card");
  if (pr.state !== "detected") { card.classList.add("hidden"); sess.refShownRefId = null; return; }
  if (sess.refShownRefId === pr.refId && !card.classList.contains("hidden")) return;       // already showing this hold
  // wait until the image store has an image inside the hold (if images are kept) before asking
  const img = sess.evidenceOn ? sess.evidence.nearest((pr.t0 + pr.t1) / 2, (pr.t1 - pr.t0) / 2 + COVERAGE_LIMITS.referenceSlackMs) : null;
  // (frame-clock based: give a pending image up to ~1 s of video after the hold before asking without one)
  if (sess.evidenceOn && !img && (sess.lastFrameT ?? 0) - pr.t1 < 1000) return;
  sess.refShownRefId = pr.refId; sess.refShownBasis = img ? "image" : "observer_watched";
  sess.core.freezeReference(pr.refId);                  // what the observer accepts is exactly what the card showed (ON-09)
  // keep this hold's images however long the observer takes before GO (overnight fix ON-01; re-pinning replaces the pin)
  if (sess.evidenceOn) sess.evidence.pin("reference", pr.t0 - COVERAGE_LIMITS.referenceSlackMs, pr.t1 + COVERAGE_LIMITS.referenceSlackMs);
  const box = $("ref-img"); box.innerHTML = "";
  if (img) box.appendChild(thumb(sess, img, true, "practice stand"));
  else box.innerHTML = `<p class="small">${sess.evidenceOn ? "No image of the practice stand was captured." : "No images are being kept."} Answer from what you saw.</p>`;
  $("ref-warn").textContent = pr.postureWarning ? "The camera sees bent knees or hips in this position. Confirm only if this is how this person normally stands fully upright." : "";
  card.classList.remove("hidden");
  card.scrollIntoView({ block: "nearest" });              // the answer buttons must be visible (overnight UX)
}
$("btn-ref-yes").addEventListener("click", () => {
  const sess = S; if (!alive(sess) || !sess.core) return;
  const ok = sess.core.acceptReference({ refId: sess.refShownRefId, basis: sess.refShownBasis });
  log(sess, ok ? `Practice stand #${sess.refShownRefId} accepted as a full stand (${sess.refShownBasis === "image" ? "with its image" : "from direct observation, no image"})` : "Practice-stand acceptance failed (it changed): please repeat");
  $("ref-card").classList.add("hidden");
  updateReadiness(sess, sess.core.readiness());                              // refresh Start immediately (no wait for a frame)
});
$("btn-ref-no").addEventListener("click", () => {
  const sess = S; if (!alive(sess) || !sess.core) return;
  sess.core.rejectReference(sess.refShownRefId); log(sess, `Practice stand #${sess.refShownRefId} rejected (not a full stand): repeat requested`);
  $("ref-card").classList.add("hidden"); sess.refShownRefId = null;
  updateReadiness(sess, sess.core.readiness());
});
$("btn-noref").addEventListener("click", () => {
  const sess = S; if (!alive(sess) || !sess.core || sess.core.phase !== "calibrating") return;
  sess.core.countWithoutReference(true); log(sess, "Observer chose to count WITHOUT a practice-stand reference (every rise will need checking)");
  updateReadiness(sess, sess.core.readiness());
});

// click-to-select the participant (object-fit: contain letterboxing handled)
$("overlay").addEventListener("click", (ev) => {
  const sess = S; if (!alive(sess) || !sess.core || sess.core.phase !== "calibrating") return;
  const v = $("video"), rect = ev.currentTarget.getBoundingClientRect();
  const sc = Math.min(rect.width / v.videoWidth, rect.height / v.videoHeight);
```

```
  const ox = (rect.width - v.videoWidth * sc) / 2, oy = (rect.height - v.videoHeight * sc) / 2;
  const x = (ev.clientX - rect.left - ox) / (v.videoWidth * sc), y = (ev.clientY - rect.top - oy) / (v.videoHeight * sc);
  if (x < 0 || x > 1 || y < 0 || y > 1) return;
  sess.core.selectParticipant(x, y); log(sess, "Observer selected the participant in the picture (practice stand must be repeate
d)");
});

$("btn-start").addEventListener("click", () => startCountdown(S));
$("btn-back").addEventListener("click", () => { teardown(); show("setup"); });

// Countdown ownership (audit E): a token that ANY hidden event, Back, New test or error invalidates immediately.
function cancelCountdown(sess, why) {
  if (!sess || !sess.countingDown) return;
  sess.countdownToken++; sess.countingDown = false;
  $("ready-status").textContent = `Countdown cancelled (${why}). Press Start again when ready.`; $("ready-status").className = "s
tatus bad";
  log(sess, `Countdown cancelled: ${why}`);
}
async function startCountdown(sess) {
  if (!alive(sess) || sess.countingDown || !sess.core || sess.core.phase !== "calibrating" || document.visibilityState === "hidde
n") return;
  const token = ++sess.countdownToken;
  sess.countingDown = true; $("btn-start").disabled = true; closeZoom();
  const st = $("ready-status");
  const valid = () => alive(sess) && !sess.ended && sess.countdownToken === token && document.visibilityState !== "hidden";
  for (const n of [3, 2, 1]) {
    st.textContent = String(n); st.className = "status countdown"; beep(660, 150);
    await sleep(1000);
    if (!valid()) { if (alive(sess) && sess.countdownToken === token) cancelCountdown(sess, "tab hidden"); return; }
  }
  const r = sess.core.readiness();
  if (!r.ready) { sess.countingDown = false; st.textContent = "Not ready at GO — test not started: " + r.reasons.map((x) => READY
_TEXT[x] || x).join(" "); st.className = "status bad"; return; }
  const v = $("video");
  const goT = liveClock(sess) ? performance.now() : v.currentTime * 1000;
  try { sess.core.start(goT); }
  catch (e) { sess.countingDown = false; st.textContent = "Test not started: " + e.message; st.className = "status bad"; return;
}
  sess.goT = goT; sess.startedAt = Date.now();
  if (sess.demo) sess.demo.script.go(goT);                                    // the simulated participant starts standing
  log(sess, `GO (${liveClock(sess) ? "performance clock" : "video time"} ${Math.round(goT)} ms)`);
  beep(880, 450);
  $("slot-test").appendChild($("videowrap"));
  show("test");
  $("count").textContent = "0"; $("count-uncertain").textContent = ""; $("timer").textContent = "30"; $("timer-label").textConten
t = "seconds left (GO!)";
  $("test-status").textContent = "Measuring"; $("test-status").className = "status ok";
  if (liveClock(sess)) sess.timer = setTimeout(() => { if (alive(sess) && !sess.ended) endTest(sess, "expired"); }, PARAMS.window
Ms + 600);
  sess.countingDown = false;
}

let lastCount = 0;
function updateRunning(sess, snap) {
  const left = Math.max(0, Math.ceil((PARAMS.windowMs - snap.elapsedMs) / 1000));
  $("timer").textContent = String(left);
  if (snap.count !== lastCount) { $("count").textContent = String(snap.count); if (snap.count > lastCount && !sess.quiet) beep(52
0, 60); lastCount = snap.count; }
  $("count-uncertain").textContent = snap.uncertain ? `(+${snap.uncertain} to check)` : "";
  const ok = snap.lastStatus === "ok";
  $("test-status").textContent = ok ? "Measuring" : "Cannot see the participant clearly";
  $("test-status").className = "status " + (ok ? "ok" : "bad");
}

$("btn-stop").addEventListener("click", () => { if (alive(S)) endTest(S, "stopped"); });
document.addEventListener("visibilitychange", () => {
  const sess = S; if (!alive(sess) || document.visibilityState !== "hidden") return;
  if (sess.countingDown) cancelCountdown(sess, "tab hidden");                 // immediately, not at the next tick
  if (sess.core && sess.core.phase === "running" && !sess.ended) endTest(sess, "page_hidden");
});
document.addEventListener("keydown", (e) => {
  if ($("zoom").open) return;                                                 // the enlarged image's dialog handles its own key
s (Esc closes only it)
  const scr = screens.find((s) => !$("scr-" + s).classList.contains("hidden"));
  if (scr === "test" && e.key === "Escape") { e.preventDefault(); if (alive(S)) endTest(S, "stopped"); }   // (overnight: a 2nd S
pace no longer stops the test)
  else if (scr === "test" && e.key === " ") e.preventDefault();
  else if (scr === "ready" && e.key === " " && !$("btn-start").disabled) { e.preventDefault(); startCountdown(S); }
});

async function endTest(sess, reason) {
  if (!alive(sess) || sess.ended) return;
  sess.ended = true;
  clearTimeout(sess.timer);
  let res = sess.core && sess.core.snapshot().result;
  if (!res && sess.core && sess.core.phase === "running") {
    const tNow = liveClock(sess) ? performance.now() : $("video").currentTime * 1000;
```

```
    res = sess.core.finish(reason || "stream_end", tNow);
  }
  if (sess.loop) { sess.loopStats = sess.loop.stats(); sess.loop.stop(); sess.loop = null; }
  stopMedia(sess);                                                    // camera off
  beep(440, 200); setTimeout(() => beep(440, 200), 260);
  lastCount = 0;
  if (!res) { teardown(); show("setup"); return; }
  // wait for image captures still being encoded (bounded), THEN freeze coverage (audit D: pending final frame)
  sess.evidenceStats = await sess.evidence.finalize(1500);
  if (!alive(sess)) return;
  sess.result = res;
  sess.coverage = sess.evidenceOn ? computeCoverage(res, sess.evidence.times()) : null;
  log(sess, `Test ended: ${res.reason}; camera measurement: ${res.status}; images kept: ${sess.evidenceStats.frames}`);
  renderReview(sess);
}

// ---------- review ----------
function evidenceAvailable(sess) { return !!(sess.evidenceOn && sess.coverage && sess.coverage.imageCount > 0); }
const covOk = (sess, n) => !!(sess.coverage && sess.coverage.events[n] && sess.coverage.events[n].ok);
function decision(sess) {
  const rv = sess.review;
  return decide({ result: sess.result, source: decisionSource(sess.source), observer: { arms: rv.arms, setup: rv.setup },
    evidence: { available: evidenceAvailable(sess), coverage: sess.coverage, identityConfirmed: rv.identityConfirmed, identityCon
firmedFor: rv.identityConfirmedFor,
      eventResolutions: rv.resolutions, flagChecks: rv.flagChecks, sessionScan: rv.sessionScan, added: rv.added },
    manual: rv.manual, basisChoice: rv.basisChoice, participant: sess.participant });
}

function boxEl(frame) {
  const b = document.createElement("div"); b.className = "box";
  b.style.left = `${frame.box[0] * 100}%`; b.style.top = `${frame.box[1] * 100}%`;
  b.style.width = `${(frame.box[2] - frame.box[0]) * 100}%`; b.style.height = `${(frame.box[3] - frame.box[1]) * 100}%`;
  return b;
}
function thumb(sess, frame, highlight = false, label = "") {
  const fig = document.createElement("figure"); fig.className = "thumb" + (highlight ? " hl" : "");
  const img = document.createElement("img"); img.src = sess.evidence.url(frame); img.alt = label || "evidence image"; fig.appendC
hild(img);
  if (frame.box) fig.appendChild(boxEl(frame));
  const cap = document.createElement("figcaption"); cap.textContent = label; fig.appendChild(cap);
  // click-to-enlarge (judge review): the Enlarge button (mouse or Enter/Space), or a click on the image itself unless the
  // image's own click adds a missed stand (section 4). It must never trigger that action or the page's Space/Esc keys.
  const z = document.createElement("button"); z.type = "button"; z.className = "zoombtn"; z.textContent = "Enlarge"; z.title = "E
nlarge this image";
  z.setAttribute("aria-label", `Enlarge image${label ? ": " + label : ""}`);
  const open = (ev) => { ev.preventDefault(); ev.stopPropagation(); openZoom(sess, frame, label, z); };
  z.addEventListener("click", open);
  z.addEventListener("keydown", (ev) => { if (ev.key === "Enter" || ev.key === " ") ev.stopPropagation(); });
  img.addEventListener("click", (ev) => { if (!fig.classList.contains("pick")) open(ev); });
  fig.appendChild(z);
  return fig;
}
// Enlarged image in a modal <dialog>: the same in-tab object URL (no download, no new URL). Not available during the
// countdown or the 30-s test; closed by Close, Esc (handled by the dialog only) or a click outside the image.
let zoomOpener = null;
function openZoom(sess, frame, label, opener) {
  const dlg = $("zoom");
  const scr = screens.find((s) => !$("scr-" + s).classList.contains("hidden"));
  if (dlg.open || !alive(sess) || scr === "test" || sess.countingDown || (sess.core && sess.core.phase === "running" && !sess.end
ed)) return;
  const img = $("zoom-img"), fig = $("zoom-fig");
  img.src = sess.evidence.url(frame); img.alt = label || "evidence image";
  const sc = Math.max(0.5, Math.min((window.innerWidth * 0.92 - 40) / frame.w, (window.innerHeight * 0.96 - 170) / frame.h, 2.
5));
  img.style.width = `${Math.round(frame.w * sc)}px`;
  fig.querySelectorAll(".box").forEach((b) => b.remove());
  if (frame.box) fig.appendChild(boxEl(frame));
  $("zoom-cap").textContent = label || "Evidence image";
  const s = sess.result ? (frame.t - sess.result.window.tGo) / 1000 : null;
  $("zoom-when").textContent = (s === null ? "" : `Image taken ${Math.abs(s).toFixed(1)} s ${s >= 0 ? "after" : "before"} GO. `)
+
    (frame.box ? (sess.source === "demo" ? "The green outline marks the simulated participant. " : "The green outline marks the p
erson the camera tracked. ") : "") + "This image stays in this browser tab.";
  zoomOpener = opener || null;
  dlg.showModal();
  $("zoom-close").focus();
}
function closeZoom() { const d = $("zoom"); zoomOpener = null; if (d && d.open) d.close(); }   // programmatic close: no focus re
turn
$("zoom-close").addEventListener("click", () => $("zoom").close());
$("zoom").addEventListener("click", (e) => { if (e.target === e.currentTarget) e.currentTarget.close(); });   // click on the bac
kdrop
$("zoom").addEventListener("close", () => {
  $("zoom-img").removeAttribute("src");
  const o = zoomOpener; zoomOpener = null;
  if (o && o.isConnected) o.focus({ preventScroll: true });
});
```

```
function missingTile(text) { const d = document.createElement("div"); d.className = "thumb missing"; d.textContent = text; return d; }
const rel = (sess, t) => ((t - sess.result.window.tGo) / 1000).toFixed(1);
const btn = (txt, on, { disabled = false, chosen = false, key = null } = {}) => { const b = document.createElement("button"); b.textContent = txt; b.disabled = disabled; b.dataset.k = key || txt; if (chosen) b.className = "chosen"; b.addEventListener("click", on); return b; };
// keyboard focus survives re-rendering (overnight UX): remember the focused control's key and its card
function rememberFocus() { const a = document.activeElement; if (!a || !a.dataset || !a.dataset.k) return null; const c = a.closest(".card, fieldset"); return { k: a.dataset.k, card: c && c.id }; }
function restoreFocus(f) {
  if (!f) return;
  const el = [...document.querySelectorAll("#scr-review [data-k]")].find((x) => x.dataset.k === f.k && !x.disabled);
  if (el) { el.focus({ preventScroll: true }); return; }
  const c = f.card && $(f.card); if (c) { const h = c.querySelector("h2, legend") || c; h.tabIndex = -1; h.focus({ preventScroll: true }); }
}
// what moment a rise's heading time is (coverage.anchorTime order); image captions show when each image was taken instead
const anchorLabel = (e, t) => e.tFull != null ? `top at ${t} s` : e.tExpiry != null ? `30-s mark (${t} s)` : e.tPeak != null ? `highest point at ${t} s` : `starts at ${t} s`;
const isExpiryRise = (e) => e.cls === "expiry" || (e.flags || []).some((x) => /^expiry_/.test(x));

function renderReview(sess) {
  const focus = rememberFocus();
  const r = sess.result, ev = evidenceAvailable(sess), cov = sess.coverage;
  const d0 = decision(sess);

  // 1. reference
  const rb = $("reference-body"); rb.innerHTML = "";
  const ref = r.reference;
  if (!ref) rb.innerHTML = `<p class="warn">No practice stand was used. ${r.extensionCheck === "no_accepted_reference" ? "You chose to count without one: every rise needs your decision." : ""}</p>`;
  else {
    rb.innerHTML = `<p>Practice stand #${ref.id} — ${ref.accepted ? `accepted as a full stand (${ref.accepted.basis === "image" ? "while looking at its image" : "from direct observation, no image"})` : "<strong>not accepted</strong>"}${ref.postureWarning ? " — <span class=\"warn\">the camera saw bent knees or hips</span>" : ""}.</p>`;
    if (ev && cov.reference.ok) { const f = sess.evidence.nearest(cov.reference.imageT, 1); if (f) { const st = document.createElement("div"); st.className = "strip"; st.appendChild(thumb(sess, f, true, "practice stand")); rb.appendChild(st); } }
    else if (sess.evidenceOn) rb.appendChild(missingTile("no image of the practice stand"));
  }

  // 2. identity (only rises that have their own image near the top)
  const idb = $("identity-body"); idb.innerHTML = "";
  const needId = r.events.filter((e) => e.counted || e.cls === "uncertain" || sess.review.resolutions[e.n] === "full");
  if (!ev) idb.innerHTML = `<p class="warn">No images were kept, so nobody can check afterwards who was measured.</p>`;
  else if (!needId.length) idb.innerHTML = "<p>No rises to check.</p>";
  else {
    const shown = needId.filter((e) => covOk(sess, e.n)), missing = needId.filter((e) => !covOk(sess, e.n));
    const strip = document.createElement("div"); strip.className = "strip";
    for (const e of needId) {
      const c = cov.events[e.n];
      if (c && c.ok) strip.appendChild(thumb(sess, sess.evidence.nearest(c.imageT, 1), false, `rise ${e.n} · ${rel(sess, c.imageT)} s`));
      else strip.appendChild(missingTile(`rise ${e.n}: no image within ${COVERAGE_LIMITS.anchorMaxMs} ms of its top`));
    }
    idb.appendChild(strip);
    if (missing.length) idb.insertAdjacentHTML("beforeend", `<p class="warn">Rises ${missing.map((e) => e.n).join(", ")} have no image near their top, so identity cannot be confirmed for them.</p>`);
    const q = document.createElement("div"); q.className = "radios";
    q.innerHTML = `<p>Is the <span class="swatch green"></span> outlined person the participant in <strong>every</strong> image shown?</p>`;
    const idsShown = shown.map((e) => e.n);
    const same = sess.review.identityConfirmedFor.length === idsShown.length && idsShown.every((n) => sess.review.identityConfirmedFor.includes(n));
    for (const [val, txt] of [[true, "Yes"], [false, "No"]]) q.appendChild(btn(txt, () => { sess.review.identityConfirmed = val; sess.review.identityConfirmedFor = val ? idsShown : []; log(sess, `Identity in images of rises ${idsShown.join(", ")}: ${txt}`); renderReview(sess); }, { disabled: !shown.length, chosen: sess.review.identityConfirmed === val && same }));
    idb.appendChild(q);
  }

  // 3. rises: required anchors (start / top / end) always shown; missing ones marked
  $("rises-help").textContent = ev ? `Images around each rise. The highlighted image is the one nearest the camera's top of the rise; image captions give the time each image was taken, which can differ from the rise's top time by up to ${COVERAGE_LIMITS.anchorMaxMs / 1000} s. Click an image or its Enlarge button to see it larger. Decisions are possible only for rises that have an image near their top.` : "No images were kept: rises cannot be checked here. Uncertain rises stay unresolved; you can enter your own count instead.";
  const rises = $("rises"); rises.innerHTML = "";
  for (const e of r.events) {
    const card = document.createElement("div"); card.className = `rise ${e.cls}`;
    const res = sess.review.resolutions[e.n];
    const tA = anchorTime(e);
    const c = cov && cov.events[e.n];
    const exp = isExpiryRise(e);
    const resTxt = exp ? { full: "over halfway at 30 s (counts)", not_full: "not over halfway at 30 s", cant_tell: "can't tell" } : { full: "full stand", not_full: "not a full stand", cant_tell: "can't tell" };
    card.innerHTML = `<h3>Rise ${e.n} · ${tA !== null ? anchorLabel(e, rel(sess, tA)) : ""} · ${esc(RISE_TEXT[e.cls])}${res ? ` · <em>your decision: ${esc(resTxt[res])}</em>` : ""}</h3>` +
      (e.flags.length ? `<p class="small">${e.flags.map((f) => esc(f.replace(/_/g, " "))).join("; ")}</p>` : "");
```

```
    if (ev && e.tRise !== null) {
      const top = c && c.ok ? sess.evidence.nearest(c.imageT, 1) : null;
      const start = sess.evidence.nearest(e.tRise, 300), end = e.tSeat ? sess.evidence.nearest(e.tSeat, 300) : null;
      const frames = sess.evidence.between(e.tRise - 400, (e.tSeat || e.tExpiry || (e.tRise + 2500)) + 300, 7, [start, top, en
d]);
      const strip = document.createElement("div"); strip.className = "strip";
      for (const f of frames) strip.appendChild(thumb(sess, f, f === top, `${rel(sess, f.t)} s`));
      if (!top) strip.appendChild(missingTile(`no image within ${COVERAGE_LIMITS.anchorMaxMs} ms of the top (${tA !== null ? rel
(sess, tA) : "?"} s)`));
      const det = document.createElement("details"); det.open = e.cls === "uncertain" || !!res || !top;
      const sm = document.createElement("summary"); sm.textContent = `Images of this rise (${frames.length})`; det.appendChild(s
m); det.appendChild(strip);
      card.appendChild(det);
    }
    const row = document.createElement("div"); row.className = "choices";
    // at the 30-s mark the CDC question is "over halfway up?", not "full stand?" (overnight protocol audit)
    const opts = exp ? (e.cls === "uncertain" ? [["full", "Over halfway up at 30 s (counts)"], ["not_full", "Not over halfway at
30 s"], ["cant_tell", "Can't tell"]]
        : e.cls === "partial" ? [["full", "Actually over halfway at 30 s (counts)"], ["cant_tell", "Can't tell"]] : [["not_full",
"Not over halfway at 30 s"], ["cant_tell", "Can't tell"]])
      : e.cls === "uncertain" ? [["full", "Full stand"], ["not_full", "Not a full stand"], ["cant_tell", "Can't tell"]]
      : e.cls === "partial" ? [["full", "Actually a full stand"], ["cant_tell", "Can't tell"]] : [["not_full", "Not a full stan
d"], ["cant_tell", "Can't tell"]];
    const can = ev && covOk(sess, e.n);
    for (const [val, txt] of opts) row.appendChild(btn(txt, () => { sess.review.resolutions[e.n] = val; log(sess, `Rise ${e.n}
(${e.cls}): observer decided "${txt}" looking at its images`); renderReview(sess); }, { disabled: !can, chosen: res === val, key: `ris
e${e.n}:${val}` }));
    if (res) row.appendChild(btn("Undo", () => { delete sess.review.resolutions[e.n]; log(sess, `Rise ${e.n}: decision undone`);
renderReview(sess); }, { key: `rise${e.n}:undo` }));
    if (!can) { const p = document.createElement("span"); p.className = "small"; p.textContent = ev ? " (no image near the top of
this rise — cannot be decided)" : " (no images — cannot be checked)"; row.appendChild(p); }
    card.appendChild(row);
    rises.appendChild(card);
  }
  if (!r.events.length) rises.innerHTML = "<p>No rises were detected.</p>";

  // 4. whole-test images: look for missed stands, add them from an image (audit C)
  const sb = $("scan-body"); sb.innerHTML = "";
  if (!ev) sb.innerHTML = `<p class="warn">No images were kept: a stand the camera missed cannot be found or added afterwards.</p
>`;
  else {
    const sc = cov.scan;
    sb.insertAdjacentHTML("beforeend", sc.ok ? `<p>Images cover the whole test (largest gap ${(sc.maxGapMs / 1000).toFixed(1)}
s). If the participant made a full stand the camera missed, click the image that shows <strong>the participant</strong> standing fully
(images inside detected rises are decided in section 3 instead).</p>`
      : `<p class="warn">Images are missing between ${sc.gaps.map(([a, z]) => `${rel(sess, a)}–${rel(sess, z)} s`).join(", ")}, s
o missed stands cannot be ruled out there.</p>`);
    const det = document.createElement("details"); const sm = document.createElement("summary"); sm.textContent = "Whole-test ima
ges (about 2 per second)"; det.appendChild(sm);
    const strip = document.createElement("div"); strip.className = "strip";
    let lastShown = -Infinity;
    for (const t of sess.evidence.times()) {
      if (t < r.window.tGo || t > r.window.tEnd || t - lastShown < 450) continue;
      lastShown = t; const f = sess.evidence.at(t); if (!f) continue;
      // overnight fix ON-02: images inside a detected rise are decided on that rise's card; one stand cannot be added twice
      const inRise = r.events.find((e) => Number.isFinite(e.tRise) && t >= e.tRise && t <= (e.tSeat ?? e.tExpiry ?? r.window.tEn
d));
      const nearAdded = sess.review.added.find((a) => Math.abs(a.t - t) < MIN_ADDED_SEPARATION_MS);
      const fig = thumb(sess, f, false, `${rel(sess, t)} s${inRise ? ` · rise ${inRise.n}` : nearAdded ? " · added" : ""}`);
      if (nearAdded) fig.classList.add("added");
      if (inRise || nearAdded) { fig.classList.add("nopick"); fig.title = inRise ? `Part of detected rise ${inRise.n}: decide on
that rise in section 3` : "A stand was already added here"; }
      else {
        fig.classList.add("pick"); fig.title = "Add a full stand BY THE PARTICIPANT that the camera missed at this moment"; fig.t
abIndex = 0; fig.setAttribute("role", "button");
        fig.setAttribute("aria-label", `Add a missed full stand by the participant at ${rel(sess, t)} s`); fig.dataset.k = `pic
k:${Math.round(t)}`;
        const add = () => { if (sess.review.added.some((a) => Math.abs(a.t - f.t) < MIN_ADDED_SEPARATION_MS)) return; const id =
`added-${Date.now()}-${sess.review.added.length + 1}`; sess.review.added.push({ id, t: f.t }); log(sess, `Observer added a missed full
stand at ${rel(sess, f.t)} s (image ${Math.round(f.t)})`); renderReview(sess); };
        fig.addEventListener("click", add); fig.addEventListener("keydown", (ev) => { if (ev.key === "Enter" || ev.key === " ") {
ev.preventDefault(); ev.stopPropagation(); add(); } });
      }
      strip.appendChild(fig);
    }
    det.appendChild(strip); sb.appendChild(det);
    if (sess.review.added.length) {
      const ul = document.createElement("ul");
      for (const a of sess.review.added) { const li = document.createElement("li"); li.textContent = `Added full stand at ${rel(s
ess, a.t)} s `; li.appendChild(btn("Undo", () => { sess.review.added = sess.review.added.filter((x) => x !== a); log(sess, `Added stan
d at ${rel(sess, a.t)} s removed`); renderReview(sess); }, { key: `added-undo:${a.id}` })); ul.appendChild(li); }
      sb.appendChild(ul);
    }
    const row = document.createElement("div"); row.className = "choices";
    row.appendChild(btn("I looked through all images: no stand is missing", () => { sess.review.sessionScan = "no_missed"; log(se
ss, "Whole-test images checked: no missed stands"); renderReview(sess); }, { disabled: !sc.ok, chosen: sess.review.sessionScan === "no
_missed" }));
```

```
    row.appendChild(btn("A stand is missing that I cannot add", () => { sess.review.sessionScan = "missed"; log(sess, "Whole-test images: a missed stand could not be added"); renderReview(sess); }, { chosen: sess.review.sessionScan === "missed" }));
    sb.appendChild(row);
  }

  // 5. flags that need their own images
  const fl = $("flags"); fl.innerHTML = "";
  const own = r.reviewFlags.filter((f) => !flagResolvedByRises(f));
  if (!own.length) fl.innerHTML = "<p>Nothing else to check.</p>";
  for (const f of own) {
    const div = document.createElement("div"); div.className = "flag";
    const fc = cov && cov.flags[f];
    div.innerHTML = `<p><strong>${esc(FLAG_TEXT[f] || f)}</strong>${sess.review.flagChecks[f] ? ` · <em>you checked: ${esc(sess.review.flagChecks[f] === "ok" ? "counts here look right" : "problem")}</em>` : ""}</p>`;
    const wins = (r.flagWindows && r.flagWindows[f]) || [];
    if (ev && wins.length) {
      const strip = document.createElement("div"); strip.className = "strip";
      for (const [a, z] of wins.slice(0, 3)) for (const fr of sess.evidence.between(a - 300, z + 300, 4)) strip.appendChild(thumb(sess, fr, false, `${rel(sess, fr.t)} s`));
      if (fc && !fc.ok) strip.appendChild(missingTile("images missing in this period"));
      div.appendChild(strip);
    }
    const can = ev && fc && fc.ok;
    const row = document.createElement("div"); row.className = "choices";
    const labels = f === "reference_posture_warning" ? [["ok", "Checked: this IS how this person stands fully upright"], ["problem", "No: the practice stand was not a full stand"]]
      : [["ok", "Checked: the camera's counts here look right"], ["problem", "Problem"]];
    for (const [val, txt] of labels) row.appendChild(btn(txt, () => { sess.review.flagChecks[f] = val; log(sess, `Checked "${f}": ${val}`); renderReview(sess); }, { disabled: !can, chosen: sess.review.flagChecks[f] === val, key: `flag:${f}:${val}` }));
    if (!can) { const p = document.createElement("span"); p.className = "small"; p.textContent = !ev ? " (no images — cannot be checked)" : " (images do not cover this period — cannot be checked)"; row.appendChild(p); }
    div.appendChild(row); fl.appendChild(div);
  }

  // 6. protocol questions
  const pf = r.protocolFlags;
  $("arms-hint").textContent = pf.includes("arms_possibly_uncrossed") ? "Camera note: the arms looked uncrossed during part of the test. Please answer from what you saw." :
    pf.includes("arms_not_observable") ? "Camera note: the arms could not be seen well; please answer from what you saw." : "";
  document.querySelectorAll("input[name=arms]").forEach((x) => { x.checked = sess.review.arms === x.value; x.onchange = () => { sess.review.arms = x.value; log(sess, `Arms used to stand: ${x.value}`); renderDecision(sess); }; });
  document.querySelectorAll("input[name=setup]").forEach((x) => { x.checked = sess.review.setup === x.value; x.onchange = () => { sess.review.setup = x.value; log(sess, `Set-up as described: ${x.value}`); renderDecision(sess); }; });
  // 7. supervisor count only for a completed timed test
  const completed = d0.completion === "completed_30s";
  $("manual-card").classList.toggle("disabled", !completed);
  $("btn-manual").disabled = !completed; $("in-manual").disabled = !completed; $("in-manual-reason").disabled = !completed;
  $("manual-msg").textContent = completed ? "" : "Not available: this was not a completed 30-second test.";
  // CDC-table inputs can be entered or corrected here (overnight UX: previously setup-only, forcing a repeat test)
  $("rv-age").value = sess.participant.ageText || ""; $("rv-sex").value = sess.participant.column || "";
  drawTimeline($("timeline"), r.trace, r, sess.core.params);
  $("quality").textContent = JSON.stringify({ quality: r.quality, frames: sess.loopStats, evidence: sess.evidenceStats, coverage: cov && { reference: cov.reference, endpoint: cov.endpoint, scan: { ok: cov.scan.ok, maxGapMs: cov.scan.maxGapMs, gaps: cov.scan.gaps } },
    extensionCheck: r.extensionCheck, reference: r.reference, cspBlockedRequests: cspBlocked.length }, null, 1);
  renderDecision(sess);
  if ($("scr-review").classList.contains("hidden")) show("review");
  else restoreFocus(focus);
}
// Result banner (rendered with every decision update, so its colour and wording follow the decision layer).
// Green ("measured") only once the decision reports the camera's count; until then a measured run is shown neutral
// ("pending"), because the detected number is not a result until checked (judge review).
function renderBanner(sess, d) {
  const r = sess.result, b = $("result-banner");
  // (review F1) a camera count reported as a range (d.reference.fromRange) is reported too, although d.value is null
  const cameraReported = d.completion === "completed_30s" && !d.manualUsed && (d.value !== null || !!d.reference.fromRange);
  const rangeOpen = d.automatic ? !d.automatic.determinate : r.countRange && r.countRange.min !== r.countRange.max;
  b.className = "banner " + (d.completion !== "completed_30s" ? "withheld" : cameraReported ? "measured" : r.status === "measured" ? "pending" : r.status);
  const range = r.countRange ? (r.countRange.min === r.countRange.max ? `${r.countRange.min}` : `${r.countRange.min}–${r.countRange.max}`) : null;
  const notReportedWhy = d.manualUsed ? " Your own count is reported instead (see “What will be reported” below)."
    : !sess.evidenceOn ? " It will not be reported: no images were kept to check it."
    : cameraReported ? " Checked: see “What will be reported” below."
    : " Not a result until checked (see “Still needed” below).";
  const who = sess.source === "demo" ? "the app (simulated participant)" : "the camera";
  b.innerHTML = (sess.source === "demo" ? `<p class="demolabel">${esc(DEMO_LABEL)}.</p>` : "") + `<strong>${esc(COMPLETION_TEXT[d.completion] || d.completion)}.</strong> ${esc(STATUS_TEXT[r.status])}${range !== null ? ` — ${who} detected <strong>${esc(range)}</strong> full stand(s)${rangeOpen ? " (some rises need your decision)" : ""}.${esc(notReportedWhy)}` : "."}` +
    ([...r.withheldReasons.map(plainReason), ...r.reviewNotes].length ? `<ul>${[...r.withheldReasons.map(plainReason), ...r.reviewNotes].map((x) => `<li>${esc(x)}</li>`).join("")}</ul>` : "") +
    (sess.evidenceOn ? `<p class="small">Images kept: ${sess.evidenceStats.frames}; failed captures: ${sess.evidenceStats.dropped}; discarded late: ${sess.evidenceStats.lateOrStale}.</p>` : "");
}
function onParticipantEdit() {
  const sess = S; if (!sess || !sess.result) return;
  const ageText = $("rv-age").value.trim(), column = $("rv-sex").value;
```

```
  if (ageText === sess.participant.ageText && column === sess.participant.column) return;
  sess.participant = { ageText, column };
  log(sess, `Observer entered age "${ageText || "—"}" and table column "${column || "—"}" on the review screen`);
  renderDecision(sess);
}
$("rv-age").addEventListener("change", onParticipantEdit); $("rv-sex").addEventListener("change", onParticipantEdit);

$("btn-manual").addEventListener("click", () => {
  const sess = S; if (!sess || !sess.result) return;
  const text = $("in-manual").value, reason = $("in-manual-reason").value;
  if (parseStrictInt(text, 2, 0, 60) === null) { $("manual-msg").textContent = "Enter a whole number from 0 to 60 (digits only)."; return; }
  if (!reason.trim()) { $("manual-msg").textContent = "Say how you counted (required)."; return; }
  sess.review.manual = { text, reason: reason.trim() }; $("manual-msg").textContent = "";
  if (sess.review.basisChoice) { log(sess, "Count basis choice reset because your own count changed"); sess.review.basisChoice = null; }
  log(sess, `Observer's own count entered: ${text} (${reason.trim()})`);
  renderDecision(sess);
});

// Count basis (docs/CONTRACTS_CP4.md s3): explicit, with a rationale; no default preference.
function renderBasis(sess, d) {
  const bb = $("basis-body"); bb.innerHTML = "";
  if (!sess.review.manual || d.completion !== "completed_30s") return;
  const auto = d.automatic;
  const p = document.createElement("p");
  p.innerHTML = d.conflict ? `<strong class="warn">The counts differ:</strong> camera (after your checks) ${auto.determinate ? auto.min : auto.min + "–" + auto.max}, your count ${d.supervisor.value}. Choose which one to report and say why.`
    : `Your count: ${d.supervisor.valid ? d.supervisor.value : "not valid"}.${auto && auto.eligible && d.supervisor.value === auto.min ? " It agrees with the checked camera count." : " You can report it instead of the camera count."}`;
  bb.appendChild(p);
  const why = document.createElement("input"); why.type = "text"; why.id = "in-basis-why"; why.placeholder = "Reason for this choice (required)"; why.maxLength = 200; why.value = sess.review.basisChoice ? sess.review.basisChoice.rationale : "";
  bb.appendChild(why);
  const row = document.createElement("div"); row.className = "choices";
  const pick = (basis) => () => { const rationale = $("in-basis-why").value.trim(); if (!rationale) { $("manual-msg").textContent = "A reason is required for choosing which count to report."; return; }
    sess.review.basisChoice = { basis, rationale, at: { automaticMin: auto ? auto.min : null, automaticMax: auto ? auto.max : null, supervisor: d.supervisor.value } }; log(sess, `Count basis chosen: ${basis === "supervisor" ? "my own count" : "camera count"} (${rationale})`); renderDecision(sess); };
  if (auto && auto.eligibleExceptRange) row.appendChild(btn("Report the camera count", pick("automatic"), { chosen: sess.review.basisChoice && sess.review.basisChoice.basis === "automatic" }));
  row.appendChild(btn("Report my own count", pick("supervisor"), { disabled: !d.supervisor.valid || !liveClock(sess), chosen: sess.review.basisChoice && sess.review.basisChoice.basis === "supervisor" }));
  if (sess.review.basisChoice) row.appendChild(btn("Undo choice", () => { sess.review.basisChoice = null; log(sess, "Count basis choice undone"); renderDecision(sess); }));
  bb.appendChild(row);
}

function renderDecision(sess) {
  let d = decision(sess);
  if (d.staleChoice) {                                  // overnight fix ON-03: counts changed after the choice -> choose again
    sess.review.basisChoice = null; log(sess, "Count basis choice cleared: a count changed after it was made. Choose again.");
    d = decision(sess); $("manual-msg").textContent = "A count changed after you chose which one to report: choose again.";
  }
  sess.lastDecision = d;
  renderBanner(sess, d);
  $("decision").innerHTML = `<p><strong>${esc(d.title)}</strong></p>
    <p>Reported number: <strong>${esc(d.value === null ? (d.reference.fromRange ? d.valueText : "not reported") : d.value)}</strong> — ${esc(d.basis)}</p>
    <p class="small">Camera (as detected): ${d.counts.machine ?? "—"} · camera after your checks: ${d.counts.reviewed ?? "—"}${d.automatic && !d.automatic.determinate ? ` (range ${d.automatic.min}–${d.automatic.max})` : ""} · your own count: ${d.counts.supervisor ?? "—"}</p>
    ${d.reference.eligible ? `<p class="cdcline">${sess.source === "demo" ? "<span class=\"demotag\">DEMO ONLY</span> " : ""}CDC table (${esc(d.reference.band)}): <strong>${d.reference.category === "below_average" ? "below average" : "not below average"}</strong> (below average = fewer than ${d.reference.threshold}). The CDC form states: “A below average score indicates a risk for falls.”</p>` : `<p>CDC table not applied because:</p><ul>${d.reference.blockers.map((x) => `<li>${esc(x)}</li>`).join("")}</ul>`}`;
  // "Still needed" summary at the top of the review (overnight UX: the list used to be only at the bottom)
  const todo = $("todo");
  todo.innerHTML = d.reference.eligible
    ? `<p><strong>Nothing more is needed.</strong> Reported number: <strong>${esc(d.value === null ? d.valueText : d.value)}</strong>. Open “Show summary”.</p>`
    : (() => {
      // blockers that no answer on this screen can remove (they need a new test, images, or a live camera)
      const fixedHere = (x) => !/^(Replay of a video file|This was not a completed 30-second test|No evidence images were kept|The practice-stand reference was accepted without looking|There is no image of the practice stand|No image near the top of rise|Images are missing between|No practice stand was accepted|No image at the 30-second mark)|no continuous images/.test(x);
      // (ON-40) without images, undecided rises cannot be resolved on this screen either
      const fixable = (x) => fixedHere(x) && !(!evidenceAvailable(sess) && /rise\(s\) are unresolved|decide every rise/.test(x));
      const can = d.reference.blockers.filter(fixable), cannot = d.reference.blockers.filter((x) => !fixable(x));
      return `<p><strong>${d.value === null ? "Not reported yet." : `Reported number: ${esc(d.value)}.`}</strong></p>` +
        (can.length ? `<p><strong>Still needed</strong> (on this screen):</p><ul>${can.map((x) => `<li>${esc(x)}</li>`).join("")}</ul>` : "") +
        (cannot.length ? `<p><strong>Cannot be fixed on this screen</strong> (the CDC table will not be shown):</p><ul>${cannot.map((x) => `<li>${esc(x)}</li>`).join("")}</ul>` : "");
    })();
  $("audit").innerHTML = sess.audit.map((a) => `<li>${esc(new Date(a.at).toLocaleTimeString())}: ${esc(a.what)}</li>`).join("");
```

```
  renderBasis(sess, d);
}

$("btn-report").addEventListener("click", () => { const sess = S; if (!sess || !sess.result) return; $("report").innerHTML = buildReport(sess, decision(sess)); show("report"); });
$("btn-again").addEventListener("click", () => { teardown(); show("setup"); });
$("btn-review").addEventListener("click", () => show("review"));
$("btn-new").addEventListener("click", () => { teardown(); show("setup"); });
$("btn-print").addEventListener("click", () => window.print());
function exportRecord(sess) {
  const d = decision(sess);
  const demo = sess.source === "demo";
  return { exportedAt: new Date().toISOString(), ...(demo ? { notice: DEMO_LABEL } : {}), core: CORE_VERSION, decisionVersion: d.version, source: sess.source, replayName: sess.replayName,
    ...(demo ? { demo: demoRecord(sess.demo && sess.demo.script) } : {}),
    participant: sess.participant, decision: d, result: sess.result, review: sess.review, audit: sess.audit, coverage: sess.coverage,
    frames: sess.loopStats, evidence: { kept: sess.evidenceOn, images: sess.evidenceStats ? sess.evidenceStats.frames : 0, failedCaptures: sess.evidenceStats ? sess.evidenceStats.dropped : 0, includedInThisFile: false },
    cspBlockedRequests: cspBlocked, engine: demo ? "none (simulated demo: body points generated by the demo script)" : engineInfo };
}
$("btn-json").addEventListener("click", () => {
  const sess = S; if (!sess || !sess.result) return;
  const a = document.createElement("a");
  a.href = URL.createObjectURL(new Blob([JSON.stringify(exportRecord(sess), null, 1)], { type: "application/json" }));
  a.download = `chair-stand-record-${sess.source === "demo" ? "SIMULATED-DEMO-" : ""}${new Date().toISOString().slice(0, 19).replace(/:/g, "")}.json`; a.click();
  setTimeout(() => URL.revokeObjectURL(a.href), 5000);
});

// ---------- test harness hooks (only with ?harness=1; used by eval/browser/*.mjs) ----------
if (HARNESS) {
  window.__steady = {
    async realtime(url, name, { evidence = false } = {}) { $("c-evidence").checked = evidence; await startReplay(url, name); return true; },
    startNow() { return startCountdown(S); },
    back() { $("btn-back").click(); },
    acceptShownReference() { $("btn-ref-yes").click(); return S && S.core ? S.core.readiness().practice : null; },
    rejectShownReference() { $("btn-ref-no").click(); },
    exportRecord() { return S && S.result ? exportRecord(S) : null; },
    reportText() { return S && S.result ? buildReport(S, decision(S)) : null; },
    evidenceTimes() { return S ? S.evidence.times() : []; },
    state() {
      const sess = S;
      return { hasSession: !!sess, source: sess && sess.source, phase: sess && sess.core && sess.core.phase, ended: sess && sess.ended, countingDown: sess && sess.countingDown,
        goT: sess && sess.goT, visible: screens.filter((s) => !$("scr-" + s).classList.contains("hidden")),
        readyStatus: $("ready-status").textContent, setupMsg: $("setup-msg").textContent, refCardVisible: !$("ref-card").classList.contains("hidden"),
        streamLive: !!(sess && sess.stream && sess.stream.getTracks().some((t) => t.readyState === "live")),
        videoSrcObject: !!$("video").srcObject, demo: sess && sess.demo ? { tGo: sess.demo.script.tGo, info: sess.demo.script.info() } : null,
        readiness: sess && sess.core && sess.core.phase === "calibrating" ? sess.core.readiness() : null,
        result: sess && sess.result && { status: sess.result.status, reason: sess.result.reason, count: sess.result.count, countRange: sess.result.countRange, events: sess.result.events.map((e) => ({ n: e.n, cls: e.cls, counted: e.counted, tFull: e.tFull, tPeak: e.tPeak, tExpiry: e.tExpiry })), withheld: sess.result.withheldReasons, flags: sess.result.reviewFlags, quality: sess.result.quality, extensionCheck: sess.result.extensionCheck, reference: sess.result.reference, window: sess.result.window },
        loopStats: sess && (sess.loop ? sess.loop.stats() : sess.loopStats), evidence: sess && sess.evidence.stats(), coverage: sess && sess.coverage && { imageCount: sess.coverage.imageCount, reference: sess.coverage.reference, events: sess.coverage.events, endpoint: sess.coverage.endpoint, scan: { ok: sess.coverage.scan.ok, gaps: sess.coverage.scan.gaps } },
        decision: sess && sess.result ? decision(sess) : null, review: sess && sess.review, cspBlocked: cspBlocked.length, audit: sess ? sess.audit.map((a) => a.what) : [] };
    },
  };
}
```

app/src/ui/audio.js

```
// Simple local tones (Web Audio). No speech services, nothing leaves the browser.
let ctx = null;
export function beep(freq = 660, ms = 150) {
  try {
    ctx = ctx || new (window.AudioContext || window.webkitAudioContext)();
    const o = ctx.createOscillator(), g = ctx.createGain();
    o.frequency.value = freq; g.gain.value = 0.2;
    o.connect(g); g.connect(ctx.destination);
    o.start(); o.stop(ctx.currentTime + ms / 1000);
  } catch { /* audio unavailable: visual cues still shown */ }
}

```

app/src/ui/demo.js

```
// Demo mode: a SIMULATED participant, so the whole workflow (setup, camera check, practice-stand card, GO, 30-s count,
// review, summary) can be tried without a camera, a chair or a helper. Nothing here measures a person.
//
// - The scene (floor, a straight-backed chair without arm rests, a drawn figure with arms crossed) is painted on an
//   offscreen canvas and piped into the app's <video> element with canvas.captureStream(), so the overlay and the
//   evidence images work exactly as with a camera. Every frame carries a burned-in "SIMULATED PARTICIPANT" label.
// - No pose model runs. The body points for each frame are generated from the SAME geometry that is drawn (a port of
//   skeleton() in eval/synth.mjs: thigh angle 90deg * alpha, hip height h = 1.1 * sin(90deg * alpha)), with small
//   seeded jitter, and go through the unchanged counting core.
// - The script reacts to the app: seated, one genuine full practice stand (repeated if the observer rejects it or it
//   goes stale), seated until GO; after GO a fixed, deterministic sequence of stands (see STANDS).
// This module has no DOM access at import time, so the script can be tested in Node (tests/demo.test.mjs).

export const DEMO_LABEL = "Simulated demo — not a real person, not a real measurement";
export const DEMO_BADGE = "SIMULATED DEMO — not a real person, not a real measurement";
export const CANVAS_LABEL = "SIMULATED PARTICIPANT — demo only";
export const DEMO_W = 1280, DEMO_H = 720;
export const DEMO_SEED = 20261012;

// ---- script (all times in ms) ----
export const PRACTICE = Object.freeze({ firstLeadMs: 2500, repeatLeadMs: 1500, riseMs: 1000, holdMs: 1500, descendMs: 1000 });
export const GO_LEAD_MS = 300;                                                  // reaction time after GO
// After GO: [riseMs, holdMs, descendMs, sitMs, topAlpha]. Cycles 2.55-2.75 s. Stand 5 tops out short (alpha 0.87:
// drawn with knees ~12 deg bent and the trunk stooped ~14 deg; hip height only ~0.02 short). The core sees the hip angle
// short beyond the full-stand band (fullHipDeg 4), so it is not counted, but not the strong evidence needed to call it
// "partial" silently (knee > 2 x 7 AND hip > 2 x 4 deg, or hip height > 2 x 0.12), so it asks for the observer's decision.
export const STANDS = Object.freeze([
  [900, 350, 850, 450, 1], [900, 350, 850, 450, 1], [900, 350, 900, 450, 1], [950, 350, 900, 400, 1],
  [950, 500, 850, 400, 0.87],
  [900, 350, 900, 550, 1], [950, 350, 900, 450, 1], [950, 350, 900, 450, 1], [950, 350, 900, 450, 1],
  [1000, 350, 900, 500, 1], [1000, 350, 900, 400, 1],
]);
// ...then one more rise (1 s) that is about 65% done, clearly over halfway in hip height, at the 30-s mark
// (GO + 300 + 29050 = 29350 ms): the CDC rule counts it.
export const LAST_RISE_MS = 1000;
/** What the core should find (verified in tests/demo.test.mjs). */
export const EXPECTED = Object.freeze({ full: 10, uncertain: 1, expiry: 1, counted: 11, rangeMin: 11, rangeMax: 12, uncertainRis
e: 5 });

const PRACTICE_AGAIN = new Set(["practice_stand_needed", "reference_stale", "reference_person_changed", "reference_camera_or_pers
on_changed"]);

// ---- geometry (reference units of eval/synth.mjs: 720x1280 portrait; mapped onto the 1280x720 canvas) ----
const SHIN = 200, THIGH = 220, TRUNK = 300, ANKLE = [400, 1150], FLOOR_Y = 1175;
const S = 0.7, OX = 640, RX = 270, OY = 662;
const cx = (x) => OX + S * (x - RX), cy = (y) => OY + S * (y - FLOOR_Y);
const clamp01 = (a) => Math.max(0, Math.min(1, a));
const ease = (u) => 0.5 - 0.5 * Math.cos(Math.PI * clamp01(u));

/** Body joints (reference units) for rise progress alpha (0 seated .. 1 standing). Legs and trunk exactly as synth skeleton().
*/
export function bodyAt(alpha) {
  const a = clamp01(alpha), th = (Math.PI / 2) * a;
  const ankle = [...ANKLE], knee = [ANKLE[0], ANKLE[1] - SHIN];
  const hip = [knee[0] - THIGH * Math.cos(th), knee[1] - THIGH * Math.sin(th)];
  const lean = (a > 0.02 && a < 0.98 ? 0.6 * Math.sin(Math.PI * a) : 0) + 0.1 * (1 - a);   // forward lean mid-rise
  const up = [Math.sin(lean), -Math.cos(lean)], fwd = [Math.cos(lean), Math.sin(lean)];
  const sh = [hip[0] + TRUNK * up[0], hip[1] + TRUNK * up[1]];
  const at = (u, f) => [sh[0] + u * up[0] + f * fwd[0], sh[1] + u * up[1] + f * fwd[1]];
  // arms crossed on the chest (side view): elbow below and in front of the shoulder, hand back on the chest
  return { alpha: a, lean, ankle, knee, hip, sh, head: at(90, 20), elbow: at(-95, 52), wrist: at(-28, 30), toe: [ankle[0] + 40, a
nkle[1]],
    shL: at(0, -12), shR: at(0, 12), elbowFar: at(-95, 44), wristFar: at(-28, 22) };
}

/** 33 normalised landmarks [x, y, z, visibility, presence] from the body (same layout as synth skeleton()), no jitter. */
export function landmarksOf(b, vis = 0.95) {
  const pts = new Array(33).fill(null).map(() => [...b.head]);
  const set = (i, p) => { pts[i] = [p[0], p[1]]; };
  for (let i = 0; i <= 10; i++) set(i, [b.head[0] + (i % 3) * 6, b.head[1] + Math.floor(i / 3) * 6]);
  set(11, b.shL); set(12, b.shR); set(13, b.elbow); set(14, b.elbowFar); set(15, b.wrist); set(16, b.wristFar);
  for (const [i, p] of [[17, b.wrist], [19, b.wrist], [21, b.wrist], [18, b.wristFar], [20, b.wristFar], [22, b.wristFar]]) set
(i, p);
  set(23, [b.hip[0] - 10, b.hip[1]]); set(24, [b.hip[0] + 10, b.hip[1]]);
  set(25, [b.knee[0] - 10, b.knee[1]]); set(26, [b.knee[0] + 10, b.knee[1]]);
  set(27, [b.ankle[0] - 10, b.ankle[1]]); set(28, [b.ankle[0] + 10, b.ankle[1]]);
  set(29, b.ankle); set(30, b.ankle); set(31, b.toe); set(32, b.toe);
  return pts.map(([x, y]) => [cx(x) / DEMO_W, cy(y) / DEMO_H, 0, vis, 0.99]);
}

function rng(seed) {                                                            // mulberry32 (as eval/synth.mjs)
  let a = seed >>> 0;
  return () => { a = (a + 0x6d2b79f5) >>> 0; let t = a; t = Math.imul(t ^ (t >>> 15), t | 1); t ^= t + Math.imul(t ^ (t >>> 7), t
| 61); return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };
```

```
}
const gauss = (r) => Math.sqrt(-2 * Math.log(r() + 1e-12)) * Math.cos(2 * Math.PI * r());

/**
 * The simulated participant's script. Times are on the caller's clock (performance.now() in the app).
 * start(t) once; react(t, readiness) on every camera-check frame; go(tGo) at GO. alphaAt / landmarksAt / bodyAtTime
 * are pure functions of time (jitter is seeded per 1/30-s slot), so the same times always give the same frames.
 */
export function createDemoScript({ seed = DEMO_SEED, noisePx = 2, vis = 0.95 } = {}) {
  let segs = [], t0 = null, tGo = null, practiceEnd = -Infinity, practices = 0;
  const plan = (tStart, steps) => { let t = tStart, a = 0; for (const [d, to] of steps) { segs.push({ t0: t, t1: t + d, a0: a, a1: to }); t += d; a = to; } return t; };
  const alphaAt = (t) => {
    let a = 0;
    for (const s of segs) { if (t < s.t0) break; if (t < s.t1) return s.a0 + (s.a1 - s.a0) * ease((t - s.t0) / (s.t1 - s.t0)); a = s.a1; }
    return a;
  };
  const self = {
    start(t) { if (t0 !== null) return; t0 = t; self.requestPractice(t, PRACTICE.firstLeadMs); },
    /** Plan a (new) practice stand: one at a time, never after GO. */
    requestPractice(t, leadMs = PRACTICE.repeatLeadMs) {
      if (tGo !== null || t < practiceEnd + 1000) return false;
      practices += 1;
      practiceEnd = plan(t + leadMs, [[PRACTICE.riseMs, 1], [PRACTICE.holdMs, 1], [PRACTICE.descendMs, 0]]);
      return true;
    },
    /** Called with the core's readiness during the camera check: repeat the practice stand when the app asks for one. */
    react(t, readiness) {
      if (t0 === null) self.start(t);
      if (readiness && (readiness.reasons || []).some((x) => PRACTICE_AGAIN.has(x))) self.requestPractice(t);
    },
    go(t) {
      if (tGo !== null) return;
      tGo = t; segs = segs.filter((s) => s.t1 <= t);
      let tt = t + GO_LEAD_MS;
      for (const [r, h, d, s, top] of STANDS) tt = plan(tt, [[r, top], [h, top], [d, 0], [s, 0]]);
      plan(tt, [[LAST_RISE_MS, 1], [1500, 1], [1000, 0]]);
    },
    alphaAt,
    bodyAtTime: (t) => bodyAt(alphaAt(t)),
    landmarksAt(t) {
      const lm = landmarksOf(bodyAt(alphaAt(t)), vis);
      const slot = Math.round((t - (t0 ?? 0)) * 0.03);
      const r = rng((seed ^ Math.imul(slot + 1, 0x9e3779b1)) >>> 0);
      return lm.map(([x, y, z, v, p]) => [x + (noisePx * gauss(r)) / DEMO_W, y + (noisePx * gauss(r)) / DEMO_H, z, v, p]);
    },
    get tGo() { return tGo; },
    info() {
      return { seed, noisePx, practiceStands: practices, goLeadMs: GO_LEAD_MS, stands: STANDS.map(([r, h, d, s, top]) => ({ riseMs: r, holdMs: h, descendMs: d, sitMs: s, topAlpha: top })),
        lastRiseMs: LAST_RISE_MS, expected: EXPECTED };
    },
  };
  return self;
}

/** Map the demo onto the decision gate's live/replay distinction: the demo runs the live workflow in real time (its own
 *  GO, 30-s clock and checks); everything it produces is labelled as a simulated demo (report, review, export). */
export const decisionSource = (source) => (source === "demo" ? "camera" : source);

/** Fields added to the downloaded record of a demo session. */
export function demoRecord(script) {
  return { simulated: true, realPerson: false, realMeasurement: false, label: DEMO_LABEL,
    note: "The participant, the picture and the body points were generated by the app's demo script (no camera, no pose model). The numbers show how the workflow behaves; they describe no one.",
    script: script ? script.info() : null };
}

// ---- drawing (browser only) ----
const COL = { wall: "#eef2f6", floor: "#d9dee5", line: "#9aa3ad", wood: "#8b5a2b", woodDark: "#6e4520", trousers: "#2f3e56", shirt: "#3f7cac", sleeve: "#24496b", skin: "#e2b48e", shoe: "#222" };
function stroke(ctx, pts, width, color) {
  ctx.strokeStyle = color; ctx.lineWidth = width * S; ctx.lineCap = "round"; ctx.lineJoin = "round";
  ctx.beginPath(); ctx.moveTo(cx(pts[0][0]), cy(pts[0][1])); for (const p of pts.slice(1)) ctx.lineTo(cx(p[0]), cy(p[1])); ctx.stroke();
}
export function drawScene(ctx, b) {
  const W = ctx.canvas.width, H = ctx.canvas.height;
  ctx.fillStyle = COL.wall; ctx.fillRect(0, 0, W, H);
  ctx.fillStyle = COL.floor; ctx.fillRect(0, cy(FLOOR_Y), W, H - cy(FLOOR_Y));
  ctx.strokeStyle = COL.line; ctx.lineWidth = 3; ctx.beginPath(); ctx.moveTo(0, cy(FLOOR_Y)); ctx.lineTo(W, cy(FLOOR_Y)); ctx.stroke();
  // straight-backed chair, no arm rests (seat top about knee height minus a cushion)
  stroke(ctx, [[110, 1000], [110, 1175]], 18, COL.woodDark); stroke(ctx, [[322, 1000], [322, 1175]], 18, COL.woodDark);
  stroke(ctx, [[100, 640], [100, 1000]], 22, COL.wood);
  stroke(ctx, [[100, 700], [100, 900]], 30, COL.wood);
```

```
  stroke(ctx, [[100, 992], [332, 992]], 24, COL.wood);
  // figure: legs, trunk, head, near arm crossed on the chest
  stroke(ctx, [b.ankle, b.toe.map((v, i) => v + (i === 0 ? 12 : 0))], 30, COL.shoe);
  stroke(ctx, [b.hip, b.knee, b.ankle], 50, COL.trousers);
  stroke(ctx, [b.hip, b.knee], 62, COL.trousers);
  stroke(ctx, [[b.hip[0] + 0.12 * (b.sh[0] - b.hip[0]), b.hip[1] + 0.12 * (b.sh[1] - b.hip[1])], b.sh], 84, COL.shirt);
  stroke(ctx, [b.sh, [b.sh[0] + 0.25 * (b.head[0] - b.sh[0]), b.sh[1] + 0.25 * (b.head[1] - b.sh[1])]], 30, COL.skin);
  ctx.fillStyle = COL.skin; ctx.beginPath(); ctx.arc(cx(b.head[0]), cy(b.head[1]), 50 * S, 0, 2 * Math.PI); ctx.fill();
  stroke(ctx, [b.sh, b.elbow, b.wrist], 28, COL.sleeve);
  ctx.fillStyle = COL.skin; ctx.beginPath(); ctx.arc(cx(b.wrist[0]), cy(b.wrist[1]), 13 * S, 0, 2 * Math.PI); ctx.fill();
  // burned-in label: every frame (and so every evidence image) says what this is
  ctx.font = "bold 40px system-ui, sans-serif"; const w = ctx.measureText(CANVAS_LABEL).width;
  ctx.fillStyle = "#fff"; ctx.fillRect(18, 16, w + 28, 60); ctx.strokeStyle = "#b3261e"; ctx.lineWidth = 4; ctx.strokeRect(18, 1
6, w + 28, 60);
  ctx.fillStyle = "#b3261e"; ctx.textBaseline = "middle"; ctx.fillText(CANVAS_LABEL, 32, 47);
  ctx.font = "26px system-ui, sans-serif"; ctx.fillStyle = "#444"; ctx.fillText("Computer-drawn figure. Not a camera image, not a
real person.", 22, H - 24);
}

/** Start the offscreen scene and return its video stream (browser only). */
export function startDemoScene(script, { fps = 30, now = () => performance.now() } = {}) {
  const canvas = document.createElement("canvas"); canvas.width = DEMO_W; canvas.height = DEMO_H;
  const ctx = canvas.getContext("2d");
  script.start(now());
  let stopped = false, raf = null;
  const frame = () => { if (stopped) return; drawScene(ctx, script.bodyAtTime(now())); raf = requestAnimationFrame(frame); };
  frame();
  const stream = canvas.captureStream(fps);
  return { stream, stop() { stopped = true; if (raf !== null) cancelAnimationFrame(raf); stream.getTracks().forEach((t) => t.stop
()); } };
}

```

app/src/ui/draw.js

```
// Canvas drawing: skeleton overlay and the review timeline.
const BONES = [[11, 12], [11, 23], [12, 24], [23, 24], [23, 25], [25, 27], [24, 26], [26, 28], [11, 13], [13, 15], [12, 14], [14, 16], [27, 31], [28, 32]];

export function drawOverlay(canvas, video, poses, participant) {
  const w = video.videoWidth, h = video.videoHeight;
  if (!w) return;
  if (canvas.width !== w) { canvas.width = w; canvas.height = h; }
  const ctx = canvas.getContext("2d");
  ctx.clearRect(0, 0, w, h);
  const lw = Math.max(3, w / 180);
  for (const p of poses) {
    const isP = participant && p === participant;
    ctx.strokeStyle = isP ? "#18c964" : "#9a9a9a"; ctx.fillStyle = ctx.strokeStyle; ctx.lineWidth = isP ? lw * 1.6 : lw;
    for (const [a, b] of BONES) {
      if ((p[a][3] ?? 0) < 0.3 || (p[b][3] ?? 0) < 0.3) continue;
      ctx.beginPath(); ctx.moveTo(p[a][0] * w, p[a][1] * h); ctx.lineTo(p[b][0] * w, p[b][1] * h); ctx.stroke();
    }
    for (const i of [11, 12, 23, 24, 25, 26, 27, 28]) {
      if ((p[i][3] ?? 0) < 0.3) continue;
      ctx.beginPath(); ctx.arc(p[i][0] * w, p[i][1] * h, lw * 1.4, 0, 2 * Math.PI); ctx.fill();
    }
  }
}

/** Review timeline from the core trace and result. Shows exactly what was measured, including gaps. */
export function drawTimeline(canvas, trace, result, params) {
  const W = canvas.clientWidth || 900, H = canvas.height;
  canvas.width = W;
  const ctx = canvas.getContext("2d");
  ctx.clearRect(0, 0, W, H);
  const { tGo, tEnd } = result.window;
  const t0 = tGo - 1000, t1 = Math.max(tEnd, tGo + result.window.observedMs) + 500;
  const hSit = result.calibration.hSit, standH = result.calibration.stand ? result.calibration.stand.h : null;
  const lo = Math.min(hSit - 0.3, 0), hi = Math.max(1.4, (standH || 1.1) + 0.2);
  const X = (t) => 50 + ((t - t0) / (t1 - t0)) * (W - 60);
  const Y = (v) => H - 25 - ((v - lo) / (hi - lo)) * (H - 40);
  ctx.font = "13px system-ui";
  ctx.fillStyle = "rgba(179,38,30,0.18)";
  for (const g of result.gaps) ctx.fillRect(X(g.t0), 10, Math.max(2, X(g.t1) - X(g.t0)), H - 35);
  ctx.strokeStyle = "#333"; ctx.setLineDash([5, 4]);
  for (const [t, label] of [[tGo, "GO"], [tEnd, "30 s"]]) { ctx.beginPath(); ctx.moveTo(X(t), 8); ctx.lineTo(X(t), H - 25); ctx.stroke(); ctx.fillStyle = "#333"; ctx.fillText(label, X(t) + 4, 20); }
  const lines = [[hSit + params.seatReturnRise, "#888", "seated"], [result.calibration.halfway, "#1d4ed8", "halfway"]];
  if (standH) lines.push([standH, "#127a3a", "practice stand"]);
  for (const [v, col, label] of lines) { if (v === null || v === undefined) continue; ctx.strokeStyle = col; ctx.beginPath(); ctx.moveTo(50, Y(v)); ctx.lineTo(W - 10, Y(v)); ctx.stroke(); ctx.fillStyle = col; ctx.fillText(label, 2, Y(v) - 2); }
  ctx.setLineDash([]);
  ctx.strokeStyle = "#1d4ed8"; ctx.lineWidth = 2; ctx.beginPath(); let pen = false;
  for (const r of trace) {
    if (r.t < t0 || r.t > t1) continue;
    if (r.h === null) { pen = false; continue; }
    if (!pen) { ctx.moveTo(X(r.t), Y(r.h)); pen = true; } else ctx.lineTo(X(r.t), Y(r.h));
  }
  ctx.stroke(); ctx.lineWidth = 1;
  const COL = { full: "#127a3a", expiry: "#1d4ed8", uncertain: "#b85c00", partial: "#888" };
  ctx.font = "bold 15px system-ui";
  for (const e of result.events) {
    const t = e.tFull || e.tExpiry || e.tPeak || e.tRise; if (t === null || t === undefined) continue;
    const x = X(t); ctx.fillStyle = COL[e.cls] || "#333"; ctx.strokeStyle = ctx.fillStyle;
    ctx.beginPath(); ctx.moveTo(x, 12); ctx.lineTo(x, H - 25); ctx.stroke();
    ctx.fillText(e.cls === "partial" ? "·" : e.cls === "uncertain" ? `${e.n}?` : String(e.n), x + 3, 34 + (e.n % 2) * 16);
  }
  ctx.fillStyle = "#555"; ctx.font = "13px system-ui";
  for (let s = 0; s <= 30; s += 5) ctx.fillText(`${s}s`, X(tGo + s * 1000) - 8, H - 6);
}

```

app/src/ui/engine.js

```
// Pose engine: loads the locally vendored MediaPipe Pose Landmarker and turns video frames into the
// core's frame format {t, poses:[[x,y,z,visibility,presence] x33]}. No network access beyond this origin.
import { FilesetResolver, PoseLandmarker } from "../../vendor/tasks-vision/vision_bundle.mjs";

const WASM = new URL("../../vendor/tasks-vision/wasm", import.meta.url).href;
const MODEL = new URL("../../vendor/models/pose_landmarker_full.task", import.meta.url).href;

// One landmarker lives for the whole page, and MediaPipe VIDEO mode needs strictly increasing timestamps
// across ALL calls. Media time restarts at 0 for every replay, so the model gets its own monotonic clock;
// the measurement core still receives the source timestamps.
let lmClock = 0;
function clockFor() { const base = lmClock + 1000; let first = null;
  return (t) => { if (first === null) first = t; const ts = Math.max(lmClock + 1, Math.round(base + (t - first))); lmClock = ts;
return ts; }; }

export async function loadLandmarker({ numPoses = 2, preferGPU = true, modelUrl = MODEL } = {}) {
  const t0 = performance.now();
  const fileset = await FilesetResolver.forVisionTasks(WASM);
  let lastErr = null;
  for (const delegate of preferGPU ? ["GPU", "CPU"] : ["CPU"]) {
    try {
      const lm = await PoseLandmarker.createFromOptions(fileset, {
        baseOptions: { modelAssetPath: modelUrl, delegate }, runningMode: "VIDEO", numPoses,
        minPoseDetectionConfidence: 0.5, minPosePresenceConfidence: 0.5, minTrackingConfidence: 0.5,
      });
      return { landmarker: lm, delegate, startupMs: Math.round(performance.now() - t0) };
    } catch (e) { lastErr = e; }
  }
  throw lastErr || new Error("pose model failed to load");
}

/** Convert a PoseLandmarkerResult to the core's pose arrays. Missing visibility is kept as 0 (never assumed visible). */
export function toPoses(res) {
  return (res.landmarks || []).map((pose) => pose.map((p) => [p.x, p.y, p.z ?? 0, p.visibility ?? 0, p.presence ?? 0]));
}

/**
 * Live / real-time frame loop on a <video> element using requestVideoFrameCallback.
 * Timestamps: live camera -> rVFC 'now' (performance clock); file replay -> mediaTime (ms).
 * Frame accounting (audit H; MDN: presentedFrames gaps reveal frames with no callback):
 *   presentedSpan  = presentedFrames(last) - presentedFrames(first) + 1   (frames the compositor presented)
 *   callbacks      = rVFC callbacks received
 *   analysed       = frames passed to the pose model
 *   notAnalysed    = presentedSpan - analysed  (missed callbacks + busy + duplicates + failures)
 * onFrame({t, poses}) is called for every analysed frame; nothing is invented for frames that were not analysed.
 * Demo mode (source "demo"): no pose model runs; simulate(t) returns the simulated participant's poses for the frame at
 * performance time t (the video shows the demo's drawn scene). Timestamps and frame accounting are as for the camera.
 */
export function startFrameLoop(video, landmarker, { source, onFrame, onError, simulate = null }) {
  let stopped = false, busy = false, lastT = -Infinity, handle = null;
  const st = { callbacks: 0, analysed: 0, busySkips: 0, duplicates: 0, failures: 0, presentedFirst: null, presentedLast: null, la
stError: null };
  const inferMs = [];
  const clock = clockFor();
  const tick = (now, meta) => {
    if (stopped) return;
    handle = video.requestVideoFrameCallback(tick);
    st.callbacks += 1;
    if (meta && Number.isFinite(meta.presentedFrames)) { if (st.presentedFirst === null) st.presentedFirst = meta.presentedFrame
s; st.presentedLast = meta.presentedFrames; }
    if (busy) { st.busySkips += 1; return; }
    const t = source === "camera" || source === "demo" ? now : meta.mediaTime * 1000;
    if (!(t > lastT)) { st.duplicates += 1; return; }
    busy = true;
    try {
      const s = performance.now();
      const res = simulate ? null : landmarker.detectForVideo(video, clock(t));
      inferMs.push(performance.now() - s); if (inferMs.length > 300) inferMs.shift();
      lastT = t; st.analysed += 1;
      onFrame({ t, poses: simulate ? simulate(t) : toPoses(res) });
    } catch (e) { st.failures += 1; st.lastError = String(e && e.message || e); onError && onError(e); }
    finally { busy = false; }
  };
  handle = video.requestVideoFrameCallback(tick);
  return {
    stop() { stopped = true; if (handle !== null && video.cancelVideoFrameCallback) video.cancelVideoFrameCallback(handle); },
    stats() {
      const s = [...inferMs].sort((a, b) => a - b);
      const span = st.presentedFirst === null ? null : st.presentedLast - st.presentedFirst + 1;
      return { timestampDomain: source === "camera" ? "performance.now (camera frame presentation)" : source === "demo" ? "perfor
mance.now (simulated demo frames; no pose model)" : "mediaTime (video file)",
        presentedSpan: span, callbacks: st.callbacks, analysed: st.analysed, notAnalysed: span === null ? null : Math.max(0, span
- st.analysed),
        busySkips: st.busySkips, duplicates: st.duplicates, failures: st.failures, lastError: st.lastError,
```

```
        medianInferMs: s.length ? +s[s.length >> 1].toFixed(1) : null, p95InferMs: s.length ? +s[Math.floor(s.length * 0.95)].toFixed(1) : null };
    },
  };
}

/**
 * Deterministic replay (test harness only): seek frame by frame, wait until the seeked frame is actually
 * presented (rVFC) with explicit timeouts, pass a decoded ImageBitmap, use monotonic timestamps.
 * Every frame that times out is reported as a failure — never silently analysed from a stale image.
 */
export async function deterministicReplay(video, landmarker, { fps, onFrame, frameTimeoutMs = 3000, maxFrames = Infinity }) {
  const clock = clockFor();
  const n = Math.min(maxFrames, Math.floor(video.duration * fps));
  const failures = []; const inferMs = [];
  const wait = (p, ms, what) => Promise.race([p, new Promise((_, rej) => setTimeout(() => rej(new Error(what + " timeout")), ms))]);
  for (let i = 0; i < n; i++) {
    const target = (i + 0.5) / fps;
    try {
      const presented = new Promise((res) => video.requestVideoFrameCallback((now, meta) => res(meta)));
      const seeked = new Promise((res) => video.addEventListener("seeked", res, { once: true }));
      video.currentTime = target;
      await wait(seeked, frameTimeoutMs, "seek");
      const meta = await wait(presented, frameTimeoutMs, "present");
      const bmp = await createImageBitmap(video);
      const t = Math.round(i * 1000 / fps);
      const s = performance.now();
      const res = landmarker.detectForVideo(bmp, clock(t));
      inferMs.push(performance.now() - s);
      bmp.close();
      onFrame({ t, poses: toPoses(res), mediaTime: meta.mediaTime });
    } catch (e) { failures.push({ frame: i, error: String(e.message || e) }); }
  }
  const s = [...inferMs].sort((a, b) => a - b);
  return { frames: n, failures, medianInferMs: s.length ? +s[s.length >> 1].toFixed(1) : null, p95InferMs: s.length ? +s[Math.floor(s.length * 0.95)].toFixed(1) : null };
}

```

app/src/ui/evidence.js

```
// Opt-in temporary evidence images for post-test review (checkpoint 4; audit B/D of checkpoint 3).
// What the app controls: images are kept only as in-memory Blob references inside this tab while the review is
// open; the app does not upload them or save them to files; clear() releases the app's references and revokes
// object URLs. The browser itself may cache Blob data in its own temporary storage; the app cannot control that.
//
// Lifecycle guarantees (tested in tests/regression_cp4.test.mjs and the browser journeys):
//  - generation ownership: clear() starts a new generation; a pending encoder callback from an older generation
//    is discarded, so a cleared store cannot be repopulated by late work;
//  - ordering: images are inserted by timestamp even if encoder callbacks finish out of order;
//  - finalize(): stops capture, waits (bounded) for pending callbacks, then freezes the store; later captures and
//    late callbacks are ignored. Coverage is computed only after finalize().
//  - null encoder results (allowed by canvas.toBlob) are counted as failures, never as images.
//  - pins (overnight fix ON-01): images inside a pinned time range (the practice stand being asked about / accepted)
//    are exempt from the rolling trim, so the reference image survives however long the observer waits before GO.
//    A pin key holds one range; pinning the same key again replaces it; clear() removes all pins.
function canvasEncoder() {
  let canvas = null;
  return (video, w, h, cb) => {
    canvas = canvas || document.createElement("canvas");
    canvas.width = w; canvas.height = h;
    canvas.getContext("2d").drawImage(video, 0, 0, w, h);
    canvas.toBlob(cb, "image/jpeg", 0.72);
  };
}

export function createEvidenceStore({ maxWidth = 480, minIntervalMs = 150, keepBeforeMs = 20000, maxPending = 4, encode = null } = {}) {
  const enc = encode || canvasEncoder();
  let frames = [], urls = new Set(), lastT = -Infinity, pending = 0, bytes = 0, dropped = 0, skippedBusy = 0, late = 0;
  let gen = 0, frozen = false, sealed = false, waiters = [], pins = new Map();   // frozen: no new captures; sealed: no more callbacks accepted
  const pinned = (t) => { for (const [a, b] of pins.values()) if (t >= a && t <= b) return true; return false; };
  const revoke = (f) => { if (f && f.url) { URL.revokeObjectURL(f.url); urls.delete(f.url); f.url = null; } };
  const settle = () => { if (pending === 0) { const w = waiters; waiters = []; w.forEach((r) => r()); } };
  const boxOf = (pose) => {
    if (!pose) return null;
    let x0 = 1, y0 = 1, x1 = 0, y1 = 0, n = 0;
    for (const p of pose) { if ((p[3] ?? 0) < 0.3) continue; n++; x0 = Math.min(x0, p[0]); y0 = Math.min(y0, p[1]); x1 = Math.max(x1, p[0]); y1 = Math.max(y1, p[1]); }
    return n ? [Math.max(0, x0 - 0.02), Math.max(0, y0 - 0.02), Math.min(1, x1 + 0.02), Math.min(1, y1 + 0.02)] : null;
  };
  return {
    capture(video, t, pose, keepFromT = null, { priority = false } = {}) {
      if (frozen || t - lastT < minIntervalMs || !video.videoWidth) return;
      // (overnight ON-38: near a required anchor such as the 30-s mark, allow twice the pending encodes so a slow encoder
      //  does not leave the endpoint without an image)
      if (pending >= (priority ? 2 * maxPending : maxPending)) { skippedBusy++; return; }
      lastT = t;
      const w = Math.min(maxWidth, video.videoWidth), h = Math.round(video.videoHeight * w / video.videoWidth);
      const box = boxOf(pose), myGen = gen;
      pending++;
      enc(video, w, h, (blob) => {
        pending--;
        if (myGen !== gen || sealed) { late++; settle(); return; }          // stale generation, or arrived after finalize() settled
        if (!blob) { dropped++; settle(); return; }
        const f = { t, blob, box, w, h, url: null };
        let i = frames.length; while (i > 0 && frames[i - 1].t > t) i--; // ordered insert
        frames.splice(i, 0, f); bytes += blob.size || 0;
        const cutoff = keepFromT !== null ? keepFromT : t - keepBeforeMs;
        if (frames.length && frames[0].t < cutoff) {
          frames = frames.filter((x) => { if (x.t >= cutoff || pinned(x.t)) return true; bytes -= x.blob.size || 0; revoke(x); return false; });
        }
        settle();
      });
    },
    /** Stop capturing and wait (max timeoutMs) for pending callbacks; afterwards the store is frozen. */
    finalize(timeoutMs = 1500) {
      frozen = true;                                                // captures already started may still land
      const seal = () => { sealed = true; return this.stats(); };
      if (pending === 0) return Promise.resolve(seal());
      return new Promise((res) => { const to = setTimeout(() => { waiters = waiters.filter((w) => w !== done); res(seal()); }, timeoutMs); const done = () => { clearTimeout(to); res(seal()); }; waiters.push(done); });
    },
    get frozen() { return frozen; },
    /** Protect images in [t0, t1] from the rolling trim (one range per key; re-pinning a key replaces it). */
    pin(key, t0, t1) { if (Number.isFinite(t0) && Number.isFinite(t1)) pins.set(key, [t0, t1]); },
    unpin(key) { pins.delete(key); },
    times() { return frames.map((f) => f.t); },
    nearest(t, maxMs = Infinity) {
      let best = null, bd = Infinity;
      for (const f of frames) { const d = Math.abs(f.t - t); if (d < bd) { bd = d; best = f; } }
      return best && bd <= maxMs ? best : null;
```

```
    },
    at(t) { return frames.find((f) => Math.abs(f.t - t) <= 1) || null; },
    /** Images in [t0, t1], at most `max`, ALWAYS keeping the required anchor times (e.g. start, top, end). */
    between(t0, t1, max = 8, anchors = []) {
      const inRange = frames.filter((f) => f.t >= t0 && f.t <= t1);
      const keep = new Set(anchors.filter(Boolean));
      if (inRange.length <= max) return inRange;
      const rest = inRange.filter((f) => !keep.has(f));
      const slots = Math.max(0, max - keep.size);
      const step = rest.length > 1 && slots > 1 ? (rest.length - 1) / (slots - 1) : 1;
      const picked = new Set(keep);
      for (let i = 0; i < slots && i * step < rest.length; i++) picked.add(rest[Math.round(i * step)]);
      return inRange.filter((f) => picked.has(f));
    },
    url(frame) { if (!frame.url) { frame.url = URL.createObjectURL(frame.blob); urls.add(frame.url); } return frame.url; },
    clear() { gen++; for (const f of frames) revoke(f); for (const u of urls) URL.revokeObjectURL(u); urls = new Set(); frames = [];
bytes = 0; lastT = -Infinity; frozen = false; sealed = false; pins = new Map(); settle(); },
    stats() { return { frames: frames.length, bytes, dropped, skippedBusy, lateOrStale: late, pending, frozen, sealed, objectUrls: urls.size, pins: pins.size, firstT: frames.length ? frames[0].t : null, lastT: frames.length ? frames[frames.length - 1].t : null }; },
  };
}

```

app/src/ui/report.js

```
// Printable summary and plain-language texts. All eligibility comes from decide() (app/src/core/decision.js):
// the title, the value, the CDC line and the export use the SAME decision object (audit B, C).
export const STATUS_TEXT = {
  measured: "Measured automatically",
  needs_review: "Counted automatically — needs checking",
  withheld: "No automatic count — the camera could not measure the participant reliably",
  incomplete: "Incomplete — the video or test ended before 30 seconds",
  stopped: "Stopped early",
};
export const COMPLETION_TEXT = {
  completed_30s: "Completed the full 30 seconds",
  protocol_stop_arms: "Observer answered that arms were used to stand (CDC rule: stop the test and record 0)",
  stopped_early: "Stopped early by the observer — not a completed test",
  incomplete: "Ended before 30 seconds — not a completed test",
  interrupted: "Interrupted (browser tab hidden or camera stopped) — not a completed test",
};
export const FLAG_TEXT = {
  uncertain_rises: "Some rises could not be confirmed as full stands from the camera — decide each one in section 3.",
  extension_unverifiable_view: "The camera was not side-on enough to see knee and hip straightening — every rise needs your decision.",
  no_practice_stand: "No practice stand was recorded, so full standing cannot be judged automatically — every rise needs your decision.",
  gap_during_movement: "The participant could not be measured for a moment while moving.",
  another_person_overlapped: "Another person overlapped the participant.",
  participant_reacquired_with_other_person_in_view: "The participant was lost from view while someone else was visible, then found again.",
  identity_or_landmark_discontinuity: "Body points jumped faster than a person can move (possible mix-up with another person, or a tracking error).",
  intermittent_tracking: "The participant was only measured in part of the video frames.",
  seated_baseline_drift: "The participant sat lower during the test than during the camera check.",
  expiry_endpoint_not_observed: "The participant was not measured at the very end of the 30 seconds.",
  expiry_without_practice_stand: "The 30-second halfway rule used a default standing height (no practice stand).",
  no_accepted_reference: "No practice stand was accepted as the full-standing reference — every rise needs your decision.",
  stand_dipped_without_sitting: "While standing, the participant dipped deeply and came back up without sitting down (a half-sit or squat is not a new stand): check whether they actually sat.",
  reference_posture_warning: "The accepted practice stand showed bent knees or hips — check its image before relying on it.",
};
import { RISE_RESOLVED_FLAGS } from "../core/coverage.js";
import { DEMO_LABEL } from "./demo.js";
export const flagResolvedByRises = (f) => RISE_RESOLVED_FLAGS.has(f);

export function plainReason(r) {
  return r
    .replace(/^tracking_lost_before_end \((.+)\)$/, "The participant could not be seen at the end of the test ($1).")
    .replace(/^tracking_lost (.+) s \((.+) s\)$/, "The participant could not be measured from $1 s ($2 s).")
    .replace(/^participant_not_measurable_for_(\d+)pct_of_the_test$/, "The participant could not be measured for about $1% of the test (beyond short tolerated gaps).")
    .replace(/^frame_rate_too_low \((.+)\)$/, "Too few video frames could be analysed ($1).")
    .replace(/^video_frames_stopped_for_(.+)_s$/, "Video frames stopped arriving for $1 s.")
    .replace(/^page_hidden$/, "The browser tab was hidden or minimised during the test, so part of the test was not observed.")
    .replace(/^camera_error$/, "The camera stopped during the test.");
}
export const RISE_TEXT = {
  full: "Full stand (camera classification)",
  expiry: "Counted by the 30-second rule (over halfway up at 30 s)",
  uncertain: "Not clearly a full stand — needs your decision",
  partial: "Partial rise — not counted",
};
const RES_TEXT = { full: "observer: full stand", not_full: "observer: not a full stand", cant_tell: "observer: can't tell" };
const RES_TEXT_EXPIRY = { full: "observer: over halfway at 30 s (counts)", not_full: "observer: not over halfway at 30 s", cant_tell: "observer: can't tell" };
const isExpiry = (e) => e.cls === "expiry" || (e.flags || []).some((x) => /^expiry_/.test(x));
const SCAN_TEXT = { no_missed: "done: no stand missing", missed: "done: a missed stand could not be added" };

const esc = (s) => String(s ?? "").replace(/[&<>"]/g, (c) => ({ "&": "&amp;", "<": "&lt;", ">": "&gt;", '"': "&quot;" }[c]));
const secs = (t, r) => `${((t - r.window.tGo) / 1000).toFixed(1)} s`;

export function buildReport(sess, d) {
  const r = sess.result, cov = sess.coverage;
  const demo = sess.source === "demo";                    // simulated participant: labelled on every line that carries a result
  const source = sess.source === "camera" ? "live camera" : demo ? "SIMULATED DEMO (computer-drawn participant; no camera, no pose model) — not a real person, not a real measurement"
    : "REPLAY of a video file (" + esc(sess.replayName || "") + ") — not a live test";
  const ref = d.reference;
  const res = sess.review.resolutions || {};
  const covTxt = (n) => (!cov ? "no images" : cov.events[n] && cov.events[n].ok ? `image ${Math.round(cov.events[n].distMs)} ms from the top` : "no image near the top");
  const rises = r.events.map((e) => `<tr><td>${e.n}</td><td>${esc(RISE_TEXT[e.cls])}</td><td>${e.tFull ? secs(e.tFull, r) : e.tExpiry ? "30.0 s" : e.tPeak ? secs(e.tPeak, r) : "—"}</td><td>${esc(res[e.n] ? (isExpiry(e) ? RES_TEXT_EXPIRY : RES_TEXT)[res[e.n]] + (cov && cov.events[e.n] && cov.events[e.n].ok ? "" : " (ignored: no image)") : "—")}</td><td>${esc(covTxt(e.n))}</td></tr>`).join("")
    // added stands: status from decide() (overnight: the report used to re-derive it and could disagree)
    + (d.addedReview || []).map((a) => `<tr><td>added</td><td>Added by the observer (missed by the camera)</td><td>${Number.isFinite(a.t) ? secs(a.t, r) : "—"}</td><td>observer: full stand</td><td>${a.counted ? "picked from its image; counted" : "NOT counted: " + esc(a.problem)}</td></tr>`).join("");
```

```
  const refLine = ref.eligible
    ? `${demo ? "<strong>Demo only (simulated participant):</strong> " : ""}${ref.category === "below_average" ? "<strong>Below average</strong>" : "Not below average"} for ${esc(ref.band)} (below average = fewer than ${ref.threshold})${ref.fromRange ? " — decided from the range " + esc(d.valueText) + ", both ends give the same category" : ""}. The CDC form states: “A below average score indicates a risk for falls.”`
    : "Not applied: " + ref.blockers.map(esc).join(" ");
  const q = r.quality, ls = sess.loopStats || {}, rr = r.reference;
  const evid = !sess.evidenceOn ? "No images were kept, so the automatic count cannot be checked afterwards."
    : `Images kept while reviewing: ${sess.evidenceStats ? sess.evidenceStats.frames : 0} (failed captures ${sess.evidenceStats ? sess.evidenceStats.dropped : 0}); not included in this record.${cov ? ` Whole-test coverage: ${cov.scan.ok ? "complete" : "gaps at " + cov.scan.gaps.map(([a, b]) => secs(a, r) + "–" + secs(b, r)).join(", ")}; practice-stand image: ${cov.reference.ok ? "yes" : "no"}; 30-s endpoint image: ${cov.endpoint.ok ? "yes" : "no"}.` : ""} Identity confirmed for rises ${esc((sess.review.identityConfirmedFor || []).join(", ") || "none")}. Missed-stand scan: ${esc(SCAN_TEXT[sess.review.sessionScan] || "not done")}.`;
  return `
<article class="report">
  <h1 id="h-report-title">${esc(d.title)}</h1>
  ${demo ? `<p class="demolabel">${esc(DEMO_LABEL)}. This summary shows how the workflow behaves on a simulated participant; it describes no one.</p>` : ""}
  <p>${esc(new Date(sess.startedAt || Date.now()).toLocaleString())} · Source: <strong>${source}</strong> · ${esc(r.core)} · ${esc(d.version)}</p>
  <table>
    <tr><th>Test</th><td>${esc(COMPLETION_TEXT[d.completion])}</td></tr>
    <tr><th>Number of full stands reported</th><td class="final">${d.value === null ? esc(d.reference.fromRange ? d.valueText : "Not reported") : d.value}${demo ? ` <span class="demotag">${esc(DEMO_LABEL)}</span>` : ""}</td></tr>
    <tr><th>Where this number comes from</th><td>${esc(d.basis)}${sess.review.basisChoice ? `<br>Your choice: ${esc(sess.review.basisChoice.basis === "supervisor" ? "your own count" : "the camera count")} — “${esc(sess.review.basisChoice.rationale)}”` : ""}</td></tr>
    <tr><th>Counts side by side</th><td>Camera as detected: ${d.counts.machine ?? "–"} · camera after your checks: ${d.counts.reviewed ?? "–"}${d.automatic && !d.automatic.determinate ? ` (range ${d.automatic.min}–${d.automatic.max})` : ""} · your own count: ${d.counts.supervisor ?? "–"}${d.conflict ? " · <strong>the counts differ</strong>" : ""}</td></tr>
    <tr><th>Camera measurement</th><td>${esc(STATUS_TEXT[r.status])}${r.withheldReasons.length ? "<br>" + r.withheldReasons.map((x) => esc(plainReason(x))).join("<br>") : ""}</td></tr>
    <tr><th>Full-standing reference</th><td>${rr ? `A practice stand before GO (not part of the CDC form; practice repetitions are suggested in the Rikli &amp;amp; Jones protocol). Practice stand #${rr.id}: ${rr.accepted ? `accepted (${rr.accepted.basis === "image" ? "with its image" : "from direct observation"})` : "not accepted"}${rr.postureWarning ? "; the camera saw bent knees or hips" : ""}; knee/hip check: ${esc({ verified_side_view: "side view", unverifiable_view: "camera too frontal — not checked", no_accepted_reference: "no accepted reference", no_practice_stand: "no practice stand" }[r.extensionCheck] || r.extensionCheck)}` : "None (every rise needed the observer's decision)"}</td></tr>
    <tr><th>Evidence check</th><td>${evid}</td></tr>
    <tr><th>Observer answers</th><td>Arms used to stand: <strong>${esc(sess.review.arms || "not answered")}</strong> · Set-up as described: <strong>${esc(sess.review.setup || "not answered")}</strong></td></tr>
    <tr><th>CDC comparison table</th><td>${refLine}</td></tr>
    <tr><th>Measurement quality</th><td>Participant measured in ${q.usableFrames}/${q.framesInWindow} analysed frames (${(100 - q.missingObservationPct).toFixed(0)}%); longest gap ${q.longestGapMs} ms; ${q.usableFps} usable frames/s.${ls.presentedSpan ? ` Video frames: ${ls.presentedSpan} presented, ${ls.analysed} analysed, ${ls.notAnalysed} not analysed.` : ""}</td></tr>
  </table>
  <h2>Rises</h2>
  ${rises ? `<table><tr><th>#</th><th>What the camera found</th><th>Time</th><th>Your decision</th><th>Image</th></tr>${rises}</table>` : "<p>No rises detected.</p>"}
  <h2>Change record</h2>
  ${sess.audit.length ? `<table><tr><th>When</th><th>What</th></tr>${sess.audit.map((a) => `<tr><td>${esc(new Date(a.at).toLocaleTimeString())}</td><td>${esc(a.what)}</td></tr>`).join("")}</table>` : "<p>No changes.</p>"}
  <p class="disclaimer">
    What this is: a record of one supervised chair-stand session measured with an experimental camera tool. What it is not: a diagnosis, a fall-risk probability, a complete STEADI fall-risk assessment, or a medical device. It has not been validated in any population, including older adults.
    The CDC table (ages 60–94) is shown only for a completed live 30-second test (or the simulated demo) with the set-up and arm-use questions answered, age and column entered, and either a camera count whose reference, rises, identity and whole test were checked against images, or your own count chosen explicitly.
    Test protocol and table: CDC STEADI “Assessment: 30-Second Chair Stand” (2017), www.cdc.gov/steadi. CDC does not endorse this tool.
    This record contains no images. The app has no upload function.${demo ? ` ${esc(DEMO_LABEL)}: the participant and every number above were simulated by the app's demo script.` : ""}
  </p>
</article>`;
}
```

tests/core.test.mjs

```
// Unit tests: run with `npm test` (node --test). No browser, camera or video needed.
import test from "node:test";
import assert from "node:assert/strict";
import { SCENARIOS, runScenario } from "../eval/challenge.mjs";
import { cdcReference } from "../app/src/core/norms.js";
import { createChairStand } from "../app/src/core/chair_stand.js";
import { render, PRACTICE, PRACTICE_GO } from "../eval/synth.mjs";
import { decide } from "../app/src/core/decision.js";
import { acceptDetectedReference } from "../eval/observer_sim.mjs";

// Known, disclosed limitation (docs/PROTOCOL_CLAIM_CONTRACT.md): a person replacing the participant at the
// same spot and posture, with no visible transition, cannot be told apart by pose geometry alone.
const KNOWN_LIMITATIONS = new Set(["identity_trap_participant_lost_helper_in_place"]);

for (const sc of SCENARIOS) {
  test(`challenge: ${sc.id}`, { todo: KNOWN_LIMITATIONS.has(sc.id) ? "known limitation, see contract" : false }, () => {
    const r = runScenario(sc);
    assert.ok(r.pass, JSON.stringify(r));
  });
}

test("CDC table: examples straight from the form", () => {
  assert.deepEqual(cdcReference(11, 67, "men"), { applies: true, belowAverage: true, threshold: 12, band: "65–69, men" });
  assert.equal(cdcReference(12, 67, "men").belowAverage, false);
  assert.equal(cdcReference(3, 92, "women").belowAverage, true);
  assert.equal(cdcReference(4, 92, "women").belowAverage, false);
});

test("CDC table: never applied outside 60–94, without sex, or without a valid count", () => {
  assert.equal(cdcReference(15, 16, "men").applies, false);       // teenage demo participant
  assert.equal(cdcReference(10, 95, "women").applies, false);     // no row for 95+
  assert.equal(cdcReference(10, 70, null).applies, false);
  assert.equal(cdcReference(null, 70, "men").applies, false);
});

test("core: timestamps must increase", () => {
  const core = createChairStand({ width: 720, height: 1280 });
  core.push({ t: 10, poses: [] });
  assert.throws(() => core.push({ t: 10, poses: [] }));
});

test("core: cannot start before readiness", () => {
  const core = createChairStand({ width: 720, height: 1280 });
  core.push({ t: 0, poses: [] });
  assert.throws(() => core.start(0), /not ready/);
});

test("core: observer stop -> 'stopped', no score", () => {
  const d = render([...PRACTICE, ["sit", 800], ["rise", 700], ["stand", 400], ["descend", 700], ["sit", 8000]]);
  const core = createChairStand({ width: d.width, height: d.height });
  for (const f of d.frames) {
    if (core.phase === "calibrating") { core.push(f); acceptDetectedReference(core); if (f.t >= PRACTICE_GO && core.readiness().r
eady) core.start(f.t); continue; }
    core.push(f); if (f.t > PRACTICE_GO + 4000) break;
  }
  const r = core.finish("stopped", PRACTICE_GO + 4000);
  assert.equal(r.status, "stopped"); assert.equal(r.count, null); assert.equal(r.countSoFar, 1);
});

test("core: page hidden -> withheld even if frames looked fine", () => {
  const d = render([...PRACTICE, ["sit", 800], ["rise", 700], ["stand", 400], ["descend", 700], ["sit", 8000]]);
  const core = createChairStand({ width: d.width, height: d.height });
  for (const f of d.frames) {
    if (core.phase === "calibrating") { core.push(f); acceptDetectedReference(core); if (f.t >= PRACTICE_GO && core.readiness().r
eady) core.start(f.t); continue; }
    core.push(f); if (f.t > PRACTICE_GO + 3000) break;
  }
  const r = core.finish("page_hidden", PRACTICE_GO + 3000);
  assert.equal(r.status, "withheld"); assert.equal(r.count, null);
});

test("identity trap (pose-only limitation): the wrong automatic count cannot become an interpreted result", () => {
  const r = runScenario(SCENARIOS.find((s) => s.id === "identity_trap_participant_lost_helper_in_place"));
  // the core alone cannot tell (known limitation) ... (core 0.8 sends one rise of this scenario to review, but the
  // automatic count is still wrong and the true count is outside the reported range)
  assert.ok(["measured", "needs_review"].includes(r.status), r.status);
  assert.ok(r.countRange.max < r.refCount, JSON.stringify({ range: r.countRange, truth: r.refCount }));
  // ... but the decision layer never interprets an automatic count without images reviewed for identity
  const base = { result: { reason: "expired", status: "measured", window: { observedMs: 30000 }, events: Array.from({ length: r.c
ount }, (_, i) => ({ n: i + 1, cls: "full", counted: true, flags: [] })), reviewFlags: [] },
    source: "camera", observer: { arms: "no", setup: "yes" }, participant: { ageText: "70", column: "men" } };
  assert.equal(decide({ ...base, evidence: { available: false } }).reference.eligible, false);
  assert.equal(decide({ ...base, evidence: { available: true, identityConfirmed: null } }).reference.eligible, false);
  assert.equal(decide({ ...base, evidence: { available: true, identityConfirmed: false } }).reference.eligible, false);
```

```
});

```

tests/decision_matrix.test.mjs

```
// Table-driven decision tests.
//  v1 oracle (70 cases, checkpoint 3; independent agent, spec docs/DECISION_RULES.md) runs through a TRANSLATOR to the
//  v2 input shape: wherever v1 said "images kept" (evidence.available), the translator supplies FULL per-decision
//  coverage, an accepted image-backed reference, identity confirmed for every rise and a completed whole-test scan -
//  i.e. the v1 notion of "evidence" made explicit. Cases whose expected outcome INTENTIONALLY changes under the v2
//  contract (docs/CONTRACTS_CP4.md s3: the supervisor count needs an explicit choice; discrepant counts are a conflict)
//  are listed in V2_EXPECTATION_CHANGES with the reason; the v1 expectation is still reported.
//  v2 oracle (tests/fixtures/decision_matrix_v2_expected.json, separate independent agent) runs unchanged when present.
import test from "node:test";
import assert from "node:assert/strict";
import { readFileSync, existsSync } from "node:fs";
import { decide, parseStrictInt } from "../app/src/core/decision.js";

const v1 = JSON.parse(readFileSync(new URL("./fixtures/decision_matrix_expected.json", import.meta.url), "utf8"));
const v2Path = new URL("./fixtures/decision_matrix_v2_expected.json", import.meta.url);
const v2 = existsSync(v2Path) ? JSON.parse(readFileSync(v2Path, "utf8")) : [];

export function translateV1(s) {
  const r = structuredClone(s.result);
  r.window = { tGo: 6000, tEnd: 36000, ...(r.window || {}) };
  r.events = (r.events || []).map((e, i) => ({ tRise: 6000 + i * 2000, tPeak: 6800 + i * 2000, tFull: e.counted ? 6700 + i * 2000
: null, tSeat: 7600 + i * 2000, ...e }));
  const ev = s.evidence || {};
  const out = { ...s, result: r };
  if (ev.available) {
    r.reference = { id: 1, t0: 2000, t1: 2600, accepted: { basis: "image" }, usedAsReference: true, postureWarning: false };
    const coverage = { imageCount: 300, imageTimes: [], reference: { ok: true }, events: Object.fromEntries(r.events.map((e) =>
[e.n, { ok: true }])),
      flags: Object.fromEntries((r.reviewFlags || []).map((f) => [f, { ok: true }])), endpoint: { ok: true, required: false }, sc
an: { ok: true, gaps: [] } };
    out.evidence = { ...ev, coverage, identityConfirmedFor: ev.identityConfirmed === true ? r.events.map((e) => e.n) : [], sessio
nScan: "no_missed" };
  } else out.evidence = { ...ev, coverage: null };
  return out;
}

// Intentional v2 changes (reason from docs/CONTRACTS_CP4.md s3). Expected v2 outcome given explicitly.
const V2_EXPECTATION_CHANGES = {};
const noValue = { value: null, referenceEligible: false, category: null };
const reasonManual = "CONTRACTS_CP4 s3: the supervisor count becomes the result only by an explicit observer choice (basisChoice)
- v1 used it automatically";
const reasonConflict = "CONTRACTS_CP4 s3: an eligible automatic count and a DIFFERENT valid supervisor count are a conflict (no v
alue until a basis is chosen) - v1 silently kept the automatic count";
for (const id of ["withheld_manual_valid", "manual_used_when_no_evidence", "manual_used_when_range_straddles", "manual_with_range
_determinate", "manual_valid_60", "manual_valid_leading_zero"])
  V2_EXPECTATION_CHANGES[id] = { reason: reasonManual, expect: { completion: "completed_30s", ...noValue, mustIncludeBlockerKeywo
rds: ["choose"] } };
V2_EXPECTATION_CHANGES.manual_ignored_when_automatic_eligible = { reason: reasonConflict, expect: { completion: "completed_30s",
...noValue, conflict: true, mustIncludeBlockerKeywords: ["differ"] } };

function check(c, d, e, label) {
  const ctx = JSON.stringify({ why: c.why, got: { completion: d.completion, value: d.value, eligible: d.reference.eligible, categ
ory: d.reference.category, automatic: d.automatic && { min: d.automatic.min, max: d.automatic.max }, conflict: d.conflict, blockers:
d.reference.blockers } });
  if ("completion" in e) assert.equal(d.completion, e.completion, label + ctx);
  if ("value" in e) assert.equal(d.value, e.value, label + ctx);
  if ("referenceEligible" in e) assert.equal(d.reference.eligible, e.referenceEligible, label + ctx);
  if ("category" in e) assert.equal(d.reference.category, e.category, label + ctx);
  if (e.automatic) { assert.equal(d.automatic.min, e.automatic.min, ctx); assert.equal(d.automatic.max, e.automatic.max, ctx); as
sert.equal(d.automatic.determinate, e.automatic.determinate, ctx); }
  if ("automaticMin" in e) assert.equal(d.automatic && d.automatic.min, e.automaticMin, ctx);
  if ("automaticMax" in e) assert.equal(d.automatic && d.automatic.max, e.automaticMax, ctx);
  if ("conflict" in e) assert.equal(d.conflict, e.conflict, ctx);
  for (const kw of e.mustIncludeBlockerKeywords || []) assert.ok(d.reference.blockers.join(" ").toLowerCase().includes(kw), `bloc
ker keyword "${kw}" missing: ${ctx}`);
}

for (const c of v1) {
  test(`decision v1->v2: ${c.id}`, () => {
    const d = decide(translateV1(c.session));
    const ch = V2_EXPECTATION_CHANGES[c.id];
    check(c, d, ch ? ch.expect : c.expect, ch ? `[v2 change: ${ch.reason}] ` : "");
  });
}
for (const c of v2) test(`decision v2 oracle: ${c.id}`, () => check(c, decide(c.session), c.expect, ""));

test("strict integer parsing rejects decimals, exponents, signs, spaces and hex", () => {
  for (const bad of ["12.0", "1e1", "-3", " 7 ", "", "0x10", "7a", "+5", "٣"]) assert.equal(parseStrictInt(bad, 2, 0, 60), null,
bad);
  assert.equal(parseStrictInt("07", 2, 0, 60), 7);
  assert.equal(parseStrictInt("61", 2, 0, 60), null);
});
test("invariant: no unfinished or replayed session is ever eligible, whatever corrections say", () => {
  for (const reason of ["stopped", "stream_end", "page_hidden", "camera_error"]) for (const source of ["camera", "replay"]) {
```

```
    const d = decide(translateV1({ result: { reason, status: "measured", window: { observedMs: 7000 }, events: [], reviewFlags: [] }, source,
      observer: { arms: "no", setup: "yes" }, evidence: { available: true, identityConfirmed: true }, manual: { text: "20", reason: "manual" },
      participant: { ageText: "70", column: "men" }, basisChoice: { basis: "supervisor", rationale: "x" } }));
    assert.equal(d.reference.eligible, false); assert.equal(d.value, null);
  }
});

```

tests/demo.test.mjs

```
// Demo mode (simulated participant, app/src/ui/demo.js). The demo must be deterministic, its practice stand must be a
// genuine full stand (so accepting it is correct), and the UNCHANGED core must find exactly the outcome the demo is built
// to show: 10 full stands, 1 rise that needs the observer's decision, and 1 rise counted by the CDC 30-s rule.
// The demo's results are labelled as simulated everywhere they appear (report, record). Engineering tests of the demo
// script only: the demo is not evidence about real people.
import test from "node:test";
import assert from "node:assert/strict";
import { createChairStand } from "../app/src/core/chair_stand.js";
import { PARAMS } from "../app/src/core/params.js";
import { decide } from "../app/src/core/decision.js";
import { computeCoverage } from "../app/src/core/coverage.js";
import { legGeometry } from "../app/src/core/pose_features.js";
import { buildReport } from "../app/src/ui/report.js";
import { createDemoScript, bodyAt, landmarksOf, STANDS, EXPECTED, GO_LEAD_MS, LAST_RISE_MS, PRACTICE, DEMO_LABEL, DEMO_W, DEMO_H,
  decisionSource, demoRecord } from "../app/src/ui/demo.js";

/** Drive the demo like the app: frames at `fps` on a performance-like clock, the observer accepts (or first rejects) the
 *  practice card, waits `goDelay` ms, then GO. Optional frame-time jitter and dropped frames (seeded). */
function runDemo({ fps = 30, phase = 0, jitterMs = 0, drop = 0, goDelay = 2500, base = 123456.7, reject = 0, seed = undefined } =
{}) {
  const script = createDemoScript(seed === undefined ? {} : { seed });
  const core = createChairStand({ width: DEMO_W, height: DEMO_H });
  let r = 7; const rnd = () => { r = (r * 16807) % 2147483647; return r / 2147483647; };
  let acceptedAt = null, tGo = null, last = -Infinity, res = null, rejected = 0, ref = null;
  const times = [];
  script.start(base);
  for (let i = 0; i < 200000 && !res; i++) {
    let t = base + phase + (i * 1000) / fps + (jitterMs ? (rnd() - 0.5) * 2 * jitterMs : 0);
    if (t <= last) t = last + 0.5;
    if (drop && rnd() < drop) continue;
    last = t; times.push(t);
    const s = core.push({ t, poses: [script.landmarksAt(t)] });
    if (s.result) { res = s.result; break; }
    if (core.phase !== "calibrating") continue;
    script.react(t, s.readiness);
    const pr = s.readiness.practice;
    if (pr.state === "detected" && acceptedAt === null) {
      if (rejected < reject) { core.rejectReference(pr.refId); rejected += 1; continue; }
      core.freezeReference(pr.refId); assert.ok(core.acceptReference({ refId: pr.refId, basis: "image" })); acceptedAt = t; ref =
pr;
    }
    if (acceptedAt !== null && t > acceptedAt + goDelay && core.readiness().ready) { tGo = t + 3; core.start(tGo); script.go(tG
o); }
  }
  return { res, script, tGo, acceptedAt, ref, times, rejected };
}
const classes = (res) => res.events.reduce((m, e) => ({ ...m, [e.cls]: (m[e.cls] || 0) + 1 }), {});
const summary = (res) => ({ status: res.status, reason: res.reason, count: res.count, range: res.countRange, classes: classes(re
s),
  uncertainN: res.events.filter((e) => e.cls === "uncertain").map((e) => e.n), withheld: res.withheldReasons, expiry: res.expiry.
decision });

test("demo legs and trunk follow eval/synth.mjs skeleton(): hip height 1.1*sin(90deg*alpha), knee angle 90+90*alpha, side view",
() => {
  for (const a of [0, 0.25, 0.5, 0.75, 0.87, 1]) {
    const g = legGeometry(landmarksOf(bodyAt(a)), DEMO_W, DEMO_H, null);
    assert.ok(Math.abs(g.h - 1.1 * Math.sin((Math.PI / 2) * a)) < 0.01, `h at alpha ${a}: ${g.h}`);
    assert.ok(Math.abs(g.knee - (90 + 90 * a)) < 0.5, `knee at alpha ${a}: ${g.knee}`);
    assert.ok(g.viewRatio < PARAMS.sideViewMaxShoulderRatio, "side view");
  }
});

test("demo script is deterministic: same times give identical landmarks; another seed gives other jitter", () => {
  const a = createDemoScript(), b = createDemoScript(), c = createDemoScript({ seed: 1 });
  for (const s of [a, b, c]) s.start(1000);
  for (const t of [1000, 1033.3, 4200, 5100.5, 9000]) {
    assert.deepEqual(a.landmarksAt(t), b.landmarksAt(t));
    assert.notDeepEqual(a.landmarksAt(t), c.landmarksAt(t));
  }
  // jitter is small: a few pixels
  const clean = landmarksOf(bodyAt(a.alphaAt(4200)));
  const d = a.landmarksAt(4200).map((p, i) => Math.hypot((p[0] - clean[i][0]) * DEMO_W, (p[1] - clean[i][1]) * DEMO_H));
  assert.ok(Math.max(...d) < 10 && d.reduce((x, y) => x + y, 0) / d.length < 4, `jitter px: max ${Math.max(...d)}`);
});

test("demo session run twice gives the identical core result", () => {
  const x = runDemo(), y = runDemo();
  assert.deepEqual(JSON.parse(JSON.stringify(x.res)), JSON.parse(JSON.stringify(y.res)));
});

test("demo practice stand is a genuine full stand: detected, side view, no posture warning, accepted", () => {
  const { res, ref } = runDemo();
  assert.equal(ref.state, "detected");
  assert.equal(ref.sideView, true);
```

```
  assert.equal(ref.postureWarning, false);
  assert.ok(ref.knee > 176 && ref.hip > 176, `reference angles ${ref.knee}/${ref.hip}`);
  assert.ok(Math.abs(ref.h - 1.1) < 0.05, `reference h ${ref.h}`);
  assert.equal(res.reference.accepted.basis, "image");
  assert.equal(res.reference.usedAsReference, true);
  assert.equal(res.extensionCheck, "verified_side_view");
});

test("demo script design: one short rise inside the review band, cycles 2.4–2.8 s, the 30-s mark clearly over halfway on a rise", () => {
  const short = STANDS.filter((s) => s[4] < 1);
  assert.equal(short.length, 1);
  assert.equal(STANDS.indexOf(short[0]) + 1, EXPECTED.uncertainRise);
  const top = short[0][4];
  const kneeShort = 90 * (1 - top), hShort = 1.1 - 1.1 * Math.sin((Math.PI / 2) * top);
  assert.ok(kneeShort > PARAMS.fullKneeDeg && kneeShort <= PARAMS.partialFactor * PARAMS.fullKneeDeg, `knee shortfall ${kneeShort}`);
  assert.ok(hShort < PARAMS.fullH, `hip-height shortfall ${hShort}`);
  for (const [r, h, d, s] of STANDS) { const c = r + h + d + s; assert.ok(c >= 2400 && c <= 2800, `cycle ${c} ms`); }
  const tLast = GO_LEAD_MS + STANDS.reduce((x, [r, h, d, s]) => x + r + h + d + s, 0);
  const u = (PARAMS.windowMs - tLast) / LAST_RISE_MS;
  assert.ok(u > 0.5 && u < 0.8, `last rise ${Math.round(u * 100)}% done at 30 s`);
  const sc = createDemoScript(); sc.start(0); sc.go(10000);
  const hAt = (t) => 1.1 * Math.sin((Math.PI / 2) * sc.alphaAt(t));
  assert.ok(hAt(10000 + PARAMS.windowMs) - 0.55 > 3 * PARAMS.expiryNearHalfBand, "over halfway by a wide margin at 30 s");
  assert.ok(hAt(10000 + PARAMS.windowMs - 150) - 0.55 > 2 * PARAMS.expiryNearHalfBand, "...also 150 ms earlier (smoothing lag)");
  assert.ok(sc.alphaAt(10000 + PARAMS.windowMs) > sc.alphaAt(10000 + PARAMS.windowMs - 50), "rising at 30 s");
});

const CONDITIONS = [
  { name: "30 fps", o: {} },
  { name: "60 fps", o: { fps: 60 } },
  { name: "24 fps, frame-time jitter 6 ms", o: { fps: 24, jitterMs: 6 } },
  { name: "15 fps", o: { fps: 15 } },
  { name: "30 fps, 4% dropped frames, odd phase", o: { drop: 0.04, phase: 13.7 } },
  { name: "observer waits 40 s before GO", o: { goDelay: 40000 } },
  { name: "observer rejects the first practice stand (the demo repeats it)", o: { reject: 1 } },
];
for (const { name, o } of CONDITIONS) {
  test(`demo outcome (${name}): 10 full + 1 uncertain (rise 5) + 1 counted at 30 s -> count 11, range 11–12`, () => {
    const { res, rejected } = runDemo(o);
    assert.ok(res, "no result");
    if (o.reject) assert.equal(rejected, 1);
    const s = summary(res);
    assert.equal(s.reason, "expired");
    assert.equal(s.status, "needs_review");
    assert.deepEqual(s.withheld, []);
    assert.deepEqual(s.classes, { full: EXPECTED.full, uncertain: EXPECTED.uncertain, expiry: EXPECTED.expiry });
    assert.deepEqual(s.uncertainN, [EXPECTED.uncertainRise]);
    assert.equal(s.count, EXPECTED.counted);
    assert.deepEqual(s.range, { min: EXPECTED.rangeMin, max: EXPECTED.rangeMax });
    assert.equal(s.expiry, "counted_over_halfway");
    assert.deepEqual(res.reviewFlags, ["uncertain_rises"]);
    assert.deepEqual(res.protocolFlags, [], "arms seen crossed");
    const u = res.events.find((e) => e.cls === "uncertain");
    assert.ok(u.flags.includes("extension_not_confirmed"));
    assert.ok(u.shortfall.hip > PARAMS.fullHipDeg && u.shortfall.knee <= PARAMS.partialFactor * PARAMS.fullKneeDeg && u.shortfall.h < PARAMS.fullH, JSON.stringify(u.shortfall));
  });
}

// What the review screen does with this result, then the report and record (module level).
function reviewed() {
  const run = runDemo();
  const { res, times } = run;
  const imgs = []; for (const t of times) if (!imgs.length || t - imgs[imgs.length - 1] >= 150) imgs.push(t);   // ~6 images/s, as evidence.js
  const coverage = computeCoverage(res, imgs);
  const ids = res.events.filter((e) => e.counted || e.cls === "uncertain").map((e) => e.n);
  const review = { resolutions: { [EXPECTED.uncertainRise]: "not_full" }, flagChecks: {}, identityConfirmed: true, identityConfirmedFor: ids, sessionScan: "no_missed", added: [],
    basisChoice: null, arms: "no", setup: "yes", manual: null };
  const participant = { ageText: "72", column: "women" };
  const session = (source) => ({ result: res, source, observer: { arms: "no", setup: "yes" },
    evidence: { available: true, coverage, identityConfirmed: true, identityConfirmedFor: ids, eventResolutions: review.resolutions, flagChecks: {}, sessionScan: "no_missed", added: [] },
    manual: null, basisChoice: null, participant });
  return { ...run, coverage, review, participant, session, imgs };
}

test("demo review: deciding the short rise 'not a full stand' reports 11 with the CDC table (age 72, women: not below average)", () => {
  const { session, coverage } = reviewed();
  assert.equal(coverage.reference.ok, true); assert.equal(coverage.endpoint.required, true); assert.equal(coverage.endpoint.ok, true); assert.equal(coverage.scan.ok, true);
  const d = decide(session(decisionSource("demo")));
  assert.equal(d.completion, "completed_30s");
```

```
  assert.equal(d.value, 11);
  assert.deepEqual(d.reference.blockers, []);
  assert.equal(d.reference.eligible, true);
  assert.equal(d.reference.category, "not_below_average");
  assert.equal(d.reference.threshold, 10);
  // the decision gate itself is unchanged: an unmapped "demo" source would be treated like a replay (not live)
  assert.ok(decide(session("demo")).reference.blockers.some((b) => /not a live test/.test(b)));
});

test("decisionSource maps only the demo; camera and replay unchanged", () => {
  assert.equal(decisionSource("camera"), "camera");
  assert.equal(decisionSource("replay"), "replay");
  assert.equal(decisionSource("demo"), "camera");
});

test("demo report and record carry the simulated-demo label; a camera report does not", () => {
  const { res, coverage, review, participant, session, imgs, script } = reviewed();
  const d = decide(session(decisionSource("demo")));
  const sess = (source) => ({ source, result: res, coverage, review, participant, audit: [{ at: 0, what: "x" }], evidenceOn: tru
e,
    evidenceStats: { frames: imgs.length, dropped: 0 }, loopStats: {}, startedAt: 0, replayName: null });
  const html = buildReport(sess("demo"), d);
  assert.ok(html.split(DEMO_LABEL).length - 1 >= 3, "label at the top, next to the number and in the footer");
  assert.match(html, /Source: <strong>SIMULATED DEMO/);
  assert.match(html, /Demo only \(simulated participant\):<\/strong> Not below average for 70-74, women/);
  assert.equal(((html.match(/class="final">([^<]*)</) || [])[1] || "").trim(), "11");
  assert.match(html, /class="final">11 <span class="demotag">Simulated demo/);
  const cam = buildReport(sess("camera"), d);
  assert.ok(!cam.includes(DEMO_LABEL) && !/SIMULATED|Demo only/.test(cam));
  assert.match(cam, /Source: <strong>live camera<\/strong>/);
  const rec = demoRecord(script);
  assert.equal(rec.simulated, true); assert.equal(rec.realPerson, false); assert.equal(rec.realMeasurement, false);
  assert.equal(rec.label, DEMO_LABEL);
  assert.deepEqual(rec.script.expected, { ...EXPECTED });
  assert.equal(rec.script.practiceStands, 1);
  assert.deepEqual(JSON.parse(JSON.stringify(rec)), rec);
});

test("demo script: a practice stand is repeated only when the app asks, never after GO", () => {
  const sc = createDemoScript(); sc.start(0);
  assert.equal(sc.info().practiceStands, 1);
  const end1 = PRACTICE.firstLeadMs + PRACTICE.riseMs + PRACTICE.holdMs + PRACTICE.descendMs;
  sc.react(3000, { reasons: ["practice_stand_needed"] });                      // still planned/running: ignored
  assert.equal(sc.info().practiceStands, 1);
  sc.react(end1 + 2000, { reasons: ["hold_still_seated"] });                    // not asked
  assert.equal(sc.info().practiceStands, 1);
  sc.react(end1 + 2000, { reasons: ["practice_stand_needed"] });                // rejected: repeat
  assert.equal(sc.info().practiceStands, 2);
  assert.ok(sc.alphaAt(end1 + 2000 + PRACTICE.repeatLeadMs + PRACTICE.riseMs + 500) === 1, "second practice stand reaches full st
anding");
  sc.go(end1 + 20000);
  sc.react(end1 + 30000, { reasons: ["reference_stale"] });
  assert.equal(sc.info().practiceStands, 2);
  assert.equal(sc.alphaAt(end1 + 20000 + GO_LEAD_MS - 1), 0, "seated until GO + reaction time");
});

```

tests/regression_cp3.test.mjs

```
// Checkpoint-3 regression tests (written BEFORE the core repair; run against checkpoint3-start to record failures).
// Expectations come from what the synthetic movement IS (generator semantics), not from the core's thresholds:
//   - a 'rise' segment reaches full extension (alpha = 1); a 'partial' segment never does.
//   - Partial depth alpha is a thigh-angle parameter: alpha 0.6 = knees ~144 deg, alpha 0.95 = knees ~175 deg + trunk lean.
// Acceptable outcomes for movements near the limit of measurement are stated explicitly (count OR explicit review).
// Invariant (audit A/B): nothing that is not a supported full stand may silently raise a 'measured' count.
import test from "node:test";
import assert from "node:assert/strict";
import { createChairStand } from "../app/src/core/chair_stand.js";
import { render, skeleton, motion, PRACTICE, PRACTICE_GO, referencePositionCount } from "../eval/synth.mjs";
import { acceptDetectedReference } from "../eval/observer_sim.mjs";   // cp4: explicit practice-stand acceptance (full practice only)

const cyc = (n, rise = 800, stand = 400, desc = 800, sit = 400) =>
  Array.from({ length: n }, () => [["rise", rise], ["stand", stand], ["descend", desc], ["sit", sit]]).flat();
const partials = (n, alpha, dur = 1600) => Array.from({ length: n }, () => [["partial", dur, alpha], ["sit", 700]]).flat();

/** Run a script through the core the way the app does: practice prefix, Go at PRACTICE_GO when ready. */
export function run(script, opts = {}, { goAt = PRACTICE_GO, select = null, coreOpts = {} } = {}) {
  const data = render(script, { durationMs: goAt + 32000, noisePx: 1.0, ...opts });
  const core = createChairStand({ width: data.width, height: data.height, ...coreOpts });
  let started = false, res = null, readyAtGo = null;
  for (const f of data.frames) {
    if (select && core.selectParticipant && f.t === select.t) core.selectParticipant(select.x, select.y);
    if (!started && f.t >= goAt && core.phase === "calibrating") {
      readyAtGo = readyAtGo ?? core.readiness();
      if (core.readiness().ready) { core.start(f.t); started = true; }
    }
    const s = core.push(f);
    acceptDetectedReference(core);
    if (s.result) { res = s.result; break; }
  }
  if (started && !res) res = core.finish("stream_end", data.frames.at(-1).t);
  return { res, started, readyAtGo, core, data, tGo: started ? res.window.tGo : null };
}
const counted = (r) => r.events.filter((e) => e.counted !== false).length;            // old core: all events counted
const uncertain = (r) => r.events.filter((e) => e.cls === "uncertain").length;

// ---------- A. full stands vs partial rises ----------
test("control: 12 ordinary full stands (side view) are measured and counted", () => {
  const { res, tGo } = run([...PRACTICE, ...cyc(12), ["sit", 30000]]);
  const ref = referencePositionCount([...PRACTICE, ...cyc(12), ["sit", 30000]], tGo);
  assert.equal(res.status, "measured", JSON.stringify(res.reviewFlags));
  assert.equal(res.count, ref.count);
});

test("control: full stands with noise 3 px are not inflated and the true count is inside the reported range", () => {
  const script = [...PRACTICE, ...cyc(12), ["sit", 30000]];
  const { res, tGo } = run(script, { noisePx: 3, seed: 5 });
  const ref = referencePositionCount(script, tGo).count;
  assert.ok(counted(res) <= ref, `counted ${counted(res)} > ref ${ref}`);
  assert.ok(counted(res) + uncertain(res) >= ref, "true count outside reported range");
  if (counted(res) !== ref) assert.equal(res.status, "needs_review");
});

for (const alpha of [0.4, 0.6, 0.7]) {
  test(`partial depth alpha=${alpha}: five clearly incomplete rises are never counted as full stands`, () => {
    const { res } = run([...PRACTICE, ...partials(5, alpha), ["sit", 30000]]);
    assert.equal(counted(res), 0, JSON.stringify(res.events.map((e) => e.cls)));
  });
}
for (const alpha of [0.85, 0.95]) {
  test(`near-full depth alpha=${alpha}: not silently certified - either not counted, or counted only with review`, () => {
    const { res } = run([...PRACTICE, ...partials(5, alpha), ["sit", 30000]]);
    assert.ok(!(res.status === "measured" && counted(res) > 0), `silently measured ${counted(res)} stands`);
  });
}

test("mixed: 5 full + 1 incomplete (alpha 0.6) + 3 full -> 8 counted, the incomplete one not counted", () => {
  const script = [...PRACTICE, ...cyc(5), ["partial", 1600, 0.6], ["sit", 700], ...cyc(3), ["sit", 30000]];
  const { res } = run(script);
  assert.equal(counted(res), 8);
});

test("audit B: 5 full + one near-full (alpha 0.95) -> never a silent measured 6", () => {
  const { res } = run([...PRACTICE, ...cyc(5), ["partial", 1600, 0.95], ["sit", 30000]]);
  assert.ok(!(res.status === "measured" && counted(res) === 6));
  if (uncertain(res) > 0 || res.events.some((e) => (e.flags || []).length)) assert.notEqual(res.status, "measured", "event flag present but status measured");
});

test("pauses: standing pauses of 3 s and a 6 s seated rest keep the count", () => {
  const script = [...PRACTICE, ...cyc(3, 800, 3000, 800, 400), ["sit", 6000], ...cyc(4), ["sit", 30000]];
  const { res, tGo } = run(script);
```

```
  assert.equal(res.count, referencePositionCount(script, tGo).count);
  assert.equal(res.status, "measured");
});

test("habitual forward lean (17 deg stoop, also in the practice stand): full stands still counted", () => {
  const script = [...PRACTICE, ...cyc(10), ["sit", 30000]];
  const { res, tGo } = run(script, { leanExtra: 0.3 });
  assert.equal(res.count, referencePositionCount(script, tGo).count);
});

test("frontal view: extension cannot be verified -> never 'measured'; range contains the true count", () => {
  const script = [...PRACTICE, ...cyc(10), ["sit", 30000]];
  const { res, tGo } = run(script, { view: "frontal", W: 720, H: 1280 });
  const ref = referencePositionCount(script, tGo).count;
  assert.notEqual(res.status, "measured");
  assert.ok(counted(res) <= ref && counted(res) + uncertain(res) >= ref);
});

test("event time: a full stand's timestamp is within 200 ms of the generator's full-extension moment", () => {
  const script = [...PRACTICE, ...cyc(6), ["sit", 30000]];
  const { res } = run(script);
  const m = motion(script); const full = m.stands.filter((t) => t > PRACTICE_GO);
  const times = res.events.filter((e) => e.counted !== false).map((e) => e.tFull ?? e.tStand);
  assert.equal(times.length, full.length);
  times.forEach((t, i) => assert.ok(Math.abs(t - full[i]) <= 200, `event ${i + 1}: ${t} vs ${full[i]}`));
});

// ---------- D. expiry: CDC position rule ----------
function expiryScript(fracTarget, { riseMs = 2000, paused = false } = {}) {
  // 10 cycles, then a rise whose hip-height fraction at expiry equals fracTarget (paused: rise stops there)
  const base = [...PRACTICE, ...cyc(10, 800, 400, 800, 400)];
  const baseEnd = base.reduce((a, s) => a + s[1], 0);
  const tEnd = PRACTICE_GO + 30000;
  const alpha = Math.asin(fracTarget) / (Math.PI / 2);
  if (paused) {
    const riseStart = tEnd - 1500;
    return [...base, ["sit", riseStart - baseEnd], ["partial_hold", 1000, alpha], ["hold", 3000, alpha]];
  }
  const u = Math.acos(1 - 2 * alpha) / Math.PI;
  return [...base, ["sit", tEnd - u * riseMs - baseEnd], ["rise", riseMs], ["stand", 3000]];
}
// A paused attempt is rendered by a custom frame source (the motion() helper has no 'hold' segment).
function runPaused(fracTarget) {
  const tEnd = PRACTICE_GO + 30000; const alpha = Math.asin(fracTarget) / (Math.PI / 2);
  const base = [...PRACTICE, ...cyc(10, 800, 400, 800, 400)];
  const m = motion(base); const baseEnd = m.totalMs; const riseStart = tEnd - 1500;
  const frames = [];
  for (let i = 0; ; i++) {
    const t = Math.round(i * 1000 / 30); if (t > tEnd + 2000) break;
    let a = t < baseEnd ? m.alphaAt(t) : 0;
    if (t >= riseStart && t < riseStart + 1000) a = alpha * (t - riseStart) / 1000; else if (t >= riseStart + 1000) a = alpha;
    frames.push({ t, poses: [skeleton(a)] });
  }
  const core = createChairStand({ width: 720, height: 1280 }); let res = null;
  for (const f of frames) { if (core.phase === "calibrating" && f.t >= PRACTICE_GO && core.readiness().ready) core.start(f.t); co
nst s = core.push(f); acceptDetectedReference(core); if (s.result) { res = s.result; break; } }
  return { res, fullBefore: m.stands.filter((t) => t > PRACTICE_GO && t <= tEnd).length };
}

test("expiry: paused above halfway (frac 0.64) at 30 s -> counted by the position rule or explicit review (audit D)", () => {
  const { res, fullBefore } = runPaused(0.64);
  assert.ok(res.count === fullBefore + 1 || res.status === "needs_review", `count ${res.count}, fullBefore ${fullBefore}, ${res.s
tatus}`);
});
test("expiry: paused below halfway (frac 0.30) -> not counted", () => {
  const { res, fullBefore } = runPaused(0.30);
  assert.equal(res.count, fullBefore);
});
for (const [name, frac, expectAdd] of [["over halfway, rising (0.75)", 0.75, 1], ["under halfway, rising (0.25)", 0.25, 0]]) {
  test(`expiry: ${name}`, () => {
    const script = expiryScript(frac);
    const { res, tGo } = run(script);
    const ref = referencePositionCount(script, tGo);
    assert.equal(ref.case, expectAdd ? "over" : "under");
    assert.equal(res.count, ref.count);
  });
}
test("expiry: exactly halfway (0.50) -> explicit review, not a silent decision", () => {
  const script = expiryScript(0.5);
  const { res, tGo } = run(script);
  assert.equal(referencePositionCount(script, tGo).case, "near");
  assert.equal(res.status, "needs_review");
});
test("expiry: descending after an already counted stand -> no double count", () => {
  const base = [...PRACTICE, ...cyc(10, 800, 400, 800, 400)];
  const baseEnd = base.reduce((a, s) => a + s[1], 0); const tEnd = PRACTICE_GO + 30000;
  const script = [...base, ["sit", tEnd - 1500 - baseEnd], ["rise", 700], ["stand", 300], ["descend", 2000], ["sit", 3000]];
```

```
  const { res, tGo } = run(script);
  assert.equal(res.count, referencePositionCount(script, tGo).count);
});
test("expiry: endpoint not visible (participant unseen for the last 0.5 s) -> not a silent measured result", () => {
  const script = [...PRACTICE, ...cyc(12), ["sit", 30000]];
  const { res } = run(script, { poseGaps: [[PRACTICE_GO + 29500, PRACTICE_GO + 31000]] });
  assert.notEqual(res.status, "measured");
});

// ---------- G/H. readiness continuity and quality accounting ----------
test("readiness: two seated frames 1 s apart are not 'still and seated'", () => {
  const c = createChairStand({ width: 720, height: 1280 });
  c.push({ t: 0, poses: [skeleton(0)] }); c.push({ t: 1000, poses: [skeleton(0)] });
  assert.equal(c.readiness().ready, false);
});
test("readiness: start() long after the last frame is refused", () => {
  const d = render([...PRACTICE], { durationMs: PRACTICE_GO });
  const c = createChairStand({ width: d.width, height: d.height });
  for (const f of d.frames) { c.push(f); acceptDetectedReference(c); }
  assert.throws(() => c.start(PRACTICE_GO + 59000));
});
test("quality: half the frames without a pose is reported as ~50% missing, and not silently 'measured'", () => {
  const script = [...PRACTICE, ["sit", 33000]];
  const data = render(script, { durationMs: PRACTICE_GO + 32000, noisePx: 0 });
  for (const f of data.frames) if (f.t > PRACTICE_GO && Math.floor((f.t - PRACTICE_GO) / 100) % 2 === 0) f.poses = [];
  const c = createChairStand({ width: 720, height: 1280 }); let res = null;
  for (const f of data.frames) { if (c.phase === "calibrating" && f.t >= PRACTICE_GO && c.readiness().ready) c.start(f.t); const
s = c.push(f); acceptDetectedReference(c); if (s.result) { res = s.result; break; } }
  assert.ok(res.quality.missingObservationPct >= 40, JSON.stringify(res.quality));
  assert.notEqual(res.status, "measured");
});

// ---------- F. people in view ----------
test("helper standing beside (static) does not disturb an ordinary measurement", () => {
  const script = [...PRACTICE, ...cyc(12), ["sit", 30000]];
  const { res, tGo } = run(script, { W: 1280, H: 1280, helper: { x: 1000 } });
  assert.equal(res.count, referencePositionCount(script, tGo).count);
});
test("helper walks in from the side during the test and stays beside: count unaffected", () => {
  const script = [...PRACTICE, ...cyc(12), ["sit", 30000]];
  const walkIn = (t) => (t < PRACTICE_GO + 8000 ? null : Math.max(1000, 1300 - (t - PRACTICE_GO - 8000) * 0.3));
  const { res, tGo } = run(script, { W: 1280, H: 1280, helper: { xAt: walkIn } });
  assert.equal(res.count, referencePositionCount(script, tGo).count);
});
test("helper crosses in front of the participant: never a silent measured result", () => {
  const script = [...PRACTICE, ...cyc(12), ["sit", 30000]];
  const cross = (t) => (t > PRACTICE_GO + 10000 && t < PRACTICE_GO + 13000 ? 1100 - (t - PRACTICE_GO - 10000) * 0.3 : 1100);
  const { res } = run(script, { W: 1280, H: 1280, helper: { xAt: cross } });
  assert.notEqual(res.status, "measured");
});
test("short participant dropout (0.4 s) while a helper is in view -> review (identity may have switched)", () => {
  const script = [...PRACTICE, ...cyc(12), ["sit", 30000]];
  const { res } = run(script, { W: 1280, H: 1280, helper: { x: 1000 }, poseGaps: [[PRACTICE_GO + 12000, PRACTICE_GO + 12400]] });
  assert.equal(res.status, "needs_review");
});
test("two SEATED people (fixed fixture): not ready until the participant is selected; then counts correctly", () => {
  const script = [...PRACTICE, ...cyc(12), ["sit", 30000]];
  const noSel = run(script, { W: 1280, H: 1280, helper: { x: 1000, seated: true } });
  assert.equal(noSel.started, false, "started without knowing who the participant is");
  const withSel = run(script, { W: 1280, H: 1280, helper: { x: 1000, seated: true } }, { select: { t: 500, x: 250 / 1280, y: 950
/ 1280 } });
  assert.equal(withSel.started, true);
  assert.equal(withSel.res.count, referencePositionCount(script, withSel.tGo).count);
});
test("no practice stand (skipped): extension unverifiable -> never 'measured'", () => {
  const script = [["sit", 3000], ...cyc(12), ["sit", 30000]];
  const { res } = run(script, {}, { goAt: 2500, coreOpts: { practiceStand: "skip" } });
  assert.ok(res && res.status !== "measured");
});

```

tests/regression_cp4.test.mjs

```
// Checkpoint-4 regression tests, written BEFORE the repair (run on checkpoint4-start to record failures).
// Sources: independent CP3 audit findings A-E (+ the float completion bug found in this run) and docs/CONTRACTS_CP4.md.
// Movement expectations come from the generator's semantics (alpha 1 = full extension), not from core thresholds.
import test from "node:test";
import assert from "node:assert/strict";
import { createChairStand } from "../app/src/core/chair_stand.js";
import { decide, completionOf } from "../app/src/core/decision.js";
import { skeleton, motion, render, PRACTICE, PRACTICE_GO } from "../eval/synth.mjs";

let computeCoverage = null, createEvidenceStore = null;
try { ({ computeCoverage } = await import("../app/src/core/coverage.js")); } catch { /* not yet implemented */ }
try { ({ createEvidenceStore } = await import("../app/src/ui/evidence.js")); } catch { /* browser-only before cp4 */ }

const cyc = (n) => Array.from({ length: n }, () => [["rise", 800], ["stand", 400], ["descend", 800], ["sit", 400]]).flat();
const SCRIPT = [...PRACTICE, ...cyc(5), ["sit", 30000]];
const M = motion(SCRIPT);

/**
 * Drive the core like the app. observer: 'honest' accepts a reference only if the generator's practice stand was full
 * (depth === 1); 'careless' accepts whatever was detected; 'none' never answers.
 */
function runDepth(calDepth, testDepth, { observer = "honest", view = "side", withoutReference = false, mutate = null, basis = "ob
server_watched" } = {}) {
  const c = createChairStand({ width: 720, height: 1280 });
  let started = false, answered = null;
  for (let i = 0; i < 1200; i++) {
    const t = Math.round(i * 1000 / 30), depth = t < PRACTICE_GO ? calDepth : testDepth;
    if (mutate) mutate(c, t);
    if (c.phase === "calibrating" && t >= PRACTICE_GO && !started) {
      const r = c.readiness();
      const pr = r.practice || {};
      if (c.acceptReference && pr.refId !== undefined && pr.state === "detected" && observer !== "none" && answered !== pr.refId)
{
        answered = pr.refId;
        if (observer === "careless" || (observer === "honest" && calDepth === 1)) c.acceptReference({ refId: pr.refId, basis });
        else if (c.rejectReference) c.rejectReference(pr.refId);
      }
      if (withoutReference && c.start.length >= 1) { try { c.start(t, { withoutReference: true }); started = true; } catch { /* n
ot ready */ } }
      else if (c.readiness().ready) { c.start(t); started = true; }
    }
    const s = c.push({ t, poses: [skeleton(M.alphaAt(t) * depth, { view })] });
    if (s.result) return { r: s.result, started, core: c };
  }
  return { r: started ? c.finish("stream_end", 39967) : null, started, core: c };
}
const baseSession = (r, extra = {}) => ({ result: r, source: "camera", observer: { arms: "no", setup: "yes" }, participant: { age
Text: "70", column: "men" },
  evidence: { available: true, identityConfirmed: true, eventResolutions: {}, flagChecks: {} }, ...extra });

// ---------- A. reference acceptance ----------
test("A control: full practice + 5 full stands, reference accepted -> 5 measured", () => {
  const { r } = runDepth(1, 1);
  assert.equal(r.status, "measured"); assert.equal(r.count, 5);
});
for (const d of [0.6, 0.7, 0.85, 0.95]) {
  test(`A: incomplete practice (depth ${d}) with an honest observer -> never 5 automatic full stands`, () => {
    const { r } = runDepth(d, d);
    assert.ok(!r || r.status !== "measured" || r.count === 0, JSON.stringify(r && { status: r.status, count: r.count }));
  });
  test(`A: incomplete practice (depth ${d}), 'count without a reference' -> every rise uncertain, nothing counted`, () => {
    const { r } = runDepth(d, d, { observer: "none", withoutReference: true });
    assert.ok(r, "could not start without a reference");
    assert.equal(r.events.filter((e) => e.counted).length, 0);
    assert.notEqual(r.status, "measured");
  });
}
test("A: a detected but unconfirmed practice stand never becomes 'verified' (start refused)", () => {
  const { started } = runDepth(1, 1, { observer: "none" });
  assert.equal(started, false);
});
test("A: careless acceptance of a strongly bent reference (depth 0.6, side view) is flagged, never silently measured", () => {
  const { r } = runDepth(0.6, 0.6, { observer: "careless" });
  assert.ok(r.reviewFlags.includes("reference_posture_warning"), JSON.stringify(r.reviewFlags));
  assert.notEqual(r.status, "measured");
  assert.equal(decide(baseSession(r)).reference.eligible, false);
});
test("A: valid practice then incomplete test rises still rejected/flagged (existing control)", () => {
  for (const d of [0.6, 0.85]) { const { r } = runDepth(1, d); assert.ok(r.status !== "measured" || r.count === 0); }
});
test("A: frontal view with an accepted reference still cannot verify extension", () => {
  const { r } = runDepth(1, 1, { view: "frontal" });
  assert.notEqual(r.status, "measured");
});
```

```
test("A: reselecting the participant after acceptance voids the acceptance", () => {
  // (test fix during cp4: the first draft used the 5-rep script, whose first rep starts at 5.7 s, so the participant
  //  was legitimately not ready at 6.6 s; a practice-only motion is used instead - the assertion is unchanged)
  const M = motion([...PRACTICE, ["sit", 30000]]);
  const c = createChairStand({ width: 720, height: 1280 });
  let refId = null;
  for (let i = 0; i < 200; i++) {
    const t = Math.round(i * 1000 / 30);
    c.push({ t, poses: [skeleton(M.alphaAt(t))] });
    const pr = c.readiness().practice || {};
    if (refId === null && pr.state === "detected") { refId = pr.refId; c.acceptReference({ refId, basis: "observer_watched" }); }
  }
  assert.ok(refId !== null && c.readiness().ready);
  c.selectParticipant(400 / 720, 900 / 1280);
  for (let i = 200; i < 260; i++) c.push({ t: Math.round(i * 1000 / 30), poses: [skeleton(0)] });
  assert.equal(c.readiness().ready, false);
});
test("A: an accepted reference older than 5 minutes is stale at GO", () => {
  const c = createChairStand({ width: 720, height: 1280 });
  let refId = null, t = 0;
  for (; t < 5700; t += 33) { c.push({ t, poses: [skeleton(M.alphaAt(t))] }); const pr = c.readiness().practice || {}; if (refId
=== null && pr.state === "detected") { refId = pr.refId; c.acceptReference({ refId, basis: "observer_watched" }); } }
  for (; t < 5700 + 5 * 60000 + 2000; t += 33) c.push({ t, poses: [skeleton(0)] });
  assert.equal(c.readiness().ready, false);
  assert.ok(c.readiness().reasons.some((x) => /stale|practice/.test(x)));
});
test("A: camera zoom after acceptance (shin length +30%) voids the reference", () => {
  const c = createChairStand({ width: 720, height: 1280 });
  let refId = null, t = 0;
  for (; t < 5700; t += 33) { c.push({ t, poses: [skeleton(M.alphaAt(t))] }); const pr = c.readiness().practice || {}; if (refId
=== null && pr.state === "detected") { refId = pr.refId; c.acceptReference({ refId, basis: "observer_watched" }); } }
  const zoom = (lm) => lm.map(([x, y, z, v, p]) => [(x - 0.55) * 1.3 + 0.55, (y - 0.9) * 1.3 + 0.9, z, v, p]);
  for (let k = 0; k < 90; k++, t += 33) c.push({ t, poses: [zoom(skeleton(0))] });
  assert.equal(c.readiness().ready, false);
});

// ---------- completion float bug (found in this run's diagnostic) ----------
test("completion: GO at a fractional video time (11125.639 ms) + expiry = a completed 30-s test", () => {
  const r = { reason: "expired", window: { observedMs: (11125.639 + 30000) - 11125.639 } };
  assert.equal(completionOf(r, { arms: "no" }), "completed_30s");
});

// ---------- B. evidence coverage ----------
function coveredSession(r, frameTimes, extra = {}) {
  const coverage = computeCoverage ? computeCoverage(r, frameTimes) : null;
  const counted = r.events.filter((e) => e.cls !== "partial").map((e) => e.n);
  return baseSession(r, { evidence: { available: frameTimes.length > 0, coverage, identityConfirmed: true, identityConfirmedFor:
counted,
    eventResolutions: {}, flagChecks: {}, sessionScan: "no_missed", referenceImage: true }, ...extra });
}
const full = () => runDepth(1, 1, { basis: "image" }).r;           // image-backed acceptance (evidence kept)
const denseTimes = (r, from, to, step = 150) => { const a = []; for (let t = from; t <= to; t += step) a.push(t); return a; };

test("B control: dense images over practice + whole test -> automatic count eligible after review", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = full();
  const d = decide(coveredSession(r, denseTimes(r, 0, r.window.tEnd + 500)));
  assert.equal(d.reference.eligible, true, JSON.stringify(d.reference.blockers)); assert.equal(d.value, 5);
});
test("B: no images -> not eligible, whatever the answers", () => {
  const r = full(); const d = decide(coveredSession(r, []));
  assert.equal(d.reference.eligible, false);
});
test("B: ONE old image (1 s, before GO) -> not eligible (audit B)", () => {
  const r = full(); const d = decide(coveredSession(r, [1000]));
  assert.equal(d.reference.eligible, false);
});
test("B: missing the image near one rise's top -> not eligible", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = full(); const e3 = r.events[2]; const top = e3.tFull;
  const times = denseTimes(r, 0, r.window.tEnd + 500).filter((t) => Math.abs(t - top) > 400);
  const d = decide(coveredSession(r, times));
  assert.equal(d.reference.eligible, false);
  assert.ok(d.reference.blockers.join(" ").match(/rise 3|Rise 3/), JSON.stringify(d.reference.blockers));
});
test("B: gap > 1 s in the whole-test images -> missed stands cannot be ruled out -> not eligible", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = full(); const g0 = r.window.tGo + 20000;
  const times = denseTimes(r, 0, r.window.tEnd + 500).filter((t) => t < g0 || t > g0 + 1500);
  assert.equal(decide(coveredSession(r, times)).reference.eligible, false);
});
test("B: no image of the practice stand -> reference acceptance by image not possible -> not eligible", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = full(); const st = r.reference || r.calibration.stand;
  const times = denseTimes(r, 0, r.window.tEnd + 500).filter((t) => t < st.t0 - 400 || t > st.t1 + 400);
  assert.equal(decide(coveredSession(r, times)).reference.eligible, false);
});
```

```
test("B: identity confirmed for fewer rises than were counted -> not eligible", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = full(); const s = coveredSession(r, denseTimes(r, 0, r.window.tEnd + 500)); s.evidence.identityConfirmedFor = [1, 2, 3];
  assert.equal(decide(s).reference.eligible, false);
});

// ---------- C. correction both ways + explicit basis ----------
test("C: a partial rise resolved 'full' WITH its image is counted", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = runDepth(1, 0.6, { basis: "image" }).r;
  const s = coveredSession(r, denseTimes(r, 0, r.window.tEnd + 500));
  s.evidence.eventResolutions = Object.fromEntries(r.events.map((e) => [e.n, "full"]));
  const d = decide(s);
  assert.equal(d.automatic.min, r.events.length);
});
test("C: a partial rise resolved 'full' WITHOUT its image is ignored", () => {
  const r = runDepth(1, 0.6).r;
  const s = baseSession(r, { evidence: { available: false, eventResolutions: Object.fromEntries(r.events.map((e) => [e.n, "full"])) } });
  const d = decide(s); assert.ok(!d.automatic || d.automatic.min === 0);
});
test("C: an observer-added missed stand at an image time is counted", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = full(); const times = denseTimes(r, 0, r.window.tEnd + 500);
  const s = coveredSession(r, times); s.evidence.added = [{ id: "added-1", t: times.find((t) => t > r.window.tGo + 25000) }];
  assert.equal(decide(s).value, 6);
});
test("C: an observer-added stand with no image at that time is not counted (overnight s9: and it blocks interpretation)", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const r = full(); const times = denseTimes(r, 0, r.window.tEnd + 500);
  const s = coveredSession(r, times); s.evidence.added = [{ id: "added-1", t: r.window.tGo + 25071 }];
  // cp4 expectation was value 5 (silently ignored). Overnight contract s9 (ON-02): an invalid added stand is still never
  // counted, but it now blocks interpretation instead of disappearing silently, so the observer sees and undoes it.
  const d = decide(s);
  assert.notEqual(d.value, 6); assert.equal(d.reference.eligible, false);
  assert.ok(/not at a kept image/.test(d.reference.blockers.join(" ")), d.reference.blockers.join(" | "));
});
function conflict(extra) {
  const r = full(); const s = coveredSession(r, denseTimes(r, 0, r.window.tEnd + 500), { manual: { text: "4", reason: "counted aloud" }, ...extra });
  return decide(s);
}
test("C: automatic 5 vs supervisor 4 -> conflict, no value until an explicit basis is chosen", () => {
  assert.ok(computeCoverage, "coverage module missing");
  const d = conflict({}); assert.equal(d.value, null); assert.equal(d.reference.eligible, false);
  assert.ok(/conflict|differ/i.test(d.reference.blockers.join(" ")));
});
test("C: conflict resolved to 'automatic' with a rationale -> 5; to 'supervisor' -> 4", () => {
  assert.ok(computeCoverage, "coverage module missing");
  assert.equal(conflict({ basisChoice: { basis: "automatic", rationale: "images show 5 clear stands" } }).value, 5);
  assert.equal(conflict({ basisChoice: { basis: "supervisor", rationale: "I lost count once but rechecked" } }).value, 4);
});
test("C: supervisor-count route needs no identity answer, and is not labelled image-verified", () => {
  const r = full();
  const d = decide(baseSession(r, { evidence: { available: false }, manual: { text: "5", reason: "counted during the test" }, basisChoice: { basis: "supervisor", rationale: "no images kept" } }));
  assert.equal(d.value, 5); assert.equal(d.reference.eligible, true);
  assert.ok(!/image/i.test(d.basis) || /not image/i.test(d.basis), d.basis);
});
test("C: a supervisor count is not used silently when the automatic basis is unavailable (explicit switch)", () => {
  const r = full();
  const d = decide(baseSession(r, { evidence: { available: false }, manual: { text: "5", reason: "counted" } }));
  assert.equal(d.value, null);
});
test("C invariant: corrections never complete an unfinished test", () => {
  const r = { ...full(), reason: "stopped", status: "stopped", window: { tGo: 0, tEnd: 30000, observedMs: 7000 } };
  const d = decide(baseSession(r, { manual: { text: "20", reason: "x" }, basisChoice: { basis: "supervisor", rationale: "x" } }));
  assert.equal(d.value, null); assert.equal(d.reference.eligible, false);
});

// ---------- D. evidence store lifecycle (injectable encoder: no DOM needed) ----------
function fakeVideo() { return { videoWidth: 64, videoHeight: 64 }; }
test("D: capture(); clear(); then the pending callback settles -> store stays empty", async () => {
  assert.ok(createEvidenceStore, "evidence module not importable in Node");
  let pendingCb = null;
  const s = createEvidenceStore({ encode: (video, w, h, cb) => { pendingCb = cb; } });
  s.capture(fakeVideo(), 1000, null); s.clear(); pendingCb({ size: 795 });
  await new Promise((r) => setTimeout(r, 10));
  assert.equal(s.stats().frames, 0);
});
test("D: out-of-order callbacks are stored in timestamp order", async () => {
  assert.ok(createEvidenceStore, "evidence module not importable in Node");
  const cbs = [];
  const s = createEvidenceStore({ encode: (video, w, h, cb) => cbs.push(cb) });
  s.capture(fakeVideo(), 1000, null); s.capture(fakeVideo(), 1200, null);
```

```
  cbs[1]({ size: 10 }); cbs[0]({ size: 10 });
  assert.deepEqual(s.times(), [1000, 1200]);
});
test("D: finalize() waits for a pending final capture before freezing", async () => {
  assert.ok(createEvidenceStore, "evidence module not importable in Node");
  let cb = null;
  const s = createEvidenceStore({ encode: (video, w, h, c) => { cb = c; } });
  s.capture(fakeVideo(), 29900, null);
  const fin = s.finalize(); setTimeout(() => cb({ size: 10 }), 50);
  await fin;
  assert.deepEqual(s.times(), [29900]);
  s.capture(fakeVideo(), 30500, null);                    // after finalize: ignored
  assert.deepEqual(s.times(), [29900]);
});
test("D: null image-creation callbacks are counted as failures, not frames", async () => {
  assert.ok(createEvidenceStore, "evidence module not importable in Node");
  const s = createEvidenceStore({ encode: (video, w, h, cb) => cb(null) });
  s.capture(fakeVideo(), 1000, null);
  assert.equal(s.stats().frames, 0); assert.equal(s.stats().dropped, 1);
});

test("A: a rejected practice stand is not re-offered while the same stand continues (asked again only after it is broken)", () =>
{
  const M2 = motion([["sit", 1500], ["rise", 800], ["stand", 3000], ["descend", 800], ["sit", 3000]]);
  const c = createChairStand({ width: 720, height: 1280 }); const asked = [];
  for (let t = 0; t < 9000; t += 33) {
    c.push({ t, poses: [skeleton(M2.alphaAt(t) * 0.6)] });
    const pr = c.readiness().practice || {};
    if (pr.state === "detected" && !asked.includes(pr.refId)) { asked.push(pr.refId); c.rejectReference(pr.refId); }
  }
  assert.equal(asked.length, 1, `asked ${asked.length} times during one continuous 3-s stand`);
});

```

tests/regression_overnight.test.mjs

```
// Overnight (post-checkpoint-4) regression tests. Each test was written against a defect reproduced on tag
// `overnight-start` (see docs/OVERNIGHT_ISSUES.json for id, reproduction and root cause). Expected outcomes come from
// the contract (docs/CONTRACTS_CP4.md + its overnight amendment s9), not from the implementation's output.
import test from "node:test";
import assert from "node:assert/strict";
import { decide } from "../app/src/core/decision.js";
import { createEvidenceStore } from "../app/src/ui/evidence.js";

globalThis.URL.revokeObjectURL = globalThis.URL.revokeObjectURL || (() => {});
const enc = (v, w, h, cb) => cb({ size: 1000 });
const VIDEO = { videoWidth: 640, videoHeight: 480 };

// ---------- ON-01: the accepted practice-stand image must survive until review ----------
for (const waitS of [5, 16, 60, 240]) {
  test(`ON-01 practice-stand image survives a ${waitS}-s wait between the practice stand and GO`, () => {
    const ev = createEvidenceStore({ encode: enc });
    const ref = { t0: 5000, t1: 5600 };
    const goT = ref.t1 + waitS * 1000;
    let t = 0, pinned = false;
    for (; t < goT; t += 33) {
      ev.capture(VIDEO, t, null, null);
      if (!pinned && t >= ref.t1 + 1500) { ev.pin("reference", ref.t0 - 150, ref.t1 + 150); pinned = true; }   // app pins when t
he card is shown
    }
    for (; t < goT + 30000; t += 33) ev.capture(VIDEO, t, null, goT - 15000);
    assert.ok(ev.nearest((ref.t0 + ref.t1) / 2, 450), "reference image was trimmed");
    // unpinned old images are still trimmed (memory stays bounded)
    if (goT - 15000 > 2100) assert.equal(ev.nearest(2000, 100), null, "an unpinned pre-test image outside the keep window should
have been trimmed");
  });
}
test("ON-01 re-pinning replaces the previous pin (a rejected hold's images are released again)", () => {
  const ev = createEvidenceStore({ encode: enc });
  let t = 0;
  for (; t < 8000; t += 33) ev.capture(VIDEO, t, null, null);
  ev.pin("reference", 1000, 1600);
  for (; t < 12000; t += 33) ev.capture(VIDEO, t, null, null);
  ev.pin("reference", 9000, 9600);                    // new hold shown: previous hold no longer protected
  for (; t < 40000; t += 33) ev.capture(VIDEO, t, null, null);
  assert.equal(ev.nearest(1300, 400), null);
  assert.ok(ev.nearest(9300, 400));
});
test("ON-01 clear() removes pins", () => {
  const ev = createEvidenceStore({ encode: enc });
  for (let t = 0; t < 3000; t += 33) ev.capture(VIDEO, t, null, null);
  ev.pin("reference", 1000, 1600); ev.clear();
  for (let t = 3000; t < 40000; t += 33) ev.capture(VIDEO, t, null, null);
  assert.equal(ev.nearest(1300, 400), null);
});

// ---------- shared decision fixture: 5 counted full stands, dense images, everything checked ----------
const tGo = 6000, tEnd = 36000;
const imageTimes = []; for (let t = 1000; t <= tEnd + 200; t += 166) imageTimes.push(t);
const imgNear = (t) => imageTimes.find((x) => x >= t);
function fixture({ events: evs } = {}) {
  const events = evs || [0, 1, 2, 3, 4].map((i) => ({ n: i + 1, cls: "full", counted: true, tRise: 7000 + i * 5000, tFull: 7800 +
i * 5000, tPeak: 7900 + i * 5000, tSeat: 8800 + i * 5000, flags: [] }));
  const coverage = { imageCount: imageTimes.length, imageTimes, reference: { ok: true }, events: Object.fromEntries(events.map
((e) => [e.n, { ok: true }])),
    flags: {}, endpoint: { ok: true, required: false }, scan: { ok: true, gaps: [] } };
  const result = { reason: "expired", status: "measured", window: { tGo, tEnd, observedMs: 30000 }, events, reviewFlags: [], flag
Windows: {},
    reference: { id: 1, t0: 2000, t1: 2600, accepted: { basis: "image" }, usedAsReference: true, postureWarning: false } };
  return { result, coverage, ids: events.map((e) => e.n) };
}
function session(fx, { added = [], resolutions = {}, manual = null, basisChoice = null } = {}) {
  return { result: fx.result, source: "camera", observer: { arms: "no", setup: "yes" }, participant: { ageText: "70", column: "wo
men" },
    evidence: { available: true, coverage: fx.coverage, identityConfirmed: true, identityConfirmedFor: fx.ids, sessionScan: "no_m
issed", added, eventResolutions: resolutions, flagChecks: {} },
    manual, basisChoice };
}

test("ON-02 control: clean fixture is eligible with value 5", () => {
  const d = decide(session(fixture()));
  assert.equal(d.value, 5); assert.equal(d.reference.eligible, true);
});
test("ON-02 an added stand inside an already-detected rise is not counted and blocks interpretation", () => {
  const d = decide(session(fixture(), { added: [{ id: "added-1", t: imgNear(7900) }] }));
  assert.notEqual(d.value, 6);
  assert.equal(d.reference.eligible, false);
  assert.ok(d.reference.blockers.join(" ").toLowerCase().includes("rise 1"), d.reference.blockers.join(" | "));
});
test("ON-02 the same image added twice (double click) is not counted twice", () => {
```

```
  const t = imgNear(31500);   // between rise 5 (ends 28800) and 30 s
  const d = decide(session(fixture(), { added: [{ id: "added-1", t }, { id: "added-2", t }] }));
  assert.ok(d.value === null || d.value <= 6, `value ${d.value}`);
  assert.equal(d.reference.eligible, false, "two added stands at the same moment must block, not count");
});
test("ON-02 two added stands closer than one sit-stand cycle block interpretation", () => {
  const d = decide(session(fixture(), { added: [{ id: "added-1", t: imgNear(31000) }, { id: "added-2", t: imgNear(31400) }] }));
  assert.equal(d.reference.eligible, false);
});
test("ON-02 a single well-separated added stand still counts (correction path remains usable)", () => {
  const d = decide(session(fixture(), { added: [{ id: "added-1", t: imgNear(31500) }] }));
  assert.equal(d.value, 6); assert.equal(d.reference.eligible, true);
});
test("ON-02 added stands before GO or after 30 s are not counted and block interpretation", () => {
  for (const t of [imgNear(1000), imgNear(36100)]) {
    const d = decide(session(fixture(), { added: [{ id: "added-1", t }] }));
    assert.notEqual(d.value, 6, `t=${t}`);
    assert.equal(d.reference.eligible, false, `t=${t}`);
  }
});
test("ON-02 an added stand inside a rise rejected as not full is still refused (correct that rise instead)", () => {
  const d = decide(session(fixture(), { added: [{ id: "added-1", t: imgNear(12900) }], resolutions: { 2: "not_full" } }));
  assert.equal(d.reference.eligible, false);
});

// ---------- ON-03: a basis choice is bound to the counts the observer saw when choosing ----------
test("ON-03 control: choosing the camera count while counts agree reports it", () => {
  const fx = fixture();
  const d = decide(session(fx, { manual: { text: "5", reason: "counted" }, basisChoice: { basis: "automatic", rationale: "agree",
at: { automaticMin: 5, automaticMax: 5, supervisor: 5 } } }));
  assert.equal(d.value, 5); assert.equal(d.reference.eligible, true);
});
test("ON-03 a later correction that changes the camera count invalidates an earlier basis choice", () => {
  const fx = fixture();
  const d = decide(session(fx, { resolutions: { 5: "not_full" }, manual: { text: "5", reason: "counted" }, basisChoice: { basis:
"automatic", rationale: "they agree", at: { automaticMin: 5, automaticMax: 5, supervisor: 5 } } }));
  assert.equal(d.value, null, "stale choice must not produce a value");
  assert.equal(d.reference.eligible, false);
  assert.equal(d.conflict, true);
  assert.ok(d.reference.blockers.join(" ").toLowerCase().includes("choose"), d.reference.blockers.join(" | "));
});
test("ON-03 a stale supervisor choice is also invalidated", () => {
  const fx = fixture();
  const d = decide(session(fx, { manual: { text: "6", reason: "counted" }, basisChoice: { basis: "supervisor", rationale: "missed
one", at: { automaticMin: 5, automaticMax: 5, supervisor: 7 } } }));
  assert.equal(d.value, null);
  assert.equal(d.reference.eligible, false);
});

// ON-38: with a slow encoder (all pending slots busy), a priority capture near the 30-s mark is still taken
test("ON-38 priority capture near the 30-s mark is taken even when normal captures are skipped as busy", () => {
  const cbs = [];
  const ev = createEvidenceStore({ encode: (v, w, h, cb) => cbs.push(cb), maxPending: 2, minIntervalMs: 0 });
  ev.capture(VIDEO, 100, null); ev.capture(VIDEO, 200, null);            // 2 pending: store is busy
  ev.capture(VIDEO, 300, null);                                          // normal: skipped
  assert.equal(ev.stats().skippedBusy, 1);
  ev.capture(VIDEO, 30000, null, null, { priority: true });              // near the endpoint: taken
  assert.equal(ev.stats().pending, 3);
  cbs.forEach((cb) => cb({ size: 1 }));
  assert.ok(ev.nearest(30000, 1));
});

```

tests/regression_overnight_core.test.mjs

```
// Overnight core/decision regressions (core 0.8.0). Each case reproduces a defect found by the overnight adversarial
// review on tag `overnight-start` (docs/OVERNIGHT_ISSUES.json). Truth comes from the motion generator (eval/synth.mjs:
// alpha 1 = full extension; referencePositionCount = CDC position rule), never from core output.
// The safety property asserted throughout: the core may ask for review, but it must never SILENTLY report a wrong count
// (status "measured" with count != truth, or a range that excludes the truth).
import test from "node:test";
import assert from "node:assert/strict";
import { createChairStand } from "../app/src/core/chair_stand.js";
import { decide } from "../app/src/core/decision.js";
import { skeleton, motion, PRACTICE, PRACTICE_GO, referencePositionCount } from "../eval/synth.mjs";

function run(script, { fps = 30, poseAt = null } = {}) {
  const M = motion(script);
  const c = createChairStand({ width: 720, height: 1280 });
  let started = false; const answered = new Set(); let lastT = 0;
  for (let i = 0; ; i++) {
    const t = Math.round(i * 1000 / fps); if (t > M.totalMs) break;
    if (c.phase === "calibrating" && t >= PRACTICE_GO && !started) {
      const pr = c.readiness().practice || {};
      if (pr.state === "detected" && !answered.has(pr.refId)) { answered.add(pr.refId); c.acceptReference({ refId: pr.refId, basis: "image" }); }
      if (c.readiness().ready) { c.start(t); started = true; }
    }
    const a = M.alphaAt(t);
    const s = c.push({ t, poses: poseAt ? poseAt(t, a, c.phase) : [skeleton(a)] });
    lastT = t;
    if (s.result) return s.result;
  }
  assert.ok(started, "test never started");
  return c.finish("stream_end", lastT);
}
const cyc = (n, { rise = 800, stand = 400, descend = 800, sit = 400 } = {}) => Array.from({ length: n }, () => [["rise", rise], ["stand", stand], ["descend", descend], ["sit", sit]]).flat();
function notSilentlyWrong(r, truth, label) {
  const ctx = `${label}: status ${r.status} count ${r.count} range ${JSON.stringify(r.countRange)} cls ${r.events.map((e) => e.cls[0]).join("")} truth ${truth}`;
  if (r.status === "withheld") return;
  if (r.status === "measured") assert.equal(r.count, truth, ctx);
  else assert.ok(r.countRange.min <= truth && truth <= r.countRange.max, ctx);
}

// ON-05: shoulders poorly visible during the test -> angles unmeasured -> must not become silent "partial"
for (const vis of [0.45, 0.3]) test(`ON-05 shoulders visibility ${vis} during the test: 5 genuine stands are never silently lost`, () => {
  const poseAt = (t, a, phase) => { const lm = skeleton(a); if (phase === "running") for (const i of [11, 12]) lm[i] = [...lm[i].slice(0, 3), vis, lm[i][4]]; return [lm]; };
  const r = run([...PRACTICE, ...cyc(5), ["sit", 30000]], { poseAt });
  notSilentlyWrong(r, 5, "shoulders hidden");
  assert.ok(r.events.filter((e) => e.cls === "partial").length === 0, "a full-height rise with unmeasured angles was classed partial");
});
test("ON-05 control: angles unmeasured but a clearly low rise (h shortfall) is still partial", () => {
  const poseAt = (t, a, phase) => { const lm = skeleton(phase === "running" ? a * 0.5 : a); if (phase === "running") for (const i of [11, 12]) lm[i] = [...lm[i].slice(0, 3), 0.3, lm[i][4]]; return [lm]; };
  const r = run([...PRACTICE, ...cyc(5), ["sit", 30000]], { poseAt });
  assert.equal(r.count, 0);
  assert.ok(r.events.every((e) => e.cls === "partial"), r.events.map((e) => e.cls).join());
});

// ON-06: the participant sits back down higher than at calibration ("perched") -> stands must not merge silently
for (const sitAlpha of [0.12, 0.15, 0.2]) test(`ON-06 perched sits at alpha ${sitAlpha}: 8 stands never silently merged`, () => {
  const script = [...PRACTICE, ...cyc(8, { sit: 500 }), ["sit", 30000]];
  const r = run(script, { poseAt: (t, a, phase) => [skeleton(phase === "running" ? sitAlpha + (1 - sitAlpha) * a : a)] });
  notSilentlyWrong(r, 8, `perched ${sitAlpha}`);
  assert.equal(r.events.filter((e) => e.counted).length + r.events.filter((e) => e.cls === "uncertain").length >= 8, true);
});
for (const floor of [0.13, 0.15, 0.2]) test(`ON-06 one sit (of 8) only down to alpha ${floor}: truth inside the range, never silently 7`, () => {
  const script = [...PRACTICE, ...cyc(8, { sit: 500 }), ["sit", 30000]];
  const w0 = PRACTICE_GO + 3 * 2500 + 1200, w1 = PRACTICE_GO + 4 * 2500;          // cycle 3's descend + sit (cycles are 2500 ms)
  const r = run(script, { poseAt: (t, a, phase) => [skeleton(phase === "running" && t >= w0 && t < w1 + 800 ? Math.max(a, floor) : a)] });
  notSilentlyWrong(r, 8, `one perched ${floor}`);
});
test("ON-06 control: normal sits are unaffected (8 counted, measured)", () => {
  const r = run([...PRACTICE, ...cyc(8), ["sit", 30000]]);
  assert.equal(r.status, "measured"); assert.equal(r.count, 8);
});

// ON-07: 30-s endpoint decided on a lagging smoothed signal at lower frame rates
test("ON-07 endpoint sweep (fps 30/20/15/12/10/9; 3 rise speeds): no silent endpoint error", () => {
  const bad = [];
  for (const fps of [30, 20, 15, 12, 10, 9]) for (const riseMs of [800, 1000, 1400]) for (let frac = 0.05; frac < 1; frac += 0.05) {
```

```
    const tEnd = PRACTICE_GO + 30000, lastRiseStart = Math.round(tEnd - frac * riseMs);
    const script = [...PRACTICE, ...cyc(11), ["sit", lastRiseStart - PRACTICE_GO - 11 * 2400], ["rise", riseMs], ["stand", 1500], ["descend", 800], ["sit", 3000]];
    const ref = referencePositionCount(script, PRACTICE_GO);
    const r = run(script, { fps });
    if (r.status === "measured" && r.count !== ref.count) bad.push(`fps ${fps} rise ${riseMs} frac ${frac.toFixed(2)}: measured ${r.count}, CDC ${ref.count} (${ref.case})`);
    if (r.status === "needs_review" && !(r.countRange.min <= ref.count && ref.count <= r.countRange.max)) bad.push(`fps ${fps} rise ${riseMs} frac ${frac.toFixed(2)}: range ${JSON.stringify(r.countRange)} excludes CDC ${ref.count}`);
  }
  assert.deepEqual(bad, []);
});

// ON-08: brief stands at low accepted frame rates must not become silent partials
test("ON-08 fast stands (no hold) at 8.5–15 fps: never silently lost", () => {
  const bad = [];
  for (const fps of [15, 12, 10, 9, 8.5]) for (const [rise, stand, descend, sit] of [[600, 100, 600, 300], [500, 0, 500, 300], [450, 0, 450, 250]]) {
    const r = run([...PRACTICE, ...cyc(8, { rise, stand, descend, sit }), ["sit", 30000]], { fps });
    if (r.status === "withheld") continue;
    const ok = r.status === "measured" ? r.count === 8 : r.countRange.min <= 8 && 8 <= r.countRange.max;
    if (!ok) bad.push(`fps ${fps} ${rise}/${stand}/${descend}/${sit}: ${r.status} ${r.count} ${JSON.stringify(r.countRange)} ${r.events.map((e) => e.cls[0]).join("")}`);
  }
  assert.deepEqual(bad, []);
});

// ON-09: the accepted reference must be the posture shown on the card (values frozen at detection)
test("ON-09 participant sags while the observer reads the card: the accepted reference is the shown (upright) posture", () => {
  const c = createChairStand({ width: 720, height: 1280 });
  const ease = (u) => 0.5 - 0.5 * Math.cos(Math.PI * Math.max(0, Math.min(1, u)));
  const alpha = (t) => t < 1500 ? 0 : t < 2300 ? ease((t - 1500) / 800) : t < 3300 ? 1 : t < 6300 ? 1 - 0.15 * (t - 3300) / 3000 : t < 6600 ? 0.85 : t < 7400 ? 0.85 * (1 - ease((t - 6600) / 800)) : 0;
  let shown = null;
  for (let t = 0; t < 9000; t += 33) {
    const pr = c.readiness().practice;
    if (pr.state === "detected" && !shown) { shown = { refId: pr.refId, knee: pr.knee, hip: pr.hip, h: pr.h }; c.freezeReference(pr.refId); }   // the app freezes when it shows the card
    if (shown && t >= 6400 && pr.state === "detected") c.acceptReference({ refId: shown.refId, basis: "image" });
    c.push({ t, poses: [skeleton(alpha(t))] });
  }
  const pr = c.readiness().practice;
  assert.ok(shown, "no card");
  if (pr.state === "accepted") { assert.equal(pr.knee, shown.knee); assert.equal(pr.hip, shown.hip); assert.equal(pr.h, shown.h); }
  else assert.notEqual(pr.state, "accepted");
});

// ON-13: reselecting the participant after a differently sized person was locked first
test("ON-13 helper (1.6x size) locked first, participant then selected: camera check can become ready", () => {
  const scale = (lm, sc, cx, cy) => lm.map(([x, y, z, v, p]) => [(x - cx) * sc + cx, (y - cy) * sc + cy, z, v, p]);
  const M = motion([...PRACTICE, ["sit", 20000]]);
  const c = createChairStand({ width: 720, height: 1280 });
  const helper = () => scale(skeleton(1, { x0: 150 }), 1.6, 150 / 720, 1150 / 1280);
  for (let t = 0; t < 2000; t += 33) c.push({ t, poses: [helper()] });
  c.selectParticipant(400 / 720, 900 / 1280);
  for (let t = 2000; t < 2000 + M.totalMs; t += 33) {
    c.push({ t, poses: [skeleton(M.alphaAt(t - 2000)), helper()] });
    const pr = c.readiness().practice; if (pr.state === "detected") c.acceptReference({ refId: pr.refId, basis: "image" });
  }
  assert.equal(c.readiness().ready, true, JSON.stringify(c.readiness().reasons));
});

// ---------- decision ----------
const tGo = 6000, tEnd = 36000;
const imageTimes = []; for (let t = 1000; t <= tEnd + 200; t += 166) imageTimes.push(t);
function covered(events, extra = {}) {
  const coverage = { imageCount: imageTimes.length, imageTimes, reference: { ok: true }, events: Object.fromEntries(events.map((e) => [e.n, { ok: true }])), flags: {}, endpoint: { ok: true, required: false }, scan: { ok: true, gaps: [] } };
  const result = { reason: "expired", status: events.some((e) => e.cls === "uncertain") ? "needs_review" : "measured", window: { tGo, tEnd, observedMs: 30000 }, events, reviewFlags: events.some((e) => e.cls === "uncertain") ? ["uncertain_rises"] : [], flagWindows: {},
    reference: { id: 1, t0: 2000, t1: 2600, accepted: { basis: "image" }, usedAsReference: true, postureWarning: false } };
  return { result, source: "camera", observer: { arms: "no", setup: "yes" }, participant: { ageText: "70", column: "women" },
    evidence: { available: true, coverage, identityConfirmed: true, identityConfirmedFor: events.filter((e) => e.counted || e.cls === "uncertain").map((e) => e.n), sessionScan: "no_missed", added: [], eventResolutions: {}, flagChecks: {} }, ...extra };
}
const ev = (n, cls) => ({ n, cls, counted: cls === "full", tRise: 7000 + n * 2500, tFull: cls === "full" ? 7800 + n * 2500 : null, tPeak: 7900 + n * 2500, tSeat: 8800 + n * 2500, flags: [] });

test("ON-11 open range + supervisor count + explicit camera-count choice: never 'no value and no reason'", () => {
  const events = [ev(1, "full"), ev(2, "full"), ev(3, "full"), ev(4, "full"), ev(5, "full")];   // rise 5 marked "can't tell" -> range 4–5
  for (const age of ["70", "61"]) {
    const s = covered(events, { manual: { text: "5", reason: "counted" }, basisChoice: { basis: "automatic", rationale: "prefer camera" }, participant: { ageText: age, column: "women" } });
    s.evidence.eventResolutions = { 5: "cant_tell" };
```

```
    const d = decide(s);
    assert.ok(d.reference.eligible || d.reference.blockers.length > 0, `age ${age}: not eligible and no blocker`);
    if (!d.reference.eligible) assert.ok(/unresolved/.test(d.reference.blockers.join(" ")), d.reference.blockers.join(" | "));
  }
});
test("ON-12 a genuine automatic 0 (all rises partial) can become interpretable after the whole-test scan", () => {
  const s = covered([ev(1, "partial"), ev(2, "partial")]);
  s.evidence.identityConfirmed = null;
  const d = decide(s);
  assert.equal(d.value, 0); assert.equal(d.reference.eligible, true);
  s.evidence.sessionScan = null;
  assert.equal(decide(s).reference.eligible, false, "the whole-test scan must still be required");
});
test("ON-12 control: identity is still required when any rise is counted", () => {
  const s = covered([ev(1, "full"), ev(2, "partial")]); s.evidence.identityConfirmed = null; s.evidence.identityConfirmedFor =
[];
  assert.equal(decide(s).reference.eligible, false);
});

// ---------- second fresh adversarial pass on the repaired tree (core 0.8.1) ----------
// ON-19: a participant sitting down at the 30-s mark must not get a made-up "rise in progress" rise
test("ON-19 sitting down across the 30-s mark (slow descents, 30/20/15 fps): no extra event after the 12 credited stands", () =>
{
  const tEnd = PRACTICE_GO + 30000; const bad = [];
  for (const fps of [30, 20, 15]) for (const desc of [1500, 2500, 3500]) for (let off = 0; off <= desc; off += 100) {
    const descStart = tEnd - off, riseStart = descStart - 500 - 800, sitLen = riseStart - PRACTICE_GO - 11 * 2400;
    if (sitLen < 200) continue;                                  // (a negative sit makes the synthetic motion discontinuous)
    const r = run([...PRACTICE, ...cyc(11), ["sit", sitLen], ["rise", 800], ["stand", 500], ["descend", desc], ["sit", 3000]], {
fps });
    if (r.events.length !== 12 || r.count !== 12) bad.push(`fps ${fps} desc ${desc} off ${off}: ${r.events.length} events, count
${r.count}, ${r.events.slice(12).map((e) => e.cls + "[" + e.flags + "]")}`);
  }
  assert.deepEqual(bad, []);
});
// ON-22: a rise after a perched (seat-not-reached) sit must never be counted automatically, also at 30 s
test("ON-22 perched dip then rising at 30 s: never auto-counted (12, or review including 12)", () => {
  const tEnd = PRACTICE_GO + 30000;
  for (const dip of [0.2, 0.25]) for (const off of [150, 250, 350, 450]) {
    const riseBack = tEnd - off, dipStart = riseBack - 300 - 900, stand12 = dipStart - 500 - 800, sitLen = stand12 - PRACTICE_GO
- 11 * 2400;
    const script = [...PRACTICE, ...cyc(11), ["sit", sitLen], ["rise", 800], ["stand", 500], ["descend", 900], ["sit", 300], ["ri
se", 800], ["stand", 2000]];
    const r = run(script, { poseAt: (t, a) => [skeleton(t >= dipStart && t < riseBack + 800 ? dip + (1 - dip) * a : a)] });
    const ctx = `dip ${dip} off ${off}: ${r.status} ${r.count} ${JSON.stringify(r.countRange)} ${r.events.slice(11).map((e) => e.
cls + (e.counted ? "*" : "") + "[" + e.flags + "]")}`;
    assert.ok(r.events.filter((e) => e.counted).length <= 12, ctx);
    if (r.status === "measured") assert.equal(r.count, 12, ctx); else assert.ok(r.countRange.min <= 12 && 12 <= r.countRange.max,
ctx);
  }
});
// ON-20: angles seen only low in the rise (hidden at the top) are not evidence of a partial stand
for (const thr of [0.9, 0.8, 0.7]) test(`ON-20 joint angles hidden above alpha ${thr}: 5 genuine full stands never silently parti
al`, () => {
  const poseAt = (t, a, phase) => { const lm = skeleton(a); if (phase === "running" && a > thr) for (const i of [11, 12]) lm[i] =
[...lm[i].slice(0, 3), 0.4, lm[i][4]]; return [lm]; };
  const r = run([...PRACTICE, ...cyc(5), ["sit", 30000]], { poseAt });
  notSilentlyWrong(r, 5, `angles hidden > ${thr}`);
});
// ON-21: a participant who straightens slowly at the top of the practice stand: the reference is the most extended window
test("ON-21 slow straightening during the practice stand: reference = full extension, incomplete test rises not counted", () => {
  const ease = (u) => 0.5 - 0.5 * Math.cos(Math.PI * Math.max(0, Math.min(1, u)));
  for (const [extFrom, extMs] of [[0.85, 1500], [0.85, 3500], [0.88, 3500]]) {
    const t1 = 1500, t2 = t1 + 800, t3 = t2 + extMs, t4 = t3 + 1000, t5 = t4 + 800, go = t5 + 1800;
    const alpha = (t) => t < t1 ? 0 : t < t2 ? extFrom * ease((t - t1) / 800) : t < t3 ? extFrom + (1 - extFrom) * (t - t2) / ext
Ms : t < t4 ? 1 : t < t5 ? 1 - ease((t - t4) / 800) : 0;
    const c = createChairStand({ width: 720, height: 1280 }); let shown = null;
    for (let t = 0; t < go; t += 33) {
      c.push({ t, poses: [skeleton(alpha(t))] });
      const pr = c.readiness().practice;
      if (pr.state === "detected" && !shown) { shown = pr; c.freezeReference(pr.refId); }             // the app shows the card
    }
    assert.ok(shown, "card never offered");
    assert.ok(shown.knee >= 178, `extFrom ${extFrom} over ${extMs} ms: reference knee ${shown.knee} (not the full extension)`);
  }
});

// ---------- third fresh adversarial pass + mutation re-run (core 0.8.2) ----------
// ON-26: stale smoothed angles at the top at 15–20 fps (shoulders hidden near upright) must not make genuine stands partial
for (const fps of [30, 20, 15]) for (const thr of [0.7, 0.8, 0.9]) test(`ON-26 ${fps} fps, angles hidden above alpha ${thr}: 6 ge
nuine stands never silently partial`, () => {
  const poseAt = (t, a, phase) => { const lm = skeleton(a); if (phase === "running" && a > thr) for (const i of [11, 12]) lm[i] =
[...lm[i].slice(0, 3), 0.4, lm[i][4]]; return [lm]; };
  const r = run([...PRACTICE, ...cyc(6), ["sit", 30000]], { fps, poseAt });
  notSilentlyWrong(r, 6, `fps ${fps} thr ${thr}`);
  assert.equal(r.events.filter((e) => e.cls === "partial").length, 0, r.events.map((e) => e.cls).join());
});
// ON-27: shoulders hidden near upright ALSO during the practice stand: the reference must not take its angles from 1–2 frames
```

```
test("ON-27 shoulders hidden near upright in practice and test: incomplete rises are never counted as full", () => {
  const poseAt = (t, a) => { const lm = skeleton(a); if (a > 0.85) for (const i of [11, 12]) lm[i] = [...lm[i].slice(0, 3), 0.4, lm[i][4]]; return [lm]; };
  for (const fps of [30, 15]) {
    const script = [...PRACTICE, ...cyc(6), ...cyc(6).map(([k, d]) => [k, d]), ["sit", 30000]];
    const M = motion(script); let n = 0;
    const r = run(script, { fps, poseAt: (t, a, phase) => { if (phase === "running") { const cycle = Math.floor((t - PRACTICE_GO) / 2400); n = cycle; if (cycle >= 6) return poseAt(t, a * 0.8); } return poseAt(t, a); } });
    const counted = r.events.filter((e) => e.counted).length;
    assert.ok(counted <= 6, `fps ${fps}: ${counted} counted (truth: 6 full + 6 incomplete) ${r.events.map((e) => e.cls[0]).join("")}`);
    void M; void n;
  }
});
// ON-28: a 2-3 frame landmark glitch mid-hold must not end the practice stand (and void an acceptance moments later)
test("ON-28 a brief hip glitch during the practice hold does not end the stand", () => {
  const M = motion([["sit", 1500], ["rise", 800], ["stand", 2500], ["descend", 800], ["sit", 3000]]);
  const c = createChairStand({ width: 720, height: 1280 }); const ids = new Set(); let offeredWhileStanding = false;
  for (let t = 0; t < 8600; t += 33) {
    const lm = skeleton(M.alphaAt(t));
    if (t >= 3900 && t < 4000) for (const i of [23, 24]) lm[i] = [lm[i][0], lm[i][1] + 150 / 1280, lm[i][2], lm[i][3], lm[i][4]];
    c.push({ t, poses: [lm] });
    const pr = c.readiness().practice; if (pr.refId) ids.add(pr.refId);
    if (pr.state === "detected" && M.alphaAt(t) > 0.9) offeredWhileStanding = true;
  }
  assert.equal(offeredWhileStanding, false, "card offered while still standing");
  assert.equal(ids.size, 1, `ids ${[...ids]}`);
});
// ON-29 + mutation gaps: rises after perched sits never counted; half-sits flagged, never silently merged
for (const sitAlpha of [0.12, 0.2, 0.3]) test(`ON-29 perched ${sitAlpha}: no counted event carries seat_return_not_confirmed`, () => {
  const r = run([...PRACTICE, ...cyc(8, { sit: 500 }), ["sit", 30000]], { poseAt: (t, a, phase) => [skeleton(phase === "running" ? sitAlpha + (1 - sitAlpha) * a : a)] });
  assert.equal(r.events.filter((e) => e.counted && e.flags.includes("seat_return_not_confirmed")).length, 0);
  notSilentlyWrong(r, 8, `perched ${sitAlpha}`);
});
for (const sitAlpha of [0.35, 0.45, 0.5]) test(`ON-29 half-sits staying above halfway (alpha ${sitAlpha}) are flagged, never silently 'measured'`, () => {
  const r = run([...PRACTICE, ...cyc(8, { sit: 500 }), ["sit", 30000]], { poseAt: (t, a, phase) => [skeleton(phase === "running" ? sitAlpha + (1 - sitAlpha) * a : a)] });
  assert.notEqual(r.status, "measured", `${r.count} ${r.reviewFlags}`);
  assert.ok(r.reviewFlags.includes("stand_dipped_without_sitting") || r.reviewFlags.includes("uncertain_rises"), r.reviewFlags.join());
});
test("mutation gap: the KNEE posture warning (knee < 158 deg) is raised on its own", async () => {
  const { PARAMS } = await import("../app/src/core/params.js");
  assert.equal(PARAMS.referenceWarnKneeDeg, 158);
  // bend only the knee of the practice stand by moving the ankle forward (hip angle stays ~straight)
  const bentKnee = (a) => { const lm = skeleton(a); if (a > 0.9) for (const i of [27, 28, 29, 30, 31, 32]) lm[i] = [lm[i][0] + 95 / 720, lm[i][1] - 10 / 1280, lm[i][2], lm[i][3], lm[i][4]]; return lm; };
  const M = motion([...PRACTICE, ["sit", 3000]]);
  const c = createChairStand({ width: 720, height: 1280 }); let pr = null;
  for (let t = 0; t < 8000; t += 33) { c.push({ t, poses: [bentKnee(M.alphaAt(t))] }); const p = c.readiness().practice; if (p.state === "detected") pr = p; }
  assert.ok(pr, "no practice stand offered");
  if (pr.knee < 158 && pr.hip >= 165) assert.equal(pr.postureWarning, true, JSON.stringify(pr));
  else assert.fail(`fixture did not isolate the knee: knee ${pr.knee} hip ${pr.hip}`);
});

// ---------- fourth fresh adversarial pass (core 0.8.3) ----------
// ON-31: a stand completed before 30 s but not auto-confirmed, with the participant descending below halfway at 30 s, was
// labelled "partial" and dropped silently (also in core 0.7). Scenarios from the 4th reviewer's realistic generator.
import { session as rsession, runCore as rrun, truthCount as rtruth, NEW as RNEW } from "../eval/synth_realistic.mjs";
import { rng as srng } from "../eval/synth.mjs";
// CDC truth: tops reached in (GO, GO+30 s], plus one if over halfway and still rising at 30 s; near halfway either is accepted
function checkRealistic(r, sess, tStart, label) {
  const t = rtruth(sess, tStart);
  const ok = (v) => (t.kase === "near" ? v === t.n || v === t.n + 1 : v === t.nWithExpiry);
  if (r.status === "withheld") return;
  if (r.status === "measured") assert.ok(ok(r.count), `${label}: measured ${r.count}, CDC truth ${t.nWithExpiry} (${t.kase})`);
  else assert.ok([t.n, t.n + 1].some((v) => ok(v) && r.countRange.min <= v && v <= r.countRange.max), `${label}: range ${JSON.stringify(r.countRange)}, CDC truth ${t.nWithExpiry} (${t.kase})`);
}
test("ON-31 a stand completed just before 30 s and descending at 30 s is never silently dropped (reviewer's two repro cases)", () => {
  const cases = [
    [{ riseMs: [900, 1300], holdMs: [100, 400], descMs: [900, 1300], sitMs: [300, 600], leanJit: 0.06, lead: 1460 }, 30, 6],
    [{ riseMs: [1200, 1800], holdMs: [0, 150], descMs: [1000, 1500], sitMs: [300, 700], pauseP: 0.1, rockP: 0.2, lateP: 0.2, topAlpha: [0.985, 1], leanJit: 0.02, practice: { late: true, sway: 0.01 }, lead: 2240 }, 20, 4],
  ];
  for (const [sty, fps, seed] of cases) {
    const sess = rsession(sty, srng(seed * 1009 + fps));
    const out = rrun(RNEW, sess, { fps, noisePx: 3, shoulderFlick: 0.05, legFlick: 0.02, tJit: fps <= 15 ? 12 : 6, drop: 0.02, seed, leanBase: (seed % 3) * 0.06 });
    const r = out.r;
    checkRealistic(r, sess, out.tStart, `fps ${fps} seed ${seed}`);
    assert.ok(!r.events.some((e) => e.cls === "partial" && e.flags.includes("expiry_under_halfway") && e.tPeak < r.window.tEnd -
```

```
300 && e.peak && e.peak.h > 1.0), "a completed top was labelled under-halfway partial");
 294   }
 295 });
 296 test("burden guard (realistic synthetic, 30 fps, 3 px jitter): no silent errors and <= 5% of genuine stands sent to review", () =
> {
 297   let genuine = 0, unc = 0;
 298   for (const [sty, seed] of [[{ riseMs: [900, 1300], holdMs: [100, 400], descMs: [900, 1300], sitMs: [300, 600], leanJit: 0.04 },
1], [{ riseMs: [1200, 2200], holdMs: [100, 500], descMs: [1000, 1600], sitMs: [300, 700], pauseP: 0.1, rockP: 0.2, lateP: 0.2, leanJi
t: 0.03, practice: { late: true, sway: 0.01 } }, 2]])
 299     for (let k = 0; k < 6; k++) {
 300       const sess = rsession(sty, srng(seed * 7919 + k));
 301       const out = rrun(RNEW, sess, { fps: 30, noisePx: 3, shoulderFlick: 0.02, legFlick: 0.01, tJit: 6, drop: 0.01, seed: seed *
31 + k, leanBase: 0.03 });
 302       if (!out.r) continue;
 303       checkRealistic(out.r, sess, out.tStart, `style ${seed} k ${k}`);
 304       genuine += rtruth(sess, out.tStart).n; unc += out.r.events.filter((e) => e.cls === "uncertain").length;
 305     }
 306   assert.ok(genuine > 50, `only ${genuine} genuine stands`);
 307   assert.ok(unc / genuine <= 0.05, `${unc}/${genuine} sent to review`);
 308 });
 309
 310 // ---------- fifth fresh adversarial pass (core 0.8.4) ----------
 311 // ON-35: a completed but unconfirmed top, then descending at/below about halfway at 30 s: the review must ask "full stand?",
 312 // never only the 30-s halfway question (an honest "not over halfway" answer would lose a completed stand)
 313 test("ON-35 descending after an unconfirmed top near/below halfway at 30 s -> 'full stand?' question, truth kept in range", () =>
{
 314   const bad = [];
 315   for (const fps of [30, 20, 15]) for (let off = 400; off <= 1150; off += 50) {
 316     const tEnd = PRACTICE_GO + 30000, top = tEnd - off - 250, firstRise = top - 1200;
 317     const n = Math.floor((firstRise - PRACTICE_GO - 100) / 3500), lead = firstRise - PRACTICE_GO - n * 3500;
 318     const script = [...PRACTICE, ["sit", lead], ...Array.from({ length: n }, () => [["rise", 1200], ["stand", 300], ["descend", 1
200], ["sit", 800]]).flat(), ["rise", 1200], ["stand", 250], ["descend", 1200], ["sit", 5000]];
 319     const r = run(script, { fps, poseAt: (t, a, phase) => { const lm = skeleton(a); if (phase === "running" && t > firstRise - 10
0 && a > 0.9) for (const k of [11, 12]) lm[k] = [lm[k][0], lm[k][1], 0, 0.3, 0.99]; return [lm]; } });
 320     const truth = n + 1, last = r.events[r.events.length - 1];
 321     const asksHalfwayOnly = last && !last.counted && last.cls !== "partial" && last.flags.some((f) => /^expiry_/.test(f)) && (r.e
xpiry.hRaw ?? r.expiry.h) < r.expiry.halfway;
 322     if (asksHalfwayOnly) bad.push(`fps ${fps} off ${off}: halfway question although below halfway (${last.flags})`);
 323     if (r.status === "measured" ? r.count !== truth : !(r.countRange && r.countRange.min <= truth && truth <= r.countRange.max) &
& r.status !== "withheld") bad.push(`fps ${fps} off ${off}: ${r.status} ${JSON.stringify(r.countRange)} truth ${truth}`);
 324   }
 325   assert.deepEqual(bad, []);
 326 });
 327
 328 // ---------- sixth fresh adversarial pass (core 0.8.5) ----------
 329 // ON-39 (introduced by ON-35 in 0.8.4): a failed rise that sinks back (not to the seat) and a SECOND push rising just over
 330 // halfway at 30 s must get the 30-s question (review), not "was the earlier top a full stand?"
 331 test("ON-39 second push rising over halfway at 30 s inside one open attempt -> never silently decided", () => {
 332   const ease = (u) => 0.5 - 0.5 * Math.cos(Math.PI * Math.max(0, Math.min(1, u)));
 333   const prof = (keys) => (t) => { if (t <= keys[0][0]) return keys[0][1]; for (let i = 1; i < keys.length; i++) { const [t0, a0]
= keys[i - 1], [t1, a1] = keys[i]; if (t <= t1) return a0 + (a1 - a0) * ease((t - t0) / Math.max(1, t1 - t0)); } return keys[keys.leng
th - 1][1]; };
 334   const bad = [];
 335   for (const fps of [30, 15]) for (const p1 of [0.6, 0.8]) for (const d of [0.14, 0.22]) for (const x of [0.3, 0.34, 0.38]) {
 336     let tStart = null, keys = null;
 337     const poseAt = (t, a, phase) => {
 338       if (phase !== "running" && tStart === null) return [skeleton(a)];
 339       if (tStart === null) {
 340         tStart = t; const E = tStart + 30000; keys = [[tStart, 0]];
 341         let tt = tStart + 1000; for (let i = 0; i < 6; i++) { keys.push([tt, 0], [tt + 1000, 1], [tt + 1400, 1], [tt + 2400, 0]);
tt += 3000; }
 342         const f = (x - d) / (1 - d), u = Math.acos(1 - 2 * f) / Math.PI, t2 = E - u * 1600, tPeak = t2 - 1300, tS = tPeak - 1500;
 343         keys.push([tS, 0], [tPeak, p1], [t2, d], [t2 + 1600, 1], [t2 + 5000, 1]);
 344       }
 345       return [skeleton(prof(keys)(t))];
 346     };
 347     const r = run([...PRACTICE, ["sit", 40000]], { fps, poseAt });
 348     const over = r.expiry.hRaw > r.expiry.halfway, near = Math.abs(r.expiry.hRaw - r.expiry.halfway) <= 0.03;
 349     const truths = near ? [6, 7] : [6 + (over ? 1 : 0)];
 350     const ok = r.status === "measured" ? truths.includes(r.count) : truths.some((v) => r.countRange.min <= v && v <= r.countRang
e.max);
 351     const asksFullAboutOldTop = r.expiry.decision === "closed_descending_rise" && over && !near;
 352     if (!ok || asksFullAboutOldTop) bad.push(`fps ${fps} p1 ${p1} d ${d} x ${x}: ${r.status} ${JSON.stringify(r.countRange)} ${r.
expiry.decision} raw ${r.expiry.hRaw} half ${r.expiry.halfway}`);
 353   }
 354   assert.deepEqual(bad, []);
 355 });
 356
```

tests/test_strengthening_overnight.test.mjs

```
// Overnight test strengthening (independent test-quality audit, docs/OVERNIGHT_AUDIT_REPORT.md s2). Each block pins a
// contract rule that a plausible mutant could previously break without any test failing. Expected values come from
// docs/CONTRACTS_CP4.md (numeric bounds) and the independent v2 oracle fixture, not from implementation output.
import test from "node:test";
import assert from "node:assert/strict";
import { readFileSync } from "node:fs";
import { computeCoverage, COVERAGE_LIMITS } from "../app/src/core/coverage.js";
import { decide } from "../app/src/core/decision.js";
import { createChairStand } from "../app/src/core/chair_stand.js";
import { createEvidenceStore } from "../app/src/ui/evidence.js";
import { skeleton, motion, PRACTICE, PRACTICE_GO } from "../eval/synth.mjs";

const V2 = JSON.parse(readFileSync(new URL("./fixtures/decision_matrix_v2_expected.json", import.meta.url), "utf8"));

// ---------- the v2 oracle must actually be present (it was silently optional) ----------
test("the independent v2 decision oracle is present with all 69 cases", () => assert.equal(V2.length, 69));

// ---------- coverage.js is the code the app runs: cross-check it against the oracle's own coverage bits ----------
test("coverage.js agrees with the independent oracle's coverage on every oracle case that carries image times", () => {
  let n = 0; const diffs = [];
  for (const c of V2) {
    const o = c.session.evidence && c.session.evidence.coverage; if (!o || !o.imageTimes) continue; n++;
    const a = computeCoverage(c.session.result, o.imageTimes);
    if (!!a.reference.ok !== !!o.reference.ok) diffs.push(`${c.id} reference`);
    for (const k of Object.keys(o.events || {})) if (!!(a.events[k] || {}).ok !== !!o.events[k].ok) diffs.push(`${c.id} event ${k}`);
    for (const k of Object.keys(o.flags || {})) if (o.flags[k].ok !== null && !!(a.flags[k] || {}).ok !== !!o.flags[k].ok) diffs.push(`${c.id} flag ${k}`);
    if (!!a.endpoint.ok !== !!o.endpoint.ok) diffs.push(`${c.id} endpoint`);
    if (!!a.scan.ok !== !!o.scan.ok) diffs.push(`${c.id} scan`);
  }
  assert.ok(n >= 50, `only ${n} cases had image times`);
  assert.deepEqual(diffs, []);
});

// ---------- coverage bounds at their exact boundaries (CONTRACTS_CP4 s2) ----------
const R = (extra = {}) => ({ window: { tGo: 10000, tEnd: 40000 }, events: [], reviewFlags: [], flagWindows: {}, reference: { t0: 5000, t1: 5600 }, ...extra });
const dense = (a, b, step = 100) => { const t = []; for (let x = a; x <= b; x += step) t.push(x); return t; };
test("coverage: rise anchor image at exactly 250 ms is ok, 251 ms is not", () => {
  const r = R({ events: [{ n: 1, cls: "full", tFull: 20000, tRise: 19500, tSeat: 21000, flags: [] }] });
  assert.equal(COVERAGE_LIMITS.anchorMaxMs, 250);
  assert.equal(computeCoverage(r, [20250]).events[1].ok, true);
  assert.equal(computeCoverage(r, [20251]).events[1].ok, false);
});
test("coverage: practice-stand image inside the hold +-150 ms is ok, 151 ms outside is not", () => {
  assert.equal(computeCoverage(R(), [5750]).reference.ok, true);
  assert.equal(computeCoverage(R(), [5751]).reference.ok, false);
  assert.equal(computeCoverage(R(), [4849]).reference.ok, false);
});
test("coverage: whole-test gap of exactly 1000 ms is ok, 1001 ms is not, including at the start and end of the test", () => {
  assert.equal(computeCoverage(R(), dense(10000, 40000, 1000)).scan.ok, true);
  const t = dense(10000, 40000, 1000); t.splice(t.indexOf(25000), 1, 24999); t.splice(t.indexOf(26000), 1, 26000);   // 24999 -> 26000 = 1001
  assert.equal(computeCoverage(R(), t).scan.ok, false);
  assert.equal(computeCoverage(R(), dense(11100, 40000, 500)).scan.ok, false, "no image in the first 1.1 s");
  assert.equal(computeCoverage(R(), dense(10000, 38900, 500)).scan.ok, false, "no image in the last 1.1 s");
});
test("coverage: flag window needs images with gaps <= 500 ms across [t0-200, t1+200]", () => {
  const r = R({ reviewFlags: ["gap_during_movement"], flagWindows: { gap_during_movement: [[20000, 21000]] } });
  assert.equal(computeCoverage(r, [19800, 20300, 20800, 21200]).flags.gap_during_movement.ok, true);
  assert.equal(computeCoverage(r, [19800, 20301, 20802, 21200]).flags.gap_during_movement.ok, false);
  assert.equal(computeCoverage(r, dense(19800, 21200, 900)).flags.gap_during_movement.ok, false);
});
test("coverage: a session-wide flag (no window) needs the whole-test images", () => {
  const r = R({ reviewFlags: ["intermittent_tracking"] });
  assert.equal(computeCoverage(r, dense(10000, 40000, 500)).flags.intermittent_tracking.ok, true);
  assert.equal(computeCoverage(r, dense(10000, 40000, 500).filter((t) => t < 20000 || t > 22000)).flags.intermittent_tracking.ok, false);
});
test("coverage: endpoint image must be within 250 ms of 30 s (an image elsewhere does not count)", () => {
  const r = R({ events: [{ n: 1, cls: "expiry", tExpiry: 40000, tRise: 39500, flags: ["expiry_over_halfway"] }] });
  assert.equal(computeCoverage(r, [39750]).endpoint.ok, true);
  assert.equal(computeCoverage(r, [39749]).endpoint.ok, false);
  assert.equal(computeCoverage(r, [15000]).endpoint.ok, false);
  assert.equal(computeCoverage(r, [39750]).endpoint.required, true);
});

// ---------- identity is required for promoted (partial -> full) and uncertain rises, not only counted ones ----------
test("identity must be confirmed for a partial rise promoted to full (oracle case C07 variant)", () => {
  const base = V2.find((c) => /^C07/.test(c.id)); assert.ok(base, "C07 missing");
  const s = structuredClone(base.session);
  const promoted = s.result.events.filter((e) => e.cls !== "full" && s.evidence.eventResolutions[e.n] === "full").map((e) => e.n);
```

```
  assert.ok(promoted.length > 0);
  assert.equal(decide(s).reference.eligible, base.expect.referenceEligible, "unmodified oracle case");
  s.evidence.identityConfirmedFor = s.evidence.identityConfirmedFor.filter((n) => !promoted.includes(n));
  const d = decide(s);
  assert.equal(d.reference.eligible, false);
  assert.ok(/identity/i.test(d.reference.blockers.join(" ")), d.reference.blockers.join(" | "));
});

// ---------- reference acceptance void rules (CONTRACTS_CP4 s1) ----------
function practiceCore() {
  const M = motion([...PRACTICE, ["sit", 30000]]);
  const c = createChairStand({ width: 1280, height: 1280 }); let refId = null, t = 0;
  for (; t < PRACTICE_GO; t += 33) { c.push({ t, poses: [skeleton(M.alphaAt(t), { W: 1280, H: 1280 })] }); const pr = c.readiness().practice || {}; if (refId === null && pr.state === "detected") refId = pr.refId; }
  return { c, refId, t };
}
test("a new tracker lock (different person) voids an accepted reference", () => {
  const { c, refId, t: t0 } = practiceCore(); let t = t0;
  assert.ok(c.acceptReference({ refId, basis: "image" }));
  for (let k = 0; k < 30; k++, t += 33) c.push({ t, poses: [skeleton(0, { W: 1280, H: 1280 })] });
  assert.equal(c.readiness().ready, true);
  for (let k = 0; k < 120; k++, t += 33) c.push({ t, poses: [skeleton(0, { x0: 1000, W: 1280, H: 1280 })] });   // someone else, elsewhere
  const r = c.readiness();
  assert.equal(r.ready, false, JSON.stringify(r.reasons));
});
test("acceptReference refuses an unknown/old refId and an invalid basis", () => {
  const { c, refId } = practiceCore();
  assert.equal(c.acceptReference({ refId: refId + 1, basis: "image" }), false);
  assert.equal(c.acceptReference({ refId, basis: "trust_me" }), false);
  assert.equal(c.acceptReference({ refId, basis: "image" }), true);
});
test("the reference is the MOST extended still window of the practice stand, offered only after the stand ended (0.8.1)", () => {
  // sit, rise to 0.9 and hold, straighten to 1.0 and hold, sag back to 0.9 and hold, sit
  const c = createChairStand({ width: 1280, height: 1280 });
  const ease = (u) => 0.5 - 0.5 * Math.cos(Math.PI * Math.max(0, Math.min(1, u)));
  const a = (t) => t < 1500 ? 0 : t < 2300 ? 0.9 * ease((t - 1500) / 800) : t < 3500 ? 0.9 : t < 3800 ? 0.9 + 0.1 * ease((t - 3500) / 300) : t < 5000 ? 1
    : t < 5300 ? 1 - 0.1 * ease((t - 5000) / 300) : t < 6500 ? 0.9 : t < 7300 ? 0.9 * (1 - ease((t - 6500) / 800)) : 0;
  const upright = skeleton(1, { W: 1280, H: 1280 });
  let seenInProgress = false, detectedAt = null, acc = null;
  for (let t = 0; t < 9500; t += 33) {
    c.push({ t, poses: [skeleton(a(t), { W: 1280, H: 1280 })] });
    const pr = c.readiness().practice || {};
    if (pr.state === "in_progress") { seenInProgress = true; assert.equal(c.acceptReference({ refId: pr.refId, basis: "image" }), false, "must not accept mid-stand"); }
    if (pr.state === "detected" && detectedAt === null) { detectedAt = t; assert.ok(c.acceptReference({ refId: pr.refId, basis: "image" })); acc = c.readiness().practice; }
  }
  assert.ok(seenInProgress, "never in progress"); assert.ok(acc, "never offered");
  assert.ok(detectedAt >= 6500, `offered at ${detectedAt} ms, before the stand ended`);
  assert.ok(acc.knee >= 179, `reference knee ${acc.knee}: not the most extended window`);
  const after = c.readiness().practice;
  assert.equal(after.refId, acc.refId); assert.equal(after.state, "accepted"); assert.equal(after.knee, acc.knee); assert.equal(after.h, acc.h);
  void upright;
});

// ---------- posture warning at intermediate depths (data-derived 158/165 thresholds) ----------
function carelessRun(depth, view = "side") {
  const cyc = Array.from({ length: 5 }, () => [["rise", 800], ["stand", 400], ["descend", 800], ["sit", 400]]).flat();
  const M = motion([...PRACTICE, ...cyc, ["sit", 30000]]);
  const c = createChairStand({ width: 720, height: 1280 }); let started = false;
  for (let i = 0; i < 1300; i++) {
    const t = Math.round(i * 1000 / 30);
    if (c.phase === "calibrating" && t >= PRACTICE_GO && !started) {
      const pr = c.readiness().practice || {};
      if (pr.state === "detected") c.acceptReference({ refId: pr.refId, basis: "image" });
      if (c.readiness().ready) { c.start(t); started = true; }
    }
    const s = c.push({ t, poses: [skeleton(M.alphaAt(t) * depth, { view })] });
    if (s.result) return s.result;
  }
  return null;
}
for (const d of [0.7, 0.85]) test(`careless acceptance of a depth-${d} practice stand raises the posture warning`, () => {
  const r = carelessRun(d); assert.ok(r, "never started");
  assert.ok(r.reviewFlags.includes("reference_posture_warning"), r.reviewFlags.join());
  assert.notEqual(r.status, "measured");
});
test("careless acceptance of a near-full (0.95) reference is NOT caught (disclosed residual)", { todo: "below pose resolution; disclosed limitation" }, () => {
  const r = carelessRun(0.95); assert.ok(r.reviewFlags.includes("reference_posture_warning"));
});
test("a genuine full practice stand gives no posture warning", () => {
  const r = carelessRun(1); assert.ok(r); assert.ok(!r.reviewFlags.includes("reference_posture_warning")); assert.equal(r.status, "measured"); assert.equal(r.count, 5);
```

```
});

// ---------- evidence store lifecycle ----------
globalThis.URL.createObjectURL = globalThis.URL.createObjectURL || (() => "blob:x");
test("evidence: a callback arriving after finalize() timed out is discarded and counted as late", async () => {
  let pendingCb = null;
  const s = createEvidenceStore({ encode: (v, w, h, cb) => { pendingCb = cb; } });
  s.capture({ videoWidth: 10, videoHeight: 10 }, 1000, null);
  const st = await s.finalize(30);
  assert.equal(st.sealed, true); assert.equal(st.frames, 0);
  pendingCb({ size: 5 });
  assert.equal(s.stats().frames, 0); assert.equal(s.stats().lateOrStale, 1);
});
test("evidence: clear() revokes every object URL it created", () => {
  const revoked = []; const orig = URL.revokeObjectURL; URL.revokeObjectURL = (u) => revoked.push(u);
  let k = 0; const origC = URL.createObjectURL; URL.createObjectURL = () => `blob:${++k}`;
  try {
    const s = createEvidenceStore({ encode: (v, w, h, cb) => cb({ size: 1 }) });
    for (let t = 0; t < 1000; t += 200) s.capture({ videoWidth: 10, videoHeight: 10 }, t, null);
    const fs = s.times().map((t) => s.at(t)); fs.forEach((f) => s.url(f));
    assert.equal(s.stats().objectUrls, fs.length);
    s.clear();
    assert.equal(s.stats().objectUrls, 0);
    for (let i = 1; i <= fs.length; i++) assert.ok(revoked.includes(`blob:${i}`), `blob:${i} not revoked`);
  } finally { URL.revokeObjectURL = orig; URL.createObjectURL = origC; }
});
test("evidence: between() keeps the required anchor images when downsampling", () => {
  const s = createEvidenceStore({ encode: (v, w, h, cb) => cb({ size: 1 }), minIntervalMs: 0 });
  for (let t = 0; t <= 3000; t += 50) s.capture({ videoWidth: 10, videoHeight: 10 }, t, null);
  const anchors = [s.at(0), s.at(1350), s.at(3000)];
  const got = s.between(0, 3000, 5, anchors);
  assert.ok(got.length <= 5);
  for (const a of anchors) assert.ok(got.includes(a), `anchor ${a.t} dropped`);
});

// ---------- mutation re-run gaps (overnight, second mutation pass) ----------
function runPoses(poseFn, { fps = 30, extraMs = 30000 } = {}) {
  const cyc = Array.from({ length: 6 }, () => [["rise", 800], ["stand", 400], ["descend", 800], ["sit", 400]]).flat();
  const M = motion([...PRACTICE, ...cyc, ["sit", extraMs]]);
  const c = createChairStand({ width: 720, height: 1280 }); let started = false;
  for (let i = 0; ; i++) {
    const t = Math.round(i * 1000 / fps); if (t > M.totalMs) break;
    if (c.phase === "calibrating" && t >= PRACTICE_GO && !started) {
      const pr = c.readiness().practice || {};
      if (pr.state === "detected") c.acceptReference({ refId: pr.refId, basis: "image" });
      if (c.readiness().ready) { c.start(t); started = true; }
    }
    const s = c.push({ t, poses: poseFn(t, M.alphaAt(t), c.phase) });
    if (s.result) return s.result;
  }
  return null;
}
// (a synthetic trigger for identity_or_landmark_discontinuity was not found: large synthetic hip jumps are rejected by the
//  tracker first; this flag remains without an isolated test - recorded in docs/OVERNIGHT_ISSUES.json ON-15)
test("quality: another person overlapping the participant -> another_person_overlapped, never 'measured'", () => {
  const r = runPoses((t, a, ph) => (ph === "running" && t > 12000 && t < 13500 ? [skeleton(a), skeleton(1, { x0: 430 })] : [skele
ton(a)]));
  assert.ok(r); assert.ok(r.reviewFlags.includes("another_person_overlapped") || r.status === "withheld", `${r.status} ${r.review
Flags}`); assert.notEqual(r.status, "measured");
});
test("quality: missing observations beyond tolerance for > 20% of the test -> withheld (no automatic count)", () => {
  const r = runPoses((t, a, ph) => (ph === "running" && (t % 2000) < 550 ? [] : [skeleton(a)]));
  assert.ok(r); assert.equal(r.status, "withheld", `${r.status} ${r.withheldReasons}`); assert.equal(r.count, null);
});
test("decision wording: an unfinished session's title says NOT completed; an unchecked completed count says (unchecked)", () => {
  const base = { source: "camera", observer: { arms: "no", setup: "yes" }, participant: { ageText: "70", column: "men" }, evidenc
e: { available: false } };
  const ev = [{ n: 1, cls: "full", counted: true, flags: [] }];
  const stopped = decide({ ...base, result: { reason: "stopped", status: "stopped", window: { tGo: 0, tEnd: 30000, observedMs: 90
00 }, events: ev, reviewFlags: [] } });
  assert.match(stopped.title, /NOT a completed/);
  const done = decide({ ...base, result: { reason: "expired", status: "measured", window: { tGo: 0, tEnd: 30000, observedMs: 3000
0 }, events: ev, reviewFlags: [] } });
  assert.equal(done.value, null); assert.match(done.valueText, /unchecked/);
});

```

eval/synth.mjs

```
// Synthetic landmark streams with exact ground truth, for engineering tests of the chair-stand core.
// These are NOT evidence about real people: they exercise protocol rules, boundaries and failure handling.
//
// Geometry (pixels, side-ish view): ankle fixed, knee fixed, thigh rotates from horizontal (seated)
// to vertical (standing). alpha in [0,1] is rise progress; thigh angle = 90deg * alpha, so the
// normalised hip height h = 1.1 * sin(90deg * alpha) (seated ~0, standing 1.1, halfway at alpha = 1/3).

const SHIN = 200, THIGH = 220, TRUNK = 300;

export function rng(seed) {                // mulberry32
  let a = seed >>> 0;
  return () => { a = (a + 0x6d2b79f5) >>> 0; let t = a; t = Math.imul(t ^ (t >>> 15), t | 1); t ^= t + Math.imul(t ^ (t >>> 7), t | 61); return ((t ^ (t >>> 14)) >>> 0) / 4294967296; };
}
const gauss = (r) => Math.sqrt(-2 * Math.log(r() + 1e-12)) * Math.cos(2 * Math.PI * r());

/** Build a skeleton (33 landmarks, normalised) for rise progress alpha. */
export function skeleton(alpha, { x0 = 400, y0 = 1150, W = 720, H = 1280, arms = "crossed", legVis = 0.95, vis = 0.95, view = "side", leanExtra = 0 } = {}) {
  if (view === "frontal") return skeletonFrontal(alpha, { x0, y0, W, H, legVis, vis, leanExtra });
  const th = (Math.PI / 2) * Math.max(0, Math.min(1, alpha));
  const ankle = [x0, y0], knee = [x0, y0 - SHIN];
  const hip = [knee[0] - THIGH * Math.cos(th), knee[1] - THIGH * Math.sin(th)];
  const lean = (alpha > 0.02 && alpha < 0.98 ? 0.6 * Math.sin(Math.PI * alpha) : 0) + 0.1 * (1 - alpha) + leanExtra; // forward lean mid-rise (+ habitual stoop)
  const sh = [hip[0] + TRUNK * Math.sin(lean), hip[1] - TRUNK * Math.cos(lean)];
  const head = [sh[0] + 20, sh[1] - 90];
  const pts = new Array(33).fill(null).map(() => [...head]);
  const set = (i, p, v = vis) => { pts[i] = [p[0], p[1], v]; };
  for (let i = 0; i <= 10; i++) set(i, [head[0] + (i % 3) * 6, head[1] + Math.floor(i / 3) * 6]);
  set(11, [sh[0] - 12, sh[1]]); set(12, [sh[0] + 12, sh[1]]);
  const lw = arms === "crossed" ? [sh[0] + 20, sh[1] + 30] : [hip[0] - 10, hip[1] + 10];
  const rw = arms === "crossed" ? [sh[0] - 20, sh[1] + 30] : [hip[0] + 10, hip[1] + 10];
  set(13, [sh[0] + 30, sh[1] + 70]); set(14, [sh[0] - 30, sh[1] + 70]);
  set(15, lw); set(16, rw);
  for (const [i, p] of [[17, lw], [19, lw], [21, lw], [18, rw], [20, rw], [22, rw]]) set(i, p);
  set(23, [hip[0] - 10, hip[1]], legVis); set(24, [hip[0] + 10, hip[1]], legVis);
  set(25, [knee[0] - 10, knee[1]], legVis); set(26, [knee[0] + 10, knee[1]], legVis);
  set(27, [ankle[0] - 10, ankle[1]], legVis); set(28, [ankle[0] + 10, ankle[1]], legVis);
  for (const [i, p] of [[29, ankle], [30, ankle], [31, [ankle[0] + 40, ankle[1]]], [32, [ankle[0] + 40, ankle[1]]]]) set(i, p, legVis);
  return pts.map(([x, y, v]) => [x / W, y / H, 0, v ?? vis, 0.99]);
}


// Frontal view (camera faces the participant): the same sagittal motion, but joint x positions collapse per leg,
// so 2-D knee/hip angles stay ~straight whatever the flexion; only vertical distances change. Shoulders are wide.
function skeletonFrontal(alpha, { x0, y0, W, H, legVis, vis, leanExtra }) {
  const th = (Math.PI / 2) * Math.max(0, Math.min(1, alpha));
  const kneeY = y0 - SHIN, hipY = kneeY - THIGH * Math.sin(th);
  const lean = (alpha > 0.02 && alpha < 0.98 ? 0.6 * Math.sin(Math.PI * alpha) : 0) + 0.1 * (1 - alpha) + leanExtra;
  const shY = hipY - TRUNK * Math.cos(lean), headY = shY - 90;
  const pts = new Array(33).fill(null).map(() => [x0, headY, vis]);
  const set = (i, x, y, v = vis) => { pts[i] = [x, y, v]; };
  for (let i = 0; i <= 10; i++) set(i, x0 - 12 + (i % 3) * 12, headY + Math.floor(i / 3) * 6);
  set(11, x0 - 120, shY); set(12, x0 + 120, shY);
  set(13, x0 - 40, shY + 60); set(14, x0 + 40, shY + 60); set(15, x0 + 70, shY + 20); set(16, x0 - 70, shY + 20);
  for (const i of [17, 19, 21]) set(i, x0 + 70, shY + 20); for (const i of [18, 20, 22]) set(i, x0 - 70, shY + 20);
  set(23, x0 - 50, hipY, legVis); set(24, x0 + 50, hipY, legVis);
  set(25, x0 - 55, kneeY, legVis); set(26, x0 + 55, kneeY, legVis);
  set(27, x0 - 55, y0, legVis); set(28, x0 + 55, y0, legVis);
  set(29, x0 - 55, y0, legVis); set(30, x0 + 55, y0, legVis); set(31, x0 - 55, y0 + 10, legVis); set(32, x0 + 55, y0 + 10, legVis);
  return pts.map(([x, y, v]) => [x / W, y / H, 0, v, 0.99]);
}

/**
 * Motion script -> alpha(t). Segments: ["sit", ms] ["rise", ms] ["stand", ms] ["descend", ms]
 * ["partial", ms, peakAlpha] (up and back down without reaching standing).
 * Returns {alphaAt(t), stands:[t when alpha first reaches 1], partials:[...] }.
 */
export function motion(script) {
  const segs = []; let t = 0; let a = 0;
  for (const [type, dur, peak] of script) {
    const a0 = a; let a1 = a;
    if (type === "rise") a1 = 1; else if (type === "descend") a1 = 0; else if (type === "sit") a1 = 0; else if (type === "stand") a1 = 1;
    segs.push({ type, t0: t, t1: t + dur, a0: type === "sit" ? 0 : type === "stand" ? 1 : a0, a1, peak });
    t += dur; a = type === "partial" ? 0 : a1;
  }
  const ease = (u) => 0.5 - 0.5 * Math.cos(Math.PI * Math.max(0, Math.min(1, u)));
  const alphaAt = (tt) => {
    for (const s of segs) {
      if (tt < s.t0 || tt >= s.t1) continue;
```

```
      const u = (tt - s.t0) / (s.t1 - s.t0);
      if (s.type === "partial") return s.peak * Math.sin(Math.PI * u);
      return s.a0 + (s.a1 - s.a0) * ease(u);
    }
    return a;
  };
  const stands = segs.filter((s) => s.type === "rise").map((s) => s.t1);
  const partials = segs.filter((s) => s.type === "partial").map((s) => s.t0);
  return { alphaAt, stands, partials, totalMs: t };
}

/**
 * Render frames. opts: fps, durationMs, seed, noisePx, dropFrac (random missing poses),
 * gaps [[t0,t1]] (no frames at all: hidden tab / decoder stall), poseGaps [[t0,t1]] (frames but no pose),
 * legOcclusion [[t0,t1]], drift {dx, dy, scale0, scale1}, arms, helper {mode, ...}, swapFrac, visScale.
 */
export function render(script, opts = {}) {
  const { fps = 30, seed = 1, noisePx = 1.5, dropFrac = 0, gaps = [], poseGaps = [], legOcclusion = [],
    drift = null, arms = "crossed", helper = null, swapFrac = 0, W = 720, H = 1280, visScale = 1, view = "side", leanExtra = 0 }
= opts;
  const m = motion(script);
  const durationMs = opts.durationMs ?? m.totalMs;
  const r = rng(seed); const frames = [];
  const inAny = (t, list) => list.some(([a, b]) => t >= a && t < b);
  for (let i = 0; ; i++) {
    const t = Math.round(i * 1000 / fps);
    if (t > durationMs) break;
    if (inAny(t, gaps)) continue;
    const alpha = m.alphaAt(t);
    const u = t / Math.max(1, durationMs);
    let x0 = 400, y0 = 1150, sc = 1;
    if (drift) { x0 += drift.dx * u; y0 += drift.dy * u; sc = drift.scale0 + (drift.scale1 - drift.scale0) * u; }
    let lm = skeleton(alpha, { W, H, arms, x0, y0, view, leanExtra, legVis: inAny(t, legOcclusion) ? 0.2 : 0.95 * visScale, vis:
0.95 * visScale });
    if (sc !== 1) lm = lm.map(([x, y, z, v, p]) => [(x - x0 / W) * sc + x0 / W, (y - y0 / H) * sc + y0 / H, z, v, p]);
    lm = lm.map(([x, y, z, v, p]) => [x + (noisePx * gauss(r)) / W, y + (noisePx * gauss(r)) / H, z, v, p]);
    if (swapFrac && r() < swapFrac) { const k = lm[25]; lm[25] = lm[23]; lm[23] = k; const k2 = lm[26]; lm[26] = lm[24]; lm[24] =
k2; }
    const poses = [];
    if (!inAny(t, poseGaps) && !(dropFrac && r() < dropFrac)) poses.push(lm);
    if (helper) {
      const hx = helper.xAt ? helper.xAt(t) : helper.x;
      if (hx !== null && hx !== undefined) {
        const ha = helper.alphaAt ? helper.alphaAt(t) : helper.seated ? 0 : 1;   // seated helper = alpha 0 (fixes checkpoint-2 f
ixture that drew a standing helper)
        const hl = skeleton(ha, { W, H, x0: hx, y0: helper.y0 ?? 1150, view }).map(([x, y, z, v, p]) => [x + (noisePx * gauss(r))
/ W, y + (noisePx * gauss(r)) / H, z, v, p]);
        if (helper.first) poses.unshift(hl); else poses.push(hl);
      }
    }
    frames.push({ t, poses });
  }
  return { width: W, height: H, fps, frames, truth: m };
}

/**
 * CDC reference count for a synthetic script with Go at tGo: full stands reached in (tGo, tGo+30s],
 * plus one if at expiry the participant is on the way up and over halfway in hip height.
 */
export function referenceCount(m, tGo, windowMs = 30000) {
  const tEnd = tGo + windowMs;
  let n = m.stands.filter((t) => t > tGo && t <= tEnd).length;
  const a = m.alphaAt(tEnd), aPrev = m.alphaAt(tEnd - 100);
  const hNow = 1.1 * Math.sin((Math.PI / 2) * a);
  const rising = a > aPrev + 1e-6 && a < 1;
  if (rising && hNow > 0.55) n += 1;         // halfway between seated (~0) and standing (1.1)
  return { count: n, halfwayCase: rising ? (hNow > 0.55 ? "over" : "under") : "none", hAtExpiry: +hNow.toFixed(3) };
}

/**
 * Independent CDC reference for scripts (checkpoint 3). Counts rise segments that reach full extension
 * (alpha = 1) at or before expiry, plus the CDC position rule at expiry: an attempt that has left the seat,
 * has not yet reached full standing in this attempt, and whose hip is more than halfway (in hip height) between
 * seated and standing -> +1. No velocity condition. Cases within +/-nearBand of halfway are reported as "near".
 * A descending attempt that never reached standing is reported as "descending_uncounted_attempt" (the CDC text
 * states position only; tests accept either counting it or a review flag).
 */
export function referencePositionCount(script, tGo, windowMs = 30000, nearBand = 0.07) {
  const m = motion(script); const tEnd = tGo + windowMs;
  const n = m.stands.filter((t) => t > tGo && t <= tEnd).length;
  let t0 = 0, cur = null;
  for (const [type, dur, peak] of script) { if (tEnd >= t0 && tEnd < t0 + dur) cur = { type, t0, dur, peak }; t0 += dur; }
  const a = m.alphaAt(tEnd);
  const frac = Math.sin((Math.PI / 2) * a);               // hip-height fraction of the seated->standing displacement
  let kase = "none";
  if (cur && (cur.type === "rise" || cur.type === "partial")) {
    const u = (tEnd - cur.t0) / cur.dur;
```

```
    const descending = cur.type === "partial" && u > 0.5;
    if (Math.abs(frac - 0.5) <= nearBand) kase = "near";
    else if (frac > 0.5) kase = descending ? "descending_uncounted_attempt" : "over";
    else kase = "under";
  }
  return { count: n + (kase === "over" ? 1 : 0), fullStandsBeforeExpiry: n, case: kase, fracAtExpiry: +frac.toFixed(3) };
}

/** Standard practice-stand prefix used by checkpoint-3 tests: sit still, one full practice stand held 1 s, sit still. */
export const PRACTICE = [["sit", 1500], ["rise", 800], ["stand", 1000], ["descend", 800], ["sit", 1600]];
export const PRACTICE_GO = 5700;   // Go time (ms) right after the practice prefix

```

scripts/setup_vendor.sh

```
#!/usr/bin/env bash
# Fetch third-party runtime assets into app/vendor (not committed; not in the review ZIP).
#   @mediapipe/tasks-vision 1.0.1 (Apache-2.0 per its package.json) and the BlazePose "full" pose model
#   (Apache-2.0 per the BlazePose GHUM model card). After this, the app makes no third-party network requests.
set -euo pipefail
ROOT="$(cd "$(dirname "$0")/.." && pwd)"
V="$ROOT/app/vendor"; mkdir -p "$V/tasks-vision/wasm" "$V/models"
TMP="$(mktemp -d)"; trap 'rm -rf "$TMP"' EXIT
( cd "$TMP" && npm pack @mediapipe/tasks-vision@1.0.1 --silent >/dev/null && tar xzf mediapipe-tasks-vision-1.0.1.tgz )
cp "$TMP/package/vision_bundle.mjs" "$TMP/package/package.json" "$V/tasks-vision/"
cp "$TMP/package/wasm/vision_wasm_internal.js" "$TMP/package/wasm/vision_wasm_internal.wasm" \
   "$TMP/package/wasm/vision_wasm_nosimd_internal.js" "$TMP/package/wasm/vision_wasm_nosimd_internal.wasm" "$V/tasks-vision/was
m/"
MODEL="$V/models/pose_landmarker_full.task"
if [ ! -f "$MODEL" ]; then
  if [ -f "$ROOT/feasibility/models/pose_landmarker_full.task" ]; then cp "$ROOT/feasibility/models/pose_landmarker_full.task"
"$MODEL";
  else curl -fsSL -o "$MODEL" https://storage.googleapis.com/mediapipe-models/pose_landmarker/pose_landmarker_full/float16/lates
t/pose_landmarker_full.task; fi
fi
# Note (overnight check 2026-09-27): the pinned model bytes are served ONLY at the 'latest' URL (the versioned
# .../float16/1/ file has different bytes). If Google replaces 'latest', the hash check below fails loudly; the fix is
# to obtain the pinned file (md5 5a9ad889...) from a teammate's copy, not to accept new bytes silently.
# Verify every file against the COMMITTED expected hashes (scripts/vendor.sha256, pinned 2026-09-26 from the npm
# tarball @mediapipe/tasks-vision@1.0.1 and the float16 'latest' pose model). A mismatch aborts: the bytes are not
# the ones the tests and evidence were produced with.
( cd "$V" && shasum -a 256 -c "$ROOT/scripts/vendor.sha256" ) || { echo "VENDOR HASH MISMATCH - refusing to use these files"; exi
t 1; }
echo "vendor assets verified against scripts/vendor.sha256 in $V"

```